

DBARS

Demand Based Bus Allocation & Routing System

A transit planning, operations and open-data platform for BMTC, Bengaluru

PROJECT DEFENSE & CODEBASE MASTERY DOCUMENT

Attribute	Detail
Project name	DBARS - Demand Based Bus Allocation & Routing System
Repository	github.com/mbharathkumar234/DBARS---Demand-Based-Bus-Allocation-and-Routing-System-BMTC-
Backend	Python 3.12, FastAPI 0.115.6, Uvicorn, Pydantic 2.10.4, Motor 3.6 (MongoDB)
Frontend	React 19, TypeScript 5.7, Vite 6, Tailwind 3.4, Leaflet 1.9, Recharts 3
Database	MongoDB (17 collections) - optional; core routing runs without it
Mobile	Capacitor 8 (Android shell around the built web app)
Open data	GTFS static + GTFS-Realtime (protobuf) export
Deployment	Docker Compose (backend / frontend / mongo), Render, Vercel, Nginx
CI	GitHub Actions - pytest + compileall + frontend build on every push
Scale of code	~13,562 lines backend Python (65 files); ~8,459 lines frontend TS/TSX (45 files); 2,440 lines of tests (15 files); 82 REST endpoints
Test suite	161 backend tests, all passing
Document date	1 September 2026

How to Use This Document

This document is written for a reader who knows basic programming but has not memorised this codebase. Read it once end to end to understand the system, then use it as a reference when defending, debugging or extending the project.

- **Sections 1-6:** the system as a whole - what it does and how the parts fit together. Read these first.
- **Sections 7-12:** the code itself - files, entry point, execution traces, functions, classes, line by line.
- **Sections 13-25:** subsystem deep dives - API, database, auth, ML, errors, config, security, testing, deployment, patterns, algorithms.
- **Sections 26-33:** explanation and defense - the full story, professor questions, recruiter answers, diagrams to draw.
- **Sections 34-39:** practical mastery - debugging, modification, weaknesses, technical debt, code quality, master map.

Confidence labels used throughout

CONFIRMED - directly supported by a file in the repository, cited in an Evidence line. INFERRED - strongly implied by the implementation but not stated outright. UNCERTAIN - insufficient evidence; stated as such rather than guessed. Where something cannot be established from the code at all, this document says "Not determinable from the supplied code/project files."

An important honesty note about this project

DBARS is an academic / portfolio project, not a deployed BMTC system. Three things inside it are deliberately simulated, and are labelled as such in the code itself: live vehicle positions (a physics simulator, unless a real feed is configured), ticket payment (a transaction record is written; there is no payment gateway), and Shakti scheme eligibility (taken from the gender on the account; no government ID check). Claiming otherwise in a viva would be false. The code contains explicit guards that prevent simulated data being published as real - that is a strength to explain, not a weakness to hide.

Table of Contents

In Microsoft Word, click the field below and press F9 (or right-click > Update Field) to generate the contents with page numbers.

Right-click here and choose 'Update Field' to build the Table of Contents.

1. Executive Project Overview

1.1 Project Name

DBARS - Demand Based Bus Allocation & Routing System.

Evidence: README.md; backend/app/core/config.py -> Settings.app_name

1.2 One-Sentence Description

DBARS is a full-stack platform that turns BMTC's published bus timetable into three things at once: a journey planner for commuters, an operations planning tool for depot managers (how many buses and how much crew a timetable actually needs), and an open GTFS data feed that any external journey planner can consume.

1.3 Problem Being Solved

Bengaluru's bus network is large and hard to navigate, and the software around it addresses only one side of the problem. The project targets three gaps:

- **Commuters** cannot easily find which bus goes from stop A to stop B, especially when a transfer is needed. The dataset holds 3,940 distinct bus numbers across 4,883 stops.
- **The operator** gets nothing from a commuter app that answers operational questions: how few buses can run this timetable, how many kilometres are run empty, and how many crew duties are needed to staff it.
- **External journey planners** (Google Maps and similar) can only be as good as the data they are given; BMTC data published as GTFS improves all of them at once.

A fourth, narrower gap is addressed by the conductor module: the daily paperwork of a bus conductor - the waybill, cash reconciliation, and counting free travel under the Karnataka Shakti scheme for state reimbursement - is not modelled by any consumer app, because it is not a consumer problem. It is where the money is.

1.4 Proposed Solution

One system built on one shared dataset, serving four user roles from a single backend. The same 6,737 route-direction records power the commuter's journey search, the depot manager's fleet and crew plans, and the GTFS export - so the answers cannot disagree with one another.

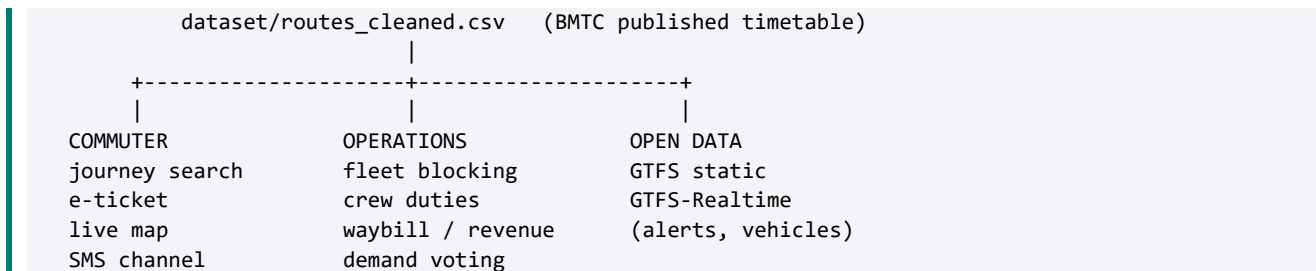


Figure 1.1 - One dataset, three products.

1.5 Target Users

Role	Who they are	What they do in DBARS	Evidence
------	--------------	-----------------------	----------

commuter	Ordinary bus passenger	Search journeys, vote for unserved routes, buy e-tickets, report crowding, share trips	auth/auth.py UserRole
conductor	Onboard staff who sell tickets	Sign on to a waybill, issue tickets offline, verify passes, sign off with cash	api/conductor.py
depot_manager	Runs one bus depot	Fleet blocking plan, crew duty plan, scenario consoles, waybill oversight	api/routes.py depot endpoints
admin	BMTC corporate / office user	Network dashboards, user management, Shakti claims, revenue reconciliation, audit log	api/admin.py
driver	Bus driver	Role exists and can be named on a waybill; no driver-facing screens yet	auth/auth.py UserRole.DRIVER

Finding - an incomplete role

UserRole.DRIVER was added so a waybill can record which driver was on the bus, but there is no driver-facing page, no driver sign-up flow and no driver endpoints. This is a declared gap, not something to hide - say so if asked. CONFIRMED: no route in frontend/src/App.tsx is gated to the driver role.

1.6 Core Features

#	Feature	What it does	Where it lives
1	Journey search	Direct and one-transfer routes between two stops, ranked, with distance, fare and metro interchange	ml/predictor.py
2	Demand voting	Commuters vote for origin-destination pairs; votes become depot dispatch recommendations	api/votes.py services/vote_service.py
3	Fleet blocking	Minimum buses and dead kilometres per depot, derived from the timetable alone	ml/blocking.py
4	Crew duty scheduling	Cuts vehicle blocks into legal crew duties; reports the crew-to-bus ratio	ml/crew.py
5	GTFS export	Full static feed plus realtime service alerts	app/gtfs/
6	Vehicle location (AVL)	Pluggable feed adapters, schedule adherence, route-following ETAs	app/tracking/
7	E-ticketing	Signed QR tokens, 6-character short codes, and Shakti free travel	api/tickets.py
8	Conductor waybill	Sign-on, offline ticket sync, cash reconciliation, Shakti claim reporting	api/conductor.py
9	Travel passes	Issued as signed tokens, verified offline against a revocation list	services/pass_service.py
10	Crowding reports	Commuter-reported occupancy with freshness and cooldown rules	services/crowding_service.py
11	Service alerts	Depot-published disruptions, also exported as GTFS-Realtime	api/alerts.py
12	SMS channel	Free-text journey queries by SMS, for feature phones	api/sms.py
13	Personal safety	Trusted contacts and expiring shared-trip links	services/safety_service.py
14	Metro integration	Nearest metro stations; interchange detection inside	services/metro_service.py

		a bus route	
15	Multilingual UI	English, Kannada, Hindi, Telugu	frontend/src/i18n/

1.7 Project Goals

1. Answer a commuter's "which bus do I take?" question from real BMTC data, including transfers.
2. Produce operational figures - fleet size, dead kilometres, crew duties - that no consumer app produces.
3. Close the loop between commuter demand (votes) and operator action (dispatch recommendations).
4. Publish the data in a standard format so the wider ecosystem benefits.
5. Model the money: fares, cash reconciliation and state reimbursement, correctly separated.
6. Never present simulated or modelled data as measured fact.

1.8 Real-World Use Case

A commuter at Majestic wants to reach Marathahalli. She opens DBARS, types both stops, and receives bus 335-G with the stop sequence, distance, estimated fare and a map of the route. Because her account records her gender as female, her e-ticket is issued free under the Karnataka Shakti scheme - with the fare value still recorded, so BMTC can claim it back from the state. If no bus served her pair, she could vote for it; enough votes surface that pair on the depot manager's dispatch screen. Meanwhile the depot manager, working from the same data, sees that the network timetable can be held by 6,849 interlined buses needing 15,636 daily crew duties.

1.9 What Makes the Project Technically Interesting

- **Two products from one dataset.** The same CSV answers a consumer question and an operations-research question. The fleet-blocking and crew-scheduling engines are genuinely uncommon in a student project.
- **A graph search, not a classifier.** Route prediction is an indexed search over 655,438 ordered stop pairs, not a trained model. The project benchmarks it against three alternative rankers and reports all four honestly.
- **Honesty enforced in code.** Simulated vehicle positions carry an `is_live=False` flag that gates the GTFS-Realtime endpoint with a hard 503. The system is built so that it cannot accidentally lie about the provenance of its data.
- **Offline-first money handling.** Conductor ticket sales use client-generated UUIDs and a unique database index, so a replayed offline batch cannot double-count revenue.
- **Degradation by design.** Route prediction, blocking, crew planning and GTFS export all work with MongoDB switched off; only the features that genuinely need a database report themselves unavailable.

1.10 Current Project Maturity

Classification: MVP / production-oriented prototype.

Signal	Evidence	Points toward
161 automated tests running in CI	backend/tests/, .github/workflows/ci.yml	Production-oriented
Containerised, three deploy targets	docker-compose.yml, render.yaml, vercel.json	Production-oriented
Role-based access control on writes	auth/auth.py -> require_role	Production-oriented
Graceful degradation without a database	db/database.py -> init_db docstring	Production-oriented
No payment gateway	api/tickets.py writes a transaction record only	Prototype

Vehicle tracking simulated by default	TRACKING_FEED defaults to "simulated"	Prototype
No frontend tests at all	frontend/package.json has no test script	Prototype
Shakti eligibility unverified	core/shakti.py -> DISCLOSURE	Prototype
In-memory, single-process rate limiter	core/rate_limit.py	Not production-ready

Reasoning: the engineering discipline is production-oriented - tests, CI, RBAC, containerisation and explicit failure modes. What keeps it short of production-ready is that three subsystems are deliberately simulated, there is no real payment or identity verification, and some infrastructure (rate limiting, in-memory state) assumes a single process. INFERRED from the combination of signals above.

2. Technology Stack

Every technology below was identified by reading the dependency files and source of this project, not assumed in advance. Unfamiliar terms are defined in plain language in the glossary in Section 31.

2.1 Backend

Layer	Technology	Purpose	Why it is used here	Evidence
Language	Python 3.12	Backend logic	Strong CSV and data handling; the routing and blocking maths is easiest to express here	ci.yml
Web framework	FastAPI 0.115.6	Serves the 82 REST endpoints	Async by default, automatic OpenAPI docs, and request validation straight from type hints	requirements.txt
Server	Uvicorn 0.34	ASGI server running FastAPI	Standard production runner for async Python	requirements.txt
Validation	Pydantic 2.10.4	Request/response models	Rejects malformed input before it can reach business logic	models/schemas.py
DB driver	Motor 3.6	Async MongoDB client	Non-blocking database access, so one slow query does not stall the event loop	db/database.py
Config	python-dotenv 1.0.1	Loads .env files	Keeps secrets out of source code	core/config.py
HTTP client	httpx 0.28.1	Outbound HTTP calls	Async client used for Google Maps and AVL feeds	ml/distance.py
Open data	gtfs-realtime-bindings 2.2.0	GTFS-RT protobuf	Official Google protobuf schema for transit realtime data	gtfs/realtime.py
Spreadsheets	openpyxl 3.1	Reads the depot workbook	The depot list ships as an .xlsx file	ml/blocking.py
Numerics	numpy, scipy, scikit-learn	TF-IDF baseline	Used only for the benchmark rankers, never for the shipped search	ml/tfidf.py
Testing	pytest 8.3.4, pytest-asyncio 0.24	161 tests	asyncio_mode = strict, so an async test cannot silently be skipped	pytest.ini

2.2 Frontend

Layer	Technology	Purpose	Why it is used here	Evidence
Language	TypeScript 5.7	Typed UI code	Catches shape mismatches between the API and the UI at compile time	tsconfig.json
Framework	React 19	Component UI	The component model suits many small panels sharing state	package.json
Build tool	Vite 6	Dev server and bundler	Fast rebuilds; produces the static bundle that Nginx and Capacitor serve	vite.config.ts
Routing	react-router-dom 7	Client-side pages	Role-gated routes through ProtectedRoute	src/App.tsx

Styling	Tailwind 3.4 plus styles.css	Visual design	Utility classes plus a hand-written design system of CSS variables	tailwind.config.ts
Maps	Leaflet 1.9 + react-leaflet 5	Route and live maps	Free and needs no API key, unlike Google Maps	components/RouteMap.tsx
Charts	Recharts 3.9	Dashboard graphs	Declarative charts for the depot and admin panels	pages/DepotDashboard.tsx
Animation	framer-motion 12	Page transitions	Declarative animation of panels and results	pages/PredictPage.tsx
QR codes	qrcode.react 4, html5-qrcode 2.3	Ticket QR and scanner	Generates the ticket QR, and reads it back on the conductor device	pages/Tickets.tsx
Icons, toasts	lucide-react, react-hot-toast	UI feedback	Consistent icon set and notification system	package.json
Mobile	Capacitor 8	Android wrapper	Packages the built web app into an APK	capacitor.config.ts

Finding - an unused dependency

@vis.gl/react-google-maps is declared in package.json, but every map component imports Leaflet instead. This looks like a leftover from an earlier approach. CONFIRMED: no file under frontend/src imports @vis.gl. Removing it would cut dead weight from the dependency tree. Note the bundle is already large at ~1.6 MB (468 KB gzipped).

2.3 Data, storage and infrastructure

Layer	Technology	Purpose	Why it is used here	Evidence
Primary data	CSV / JSON / XLSX in dataset/	The BMTC timetable	The engine loads from files, not a database, so it keeps working when Mongo is down	ml/data_loader.py
Database	MongoDB	Accounts, votes, tickets, waybills, alerts	Schema-free documents suited records whose shape evolved during development	db/database.py
Containers	Docker and Docker Compose	backend, frontend, mongo	One command brings up the entire stack	docker-compose.yml
Web server	Nginx	Serves the built frontend	Static file serving inside the frontend image	deployment/nginx.conf
Hosting	Render / Vercel	Deployment targets	render.yaml deploys both services; vercel.json hosts the frontend alone	deployment/render.yaml
CI	GitHub Actions	Tests on every push	Runs pytest, compileall and the frontend build	.github/workflows/ci.yml

2.4 What is NOT in the stack

Being explicit about absences is what prevents overclaiming in an interview.

- **No ORM.** MongoDB is accessed directly through Motor. There is no SQLAlchemy, Django ORM or Prisma anywhere.

- **No JWT library.** The JSON Web Token implementation is hand-written using hmac and hashlib in auth/auth.py.
- **No trained model file.** There are no .pkl or .h5 weights. scikit-learn appears only in a TF-IDF benchmark baseline.
- **No LLM in the routing engine.** The bus predictor contains no language model. A separate AI layer (Section 40) does use embeddings, a FAISS vector store and an optional Gemini call - keep the two apart. See Section 16.
- **No payment gateway.** No Razorpay, Stripe or UPI SDK is present.
- **No queue and no cache server.** No Redis, RabbitMQ or Celery. Expensive computations are cached in process memory instead.

3. Project Architecture

3.1 High-Level Architecture

DBARS is a client-server web application with an unusually heavy computational layer inside the server. The browser holds no business logic. Every decision - which bus, what fare, how many crew - is made in Python and sent to the interface as JSON.

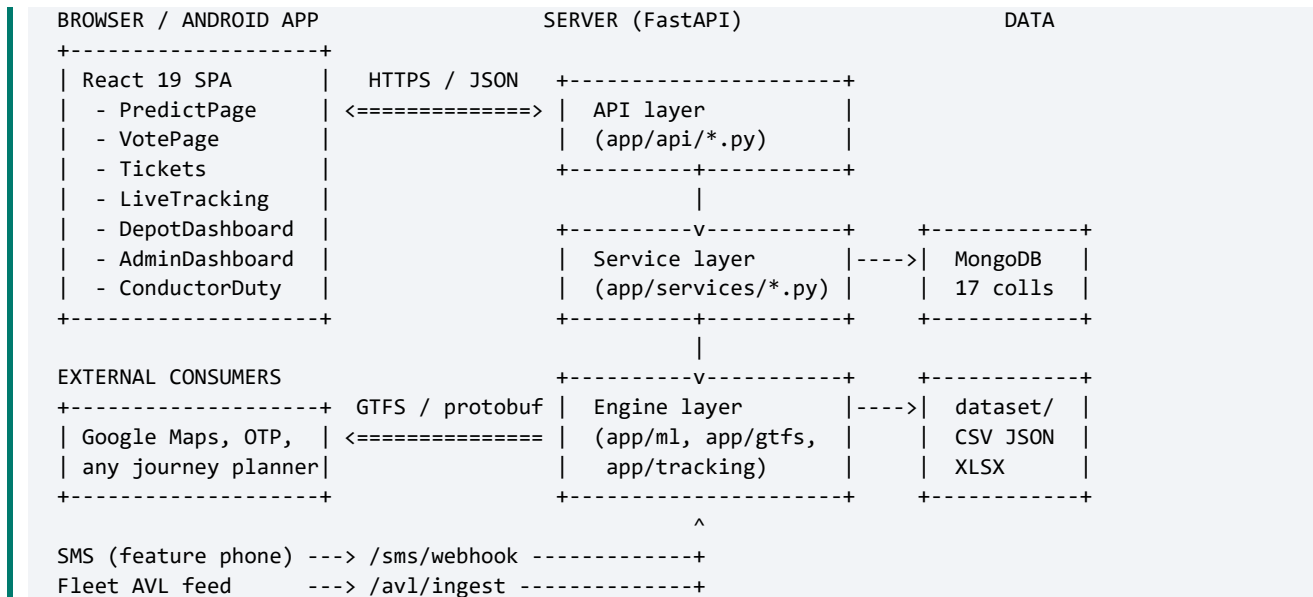


Figure 3.1 - High-level architecture. The engine layer reads the dataset directly and has no dependency on MongoDB.

3.2 Component Architecture - the four layers

The backend is organised in four layers. A request travels downward and a response travels back up. This is the single most likely architecture question in a viva, so learn this table.

Layer	Folder	Responsibility	May it touch the DB?
1. API	app/api/, app/auth/	HTTP concerns only: parse the request, check the role, call a service, shape the response	Only through services
2. Service	app/services/	Business rules and orchestration: what a vote means, what closing a waybill implies	Yes - this is the database layer
3. Engine	app/ml/, app/gtfs/, app/tracking/	Pure computation over the dataset: route search, blocking, crew, GTFS building, ETAs	No - deliberately file-only
4. Data	app/db/, dataset/	Connection handling, index creation, and the CSV, JSON and XLSX source files	Yes

The most important architectural decision in this project

The engine layer never touches MongoDB. Route prediction, fleet blocking, crew scheduling and GTFS export are pure functions of files that ship inside the repository. That is why the app still answers journey queries, and still shows a depot manager a full fleet plan, when the database is switched off - and why db/database.py can fail politely instead of taking the whole application down. CONFIRMED: services/blocking_service.py states this in its module docstring, and db/database.py

`init_db()` catches `PyMongoError` and returns.

3.3 Frontend Architecture

The frontend is a single-page application: React Router swaps pages without reloading the browser, and two React Contexts hold state that many pages need at once.

```
main.tsx  (mounts React into #root)
|
App.tsx   (Router + AuthProvider + LanguageProvider)
|
+-- ProtectedRoute (checks token, checks role, else redirects)
|   |
|   +-- /predict    PredictPage    -> RouteMap, NearestMetroPanel, SafetyPanel
|   +-- /vote       VotePage
|   +-- /track      LiveTracking
|   +-- /tickets    Tickets
|   +-- /depot      DepotDashboard -> BlockingPanel, ScenarioConsole, CrewPanel
|   +-- /admin      AdminDashboard -> MetricsDashboard, BlockingSummaryCard
|   +-- /duty       ConductorDuty  -> lib/offlineQueue.ts
|   +-- /scanner    ConductorScanner
|
+-- public routes: /  /login  /register  /trip/:code

Shared state: AuthContext    (user, token, isAdmin / isDepot / isConductor)
               LanguageContext (en / kn / hi / te, with English fallback)
Shared I/O:   lib/apiClient.ts -> apiFetch() wraps every call to the backend
```

Figure 3.2 - Frontend component tree and routing.

Evidence: `frontend/src/App.tsx` lines 50-77; `frontend/src/contexts/`

3.4 Backend Architecture - router registration

FastAPI composes the API from routers. Each router owns one subject area and is attached to the application inside `create_app()`.

Router	Prefix	Owns
<code>predict_router</code>	(none)	<code>/predict</code> , <code>/routes</code> , <code>/metrics</code> , <code>/autocomplete</code> , <code>/health</code> , <code>/train</code> , all <code>/depot/*</code> planning endpoints, <code>/tracking/source</code> , <code>/directions</code>
<code>auth_router</code>	<code>/auth</code>	<code>register</code> , <code>login</code> , <code>me</code> , <code>refresh</code>
<code>votes_router</code>	<code>/votes</code>	Demand voting, aggregation, allocation
<code>admin_router</code>	<code>/admin</code>	Dashboards, user management, fraud alerts, pilot metrics
<code>tracking_router</code>	<code>/tracking</code>	Vehicle positions, single bus, ETA, favourites, tick
<code>avl_router</code>	<code>/avl</code>	Pushed vehicle positions from a fleet system
<code>tickets_router</code>	<code>/api/tickets</code>	Fare quote, purchase, active, history, verify
<code>metro_router</code>	<code>/metro</code>	Metro stations, nearest station, map PDF
<code>crowding_router</code>	<code>/crowding</code>	Crowd reports and per-route crowding
<code>alerts_router</code>	<code>/alerts</code>	Service disruption alerts

sms_router	/sms	Twilio webhook and a JSON test twin
safety_router	/safety	Trusted contacts and expiring trip shares
gtfs_router, gtfs_realtime_router	/gtfs, /gtfs-rt	Static feed, feed info, rebuild; realtime alerts and vehicles
conductor_router, depot_router, office_router	/conductor, /depot, /admin	Waybills, onboard tickets, passes, Shakti claim, revenue reconciliation, audit log

Evidence: backend/app/main.py -> create_app()

3.5 Database Architecture

MongoDB is a document database. It stores JSON-like records called documents inside named groups called collections. There is no fixed table schema, so two documents in one collection may carry different fields. DBARS uses 17 collections; the field-level breakdown is in Section 14.

Crucially the database is optional. `init_db()` tries to create indexes; if MongoDB cannot be reached it sets an internal `_unavailable` flag, logs a warning and returns. Endpoints that genuinely need the database then raise a clear 503, while route prediction, blocking, crew planning and GTFS export continue unaffected.

Evidence: backend/app/db/database.py -> `init_db()`, `db_available()`, `get_db()`

3.6 External Services

Service	Status in the code	Purpose	Fallback when absent
Google Maps Distance Matrix	Optional; disabled by default (the API key is empty)	Road distance and duration between two stops	Straight-line distance from stop coordinates, x1.28 road factor
Twilio SMS	Webhook implemented; needs TWILIO_AUTH_TOKEN to accept anything	Receives SMS journey queries from feature phones	POST /sms/query exercises the same pipeline with no gateway
Fleet AVL feed	Adapter written; not connected to any real feed	Real vehicle positions	Physics simulator, flagged <code>is_live = False</code>
OpenStreetMap / CARTO tiles	Active	Map background tiles	None - the map would be blank

3.7 AI / ML Architecture

Two separate things live under this heading, and conflating them is the mistake to avoid. The ROUTING ENGINE - the thing that predicts buses - contains no artificial intelligence, no language model and no trained neural network. It is classical information retrieval and graph search: an inverted index, an ordered stop-pair index and a hand-weighted ranking function, benchmarked against three baselines.

A separate AI Intelligence Layer was added afterwards, under `backend/app/ai`. That one does contain machine-learning machinery: sentence-level retrieval over an AST-chunked code index and a documentation index, a FAISS vector store, BM25 fused with vector similarity by Reciprocal Rank Fusion, and an optional Gemini call. It answers questions ABOUT the system; it does not route buses, and the engine does not import it.

The honest framing, and the one to use in a viva: no model decides which bus you catch. Sections 40 to 42 cover the AI layer in full, including the measurement showing that only 1.8% of its benchmark answers actually reach a language model.

3.8 Authentication Architecture

```

Register / Login
| password hashed with PBKDF2-SHA256, per-user random salt
v
users collection ---> access token (24 h) + refresh token (7 d)
|
v
Browser stores both tokens in localStorage
|
| Authorization: Bearer <token> on every protected request
v
get_current_user() -> verify HMAC-SHA256 signature, check exp, check type == access
|
v
require_role(...) -> compare the role claim against the roles the endpoint allows

THREE SEPARATE SIGNING KEYS, so one stolen key cannot mint another kind of credential:
SECRET_KEY      -> session tokens      (type: access / refresh)
TICKET_SECRET_KEY -> e-ticket QR tokens (type: qr_ticket)
PASS_SECRET_KEY  -> travel pass tokens  (type: travel_pass)

```

Figure 3.3 - Authentication flow and the three token domains.

Evidence: backend/app/auth/auth.py; backend/app/auth/routes.py

3.9 Deployment Architecture

```

docker compose up --build
|
+-- mongo      : official image, port 27017, named volume for persistence
+-- backend    : python image, uvicorn on port 8000, mounts dataset/ read-only
+-- frontend   : node build stage -> nginx serving the bundle on port 8080

Alternative targets present in the repository:
deployment/render.yaml  two Render services (backend API + static frontend)
vercel.json              frontend only; VITE_API_URL points at the backend

```

Figure 3.4 - Deployment topology.

Evidence: docker-compose.yml; deployment/render.yaml; vercel.json; frontend/Dockerfile

4. Complete System Data Flow

This section traces what actually happens for each major action a user can perform. Only flows that exist in the code are included. Each flow names the real file and function at every step, so you can open the source and follow along.

4.1 The generic request lifecycle

```

User action in the browser
|
React event handler (e.g. handleSubmit in PredictPage.tsx)
|
apiFetch() (lib/apiClient.ts - prefixes the base URL, converts network errors)
|
HTTP request over the network
|
CORSMiddleware (outermost - allows the browser origin)
|
RateLimitMiddleware (per-IP sliding window, 120 requests/minute)
|
FastAPI router (matches the path to a function)
|
Pydantic validation (rejects a malformed body with 422 before any logic runs)
|
Depends(get_current_user) / Depends(require_role(...)) [protected endpoints only]
|
Endpoint function (app/api/*.py)
|
Service or engine (app/services/*.py or app/ml/*.py)
|
MongoDB or dataset files
|
Return value -> JSON serialisation -> HTTP response
|
React setState -> re-render -> user sees the result

```

Figure 4.1 - The path every request takes. Middleware order is set in `create_app()`.

Evidence: `backend/app/main.py -> create_app()`; `frontend/src/lib/apiClient.ts`

4.2 Flow A - Registration

```

User fills the form on /register (name, email, password, gender, language)
|
AuthContext.register() strips gender when it is the empty string, so the server
                        receives an absent field rather than an invalid enum value
|
POST /auth/register body validated by RegisterRequest (pydantic)
|
register() auth/routes.py
  |-- db.users.find_one({email}) -> 400 if the email already exists
  |-- hash_password(password) -> PBKDF2-SHA256, random 16-byte salt
  |-- db.users.insert_one({...role: 'commuter' forced...})
  |-- create_access_token(...) -> 24 hour token
  |-- create_refresh_token(...) -> 7 day token
  v
Response { access_token, refresh_token, user }

```

```
|
Browser stores both tokens in localStorage; setUser() re-renders; redirect /dashboard
```

Figure 4.2 - Registration. Note the role is forced to commuter server-side.

Why the role is forced

RegisterRequest deliberately has no role field, and register() hard-codes role = 'commuter'. If the role came from the request body, anyone could register themselves as an admin. Privilege must never be self-assigned by the client. CONFIRMED: auth/routes.py, comment "role intentionally absent".

4.3 Flow B - Login

```
POST /auth/login { email, password }
  |-- db.users.find_one({email})      -> 401 if missing
  |-- verify_password(plain, stored)  -> constant-time compare of PBKDF2 output
  |-- is_active check                 -> 403 if the account was disabled
  v
access_token + refresh_token + user profile (including shakti_eligible)
```

Figure 4.3 - Login.

4.4 Flow C - Journey search (the core feature)

This is the flow to know best. It is the feature the project is named after and the one an examiner is most likely to probe.

```
User types two stop names on /predict, assisted by AutocompleteInput
  |                                     (each keystroke -> GET /autocomplete?q=...)
  v
POST /predict { current_stop, destination, limit }
  |
predict() api/routes.py
  |-- asyncio.to_thread(predictor.predict, ...)  <-- CPU work moved OFF the
  |                                               event loop, ~1 second
  v
BMTCBusPredictor.predict() ml/predictor.py
  |-- _resolve_stop_name(origin)      fuzzy match to a real stop name
  |-- _resolve_stop_name(destination)
  |-- _find_transfer_suggestions()    the actual search:
  |   |-- stop_to_route_indices      which routes touch each stop
  |   |-- stop_pair_to_segments      655,438 ordered (A,B) pairs
  |   |-- build_direct_candidates, then one-transfer chains
  |   |-- _interchange_is_walkable()  reject transfers whose two stops
  |   |                               are more than 1.2 km apart
  |   |-- _make_transfer_leg() per leg -> distance, fare inputs, EV specs
  |   |-- _reanchor_legs()           resolve the whole journey as ONE
  |   |                               anchored sequence of stops
  |   +-- rank and deduplicate by bus chain
  |-- _detect_metro_interchange()     does the path pass a metro station?
  v
{ best_match, alternatives, message, resolved stop names, route_coordinates, legs }
  |
api/routes.py enriches with metro info, logs a predict_query usage event
  v
Frontend renders: bus number, stop list, distance, fare, RouteMap polyline per leg
```

Figure 4.4 - Journey search, end to end.

Evidence: backend/app/api/routes.py -> predict(); backend/app/ml/predictor.py

4.5 Flow D - Demand voting, and the loop back to the depot

This flow is the project name in action: commuter demand becoming an operator decision.

```

COMMUTER SIDE
POST /votes { current_stop, destination, time_preference }
  |-- VoteFraudDetector.check()  four heuristics:
  |   1. same pair more than 10 times in 24 h
  |   2. more than 30 votes in one hour from one user
  |   3. more than 50 votes in one hour from one IP
  |   4. an implausible spread of distinct destinations
  |-- db.votes.insert_one({... flagged: bool, reason })
  v
GET /votes/allocation -> which bus already serves that pair + how many votes it has

DEPOT SIDE
GET /depot/dispatch-recommendations (admin / depot_manager only)
  |-- get_vote_aggregation(days)      group votes by (origin, destination)
  |-- rank_dispatch_recommendations()  genuine votes = total minus flagged
  |   +--- predictor.predict(pair)    resolve each pair to a REAL bus
  v
Ranked list with surge level HIGH / MEDIUM / LOW, or an honest empty list

```

Figure 4.5 - The demand loop.

A design decision worth quoting in a viva

rank_dispatch_recommendations() is a separate, pure function rather than inline endpoint code, specifically so it can be unit-tested against fixture vote data with no MongoDB running. If it cannot resolve a voted pair to a real bus it skips the pair instead of inventing a route number. CONFIRMED: api/routes.py, and tests/test_votes.py.

4.6 Flow E - Buying an e-ticket (including Shakti free travel)

```

POST /api/tickets/purchase { source_stop, destination_stop, is_ac }
  |-- get_fare()                resolve both stops, compute distance,
  |                             look up the stage fare in fares.json
  |-- _account_gender(db, user_id) read gender from the ACCOUNT, never the token
  |-- is_shakti_eligible(gender, is_ac) core/shakti.py, the one shared rule
  |   female or transgender AND not AC -> free
  |   anything else                    -> chargeable
  |-- fare_value_inr = the full fare (what the state reimburses)
  |-- fare_collected_inr= 0 for Shakti, else the full fare
  |-- create_ticket_token(...)    signed with TICKET_SECRET_KEY, 2 h expiry
  |-- short_code                 6 random characters from A-Z0-9
  |-- db.tickets.insert_one(...)
  |-- db.transactions.insert_one(...) amount 0 for a scheme ticket
  v
Ticket rendered as a QR code plus the short code
|
CONDUCTOR SIDE: POST /api/tickets/verify { qr_token }
  |-- decode_ticket_token()      signature + expiry + type must be qr_ticket
  v
{ success: true, ticket_id, status }

```

Figure 4.6 - E-ticket purchase and verification.

The single most important idea in the ticketing code

fare_value_inr and fare_collected_inr are two separate fields. A Shakti ticket costs the passenger nothing and costs BMTC the full fare, which the state reimburses. One combined fare_amount field cannot express both, so revenue reporting and the reimbursement claim would misstate one or the other. Revenue uses fare_collected_inr; the claim uses fare_value_inr. CONFIRMED: api/tickets.py and services/waybill_service.py.

4.7 Flow F - Conductor waybill, offline sales and cash reconciliation

```

POST /conductor/sign-on { route_number, bus_reg, depot, opening_km }
  |-- refuse if this conductor already has an OPEN waybill -> 409
  |-- create waybill WB-YYYYMMDD-XXXXXXX, status OPEN
  |-- audit_service.record('waybill.sign_on')
  v
DURING THE SHIFT (signal is unreliable, so offline is the DEFAULT path)
issueTicket() on the device
  |-- enqueue() into localStorage with a client-generated UUID
  |-- if online, immediately flush()
  v
POST /conductor/tickets/sync { tickets: [...] }
  |-- for each ticket: price it with the SAME core/fares.py used by the app
  |-- insert into conductor_tickets
  |-- unique index (waybill_id, client_ticket_uuid) -> DuplicateKeyError on a
  |   replay, which is reported as 'duplicate', not an error
  v
{ accepted: [...], duplicates: [...], rejected: [...], counts }
  |-- the device clears ONLY what the server acknowledged

POST /conductor/sign-off { closing_km, declared_cash }
  |-- recompute_totals() derived from the ticket rows, never from the client
  |-- variance = declared_cash - expected_cash (reported, never auto-corrected)
  v
Waybill CLOSED; visible to the depot at GET /depot/waybills

```

Figure 4.7 - The conductor money path.

4.8 Flow G - GTFS export

```

Startup (or POST /gtfs/rebuild, or the first GET /gtfs/static.zip)
|
GtfsFeedService.ensure_built() asyncio.Lock so concurrent callers share one build
  |-- adopt an existing feed on disk if both the zip and its report are present
  |-- else asyncio.to_thread(_build_sync, ...)
  |
  GtfsFeedBuilder.build()
  |-- pass 1: resolve every route to anchored published stops
  |   |   exclude depot pull-out runs (325 dropped)
  |   |   exclude services over 100 km (55 dropped)
  |-- write agency, stops, routes, calendar, trips
  |-- STREAM stop_times straight into the zip (1,459,066 rows)
  |-- write shapes, feed_info, README.txt (provenance)
  +-- atomic rename of the temp file into place
  v
GET /gtfs/static.zip -> 14.4 MB feed, built in about 11 seconds

```

Figure 4.8 - GTFS static build.

4.9 Flow H - Vehicle positions and the honesty gate

```

Startup: vehicle_ingest.start() builds the adapter named by TRACKING_FEED
        simulated | gtfs_rt | http_json | push          (default: simulated)
        |
Background loop every TRACKING_POLL_SECONDS (default 10 s)
        |-- feed.poll() -> list[VehicleObservation] each carrying is_live
        |-- vehicle_store.replace_all(observations)
        |-- on failure: exponential backoff up to 300 s, error recorded
        v
GET /tracking/buses -> fresh observations only; anything older than
                     AVL_STALE_SECONDS (120 s) is WITHHELD, not shown
GET /tracking/eta   -> project the bus onto the route polyline, sum the REMAINING
                     distance along the route, return a range plus a confidence
                     label (high / medium / low) naming which speed source was used
GET /gtfs-rt/vehicle-positions.pb
        |-- if the active feed is NOT live -> HTTP 503 with an explanation
        +-- GTFS-Realtime has no field meaning 'this is simulated', so a consumer
            could not tell. The gate is the whole reason this layer exists.

```

Figure 4.9 - Vehicle location and the simulated-data gate.

4.10 Flow I - SMS journey query (feature phones)

```

Feature phone sends an SMS: 'Majestic to Whitefield'
        v
Twilio POSTs to /sms/webhook (form-encoded Body, From)
        |-- X-Twilio-Signature verified; with no TWILIO_AUTH_TOKEN set, everything
        |   is refused rather than trusted
        |-- parse_sms_query() splits on 'to', '->' or ',' but deliberately NOT '-',
        |   because real stop names contain hyphens
        |-- predictor.predict()
        |-- if stop-name confidence < 0.55 -> format_low_confidence_reply(), which
        |   says what it understood instead of stating a route as fact
        v
Twiml reply, truncated to 320 characters (about two SMS segments)

```

Figure 4.10 - The low-bandwidth channel.

5. User Journey

This section walks through the application screen by screen, in the order a new user would meet them. Every screen names the file that implements it.

5.1 Landing page (/) - pages/Landing.tsx

Aspect	What happens
Purpose	Introduce the project and route the visitor to sign in or sign up
User sees	Project name, description and call-to-action buttons
Interaction	Click Login or Register
Request	None - this page is static
Note	If already authenticated, App.tsx redirects straight to /dashboard

5.2 Register (/register) - pages/Register.tsx

Aspect	What happens
Purpose	Create a commuter account
User sees	Name, email, password, phone, gender and preferred language fields
Interaction	Fill and submit. Choosing Female or Transgender shows a green line confirming that ordinary (non-AC) tickets will be free under Shakti
Request	POST /auth/register
Backend	Duplicate email check, PBKDF2 password hash, insert with role commuter, mint access and refresh tokens
Database	db.users.insert_one(...)
Response	Tokens plus the user profile, including shakti_eligible
Afterwards	Tokens go to localStorage; the user lands on /dashboard

Why gender is collected here

Gender is a sensitive attribute, so the form states its single purpose at the point of asking: applying the Karnataka Shakti scheme, which gives free travel to women and transgender passengers. It is optional, "Prefer not to say" is a first-class answer, and it is used nowhere else in the codebase - not in prediction, not in analytics, not in any dashboard. It is also never placed in the JWT, because a bearer token travels through logs and browser storage. CONFIRMED: core/shakti.py; auth/routes.py; api/tickets.py reads it from the account with a database lookup.

5.3 Dashboard (/dashboard) - DashboardRouter

A dispatcher rather than a screen. It reads the role from AuthContext and renders the right dashboard: CommuterDashboard, DepotDashboard or AdminDashboard.

5.4 Journey search (/predict) - pages/PredictPage.tsx (861 lines)

Aspect	What happens
Purpose	The core feature: find the bus from A to B
User sees	Two autocomplete inputs, a voice button, a swap button, and an assistant panel that accepts free text such as "Majestic to Whitefield"
Interaction	Type or speak two stops and submit
Request	GET /autocomplete on each keystroke, then POST /predict
Backend	predictor.predict() on a worker thread; metro interchange enrichment; usage event logged
Response	best_match, alternatives, resolved stop names, coordinates, legs
Afterwards	Result card with bus number, confidence, distance, fare, stop list; RouteMap draws one coloured polyline per leg; service alerts for the route appear above; NearestMetroPanel and SafetyPanel appear below

5.5 Vote (/vote) - pages/VotePage.tsx (697 lines)

Aspect	What happens
Purpose	Register demand for a journey the network serves poorly
User sees	Origin and destination inputs, a time-of-day preference, and the current aggregated demand
Request	POST /votes, then GET /votes/allocation and GET /votes/aggregate
Backend	Fraud heuristics, insert, then resolve the pair to a real bus so the voter sees what already serves it
Afterwards	Vote count and the allocated bus, or an honest note if MongoDB is down

5.6 Live tracking (/track) - pages/LiveTracking.tsx

Aspect	What happens
Purpose	Show where buses are on a map
User sees	A Leaflet map with bus markers coloured by occupancy, a searchable list, and a detail panel with speed, occupancy and next stop
Request	GET /tracking/buses, polled
Backend	Reads the vehicle store; withholds observations older than 120 s; enriches each bus with its next stop by projecting it onto the route
Honesty	Every payload carries is_live. With the default simulator this is false, and mode is reported as "simulated"

5.7 Tickets (/tickets) - pages/Tickets.tsx

Aspect	What happens
Purpose	Buy and display an e-ticket
User sees	Two stop inputs, a fare quote, then a QR code with a 6-character code
Shakti path	An eligible passenger sees Rs 0 with the real fare struck through, and a "Get free Shakti ticket" button, BEFORE the payment step - never a surprise zero afterwards
Request	GET /api/tickets/fare, then POST /api/tickets/purchase
Afterwards	Active ticket with a live pulse indicator and an expiry countdown

5.8 Depot dashboard (/depot) - pages/DepotDashboard.tsx

The operations screen, and the one worth demonstrating to an examiner, because it shows work no consumer app does. It stacks four panels:

- **BlockingPanel.** Minimum fleet and dead kilometres per depot, with per-route re-block proposals a manager approves or rejects. Advisory only - nothing is dispatched.
- **ScenarioConsole.** "What would this frequency change cost?" - re-blocks one depot with and without a timetable change.
- **CrewPanel.** Daily crew duties, crew-to-bus ratio, split-duty percentage, unstaffable blocks, and a rule console for testing crew-agreement changes.
- **Dispatch recommendations.** The vote-driven list from Flow D, with a Deploy button that records a human acknowledgement.

5.9 Conductor duty (/duty) - pages/ConductorDuty.tsx

Aspect	What happens
Purpose	The conductor working day
User sees	Sign-on form; then waybill id, an issue-ticket form, and a sign-off form. An always-visible badge shows how many sales are still unsynced
Offline	Every sale is written to localStorage first and pushed opportunistically; the queue drains automatically when the browser reports it is online
Sign-off	Refuses to close while tickets are unsynced, because the server rejects tickets against a closed waybill

5.10 Conductor scanner (/scanner) - pages/ConductorScanner.tsx

Uses the device camera through html5-qrcode. A scanned token is inspected for its declared type - qr_ticket or travel_pass - and routed to the correct verifier, because both are three-segment signed tokens and cannot be told apart by shape alone. The signature itself is always checked on the server, with a different key per type.

5.11 Admin dashboard (/admin) - pages/AdminDashboard.tsx

Network-level view: user counts, vote analytics, fraud alerts, model metrics (MetricsDashboard), and the citywide fleet roll-up (BlockingSummaryCard).

5.12 Shared trip view (/trip/:code) - pages/SharedTripView.tsx

The only authenticated-user-generated page that a stranger can open without logging in. A commuter shares a trip; a trusted contact opens the link and sees the journey. The share expires after 6 hours, enforced by a MongoDB TTL index that physically deletes the document rather than merely hiding it.

Evidence: backend/app/services/safety_service.py; db/database.py trip_shares TTL index

6. Project Directory Structure

Reconstructed from the repository. Only folders that exist are listed. Build outputs and dependency folders (node_modules, .venv, dist, __pycache__) are excluded from version control by .gitignore and are not shown.

```

dbars/
|
+-- backend/                                Python API and computation engines
|   +-- app/
|   |   +-- main.py                        APPLICATION ENTRY POINT (create_app, lifespan)
|   |   +-- api/                          HTTP endpoints, one module per subject area
|   |   |   +-- routes.py                  predict, autocomplete, metrics, all depot planning
|   |   |   +-- votes.py                   demand voting and aggregation
|   |   |   +-- tickets.py                 e-ticket purchase and verification
|   |   |   +-- conductor.py               waybills, onboard sales, passes, revenue, audit
|   |   |   +-- tracking.py                vehicle positions, ETA, favourites, AVL ingest
|   |   |   +-- gtfs.py                   GTFS static and realtime endpoints
|   |   |   +-- admin.py                  admin dashboards and user management
|   |   |   +-- alerts.py                  crowding.py metro.py safety.py sms.py
|   |   +-- auth/                         authentication and authorisation
|   |   |   +-- auth.py                   tokens, hashing, roles, dependencies
|   |   |   +-- routes.py                 register, login, me, refresh
|   |   +-- core/                         cross-cutting configuration and rules
|   |   |   +-- config.py                 every environment variable, in one frozen dataclass
|   |   |   +-- fares.py                  BMTc stage fare calculation
|   |   |   +-- shakti.py                  the single shared Shakti eligibility rule
|   |   |   +-- rate_limit.py             per-IP sliding-window middleware
|   |   +-- db/
|   |   |   +-- database.py               Motor client, index creation, graceful degradation
|   |   |   +-- seed.py                   demo data seeding
|   |   +-- ml/                           THE ENGINES - pure computation, no database
|   |   |   +-- predictor.py              route search (1,600+ lines - the biggest file)
|   |   |   +-- blocking.py               vehicle blocking: fleet size and dead kilometres
|   |   |   +-- crew.py                   crew duty scheduling
|   |   |   +-- distance.py               stop coordinate resolution and distance
|   |   |   +-- route_geometry.py         anchored stop-sequence resolution (shared)
|   |   |   +-- data_loader.py            CSV -> RouteRecord
|   |   |   +-- text.py                   tfidf.py stop_registry.py
|   |   +-- gtfs/                         open-data export
|   |   |   +-- builder.py                 GTFS static feed builder (streaming)
|   |   |   +-- realtime.py               GTFS-Realtime protobuf feeds
|   |   +-- tracking/                     vehicle location layer
|   |   |   +-- base.py                   VehicleFeed protocol, VehicleObservation
|   |   |   +-- adapters.py               simulated / gtfs_rt / http_json / push
|   |   |   +-- store.py                  current state with staleness rules
|   |   |   +-- ingest.py                 the polling loop and health reporting
|   |   |   +-- matcher.py                projection, trip matching, ETA
|   |   +-- services/                     business logic and all database access
|   |   +-- models/                       pydantic request/response schemas
|   +-- tests/                            161 tests across 15 files
|   +-- artifacts/                        generated: metrics.json, gtfs/ (gitignored)
|   +-- scripts/                          blocking_report.py, offline analysis
|   +-- train.py                           standalone training entry point
|   +-- requirements.txt                   Dockerfile pytest.ini conftest.py
|
+-- frontend/                             React single-page application
|   +-- src/
|   |   +-- main.tsx                      React mount point
|   |   +-- App.tsx                       router, providers, role-gated routes

```

```

| | +-- pages/           one file per screen (13 pages)
| | +-- components/     reusable panels (17 components)
| | +-- contexts/       AuthContext, LanguageContext
| | +-- lib/            apiClient.ts, offlineQueue.ts
| | +-- services/       api.ts, ticketApi.ts
| | +-- hooks/  utils/  types/  i18n/  styles.css
| +-- android/          Capacitor Android shell
| +-- public/           static assets
| +-- package.json vite.config.ts tailwind.config.ts Dockerfile
|
+-- dataset/            THE SOURCE OF TRUTH
| +-- routes_cleaned.csv 6,737 route-directions (5.6 MB)
| +-- stops_cleaned.csv  9,507 stop rows, 4,883 unique names
| +-- bengaluru_metro_network.csv
| +-- fares.json / fares.csv stage fare table with confidence notes
| +-- BMTC_depot_place_zone.xlsx the depot list
| +-- Metro_Map_2025_-_Bengaluru_City.pdf
|
+-- deployment/        nginx.conf, render.yaml
+-- docs/              implementation-plan.md, resume-entry.md, this document
+-- .github/workflows/ci.yml continuous integration
+-- docker-compose.yml vercel.json README.md .gitignore

```

Figure 6.1 - Complete project structure.

6.2 Why the folders are split this way

Folder	Purpose	Depends on
backend/app/api/	Translate HTTP into function calls. No business rules live here.	auth, services, ml
backend/app/auth/	Decide who the caller is and what role they hold.	core, db
backend/app/core/	Configuration and rules shared by everything.	nothing in the app
backend/app/db/	Own the MongoDB connection and index definitions.	nothing in the app
backend/app/ml/	Compute answers from the dataset. Never imports services or api.	dataset files only
backend/app/gtfs/	Turn the dataset into standard open-data formats.	ml
backend/app/tracking/	Ingest and interpret vehicle positions.	ml, core
backend/app/services/	Business logic, and the only layer that reads and writes MongoDB.	db, ml, core
backend/app/models/	Define the shape of requests and responses.	nothing
frontend/src/pages/	One file per screen the user can navigate to.	components, contexts, lib
frontend/src/components/	Reusable panels used by several pages.	lib, contexts
frontend/src/lib/	Cross-cutting client utilities: HTTP wrapper, offline queue.	nothing

The dependency rule that keeps the design honest

Dependencies point in one direction only: api -> services -> db, and api -> ml -> dataset. The ml folder never imports from services or api. That is what makes the engines unit-testable without a database and without an HTTP server, and it is why 161 tests can run in CI with no MongoDB container. INFERRED from the import graph; CONFIRMED by the CI workflow, which runs pytest with no database service defined.

7. File-by-File Codebase Map

Files analysed in depth are listed first. Section 39 states which files were treated as lower priority.

7.1 Backend - entry point, core and data

File	Lang	Purpose	Important elements	Called by	Calls
app/main.py	Py	Entry point: builds the app, runs startup and shutdown	create_app(), lifespan(), _seed_default_admin()	uvicorn, tests	all routers, init_db, predictor.train
core/config.py	Py	Every environment variable in one frozen dataclass	Settings, settings	everything	os.getenv, dotenv
core/fares.py	Py	BMTC stage fare from distance	get_fare(), calculate_stage_fare(), _load_fare_table()	tickets.py, waybill_service	predictor, distance
core/shakti.py	Py	The single shared Shakti eligibility rule	Gender, is_shakti_eligible(), DISCLOSURE	tickets.py, waybill_service, auth/routes	nothing
core/rate_limit.py	Py	Per-IP sliding-window limiter	RateLimitMiddleware, EXEMPT_PATH_PREFIXES	main.py	nothing
db/database.py	Py	Mongo client, indexes, and graceful degradation	init_db(), get_db(), db_available()	every service	motor

7.2 Backend - the engines (app/ml, app/gtfs, app/tracking)

File	Lang	Purpose	Important elements	Called by	Calls
ml/predictor.py	Py	THE core engine: route search, ranking, transfers, benchmarks	BMTCTBusPredictor, predict(), _find_transfer_suggestions(), _interchange_is_walkable()	api/routes, votes, sms, tickets	distance, data_loader, route_geometry
ml/blocking.py	Py	Vehicle blocking: minimum fleet and dead kilometres	BlockingEngine, BlockingParameters, chain_blocks(), concurrency_lower_bound()	blocking_service, crew_service	distance, data_loader
ml/crew.py	Py	Crew duties cut from vehicle blocks	CrewParameters, cut_block_into_pieces(), combine_pieces_into_duties()	crew_service	blocking
ml/distance.py	Py	Stop name -> coordinate, and distance between stops	GoogleMapsDistanceService, StopRecord, resolve_stop_record()	predictor, blocking, gtfs	httpx (optional)
ml/route_geometry.py	Py	Anchored resolution of an ordered stop list	resolve_stop_sequence(), resolve_route_geometry()	predictor, gtfs builder, vehicle_service	distance

ml/data_loader.py	Py	CSV -> RouteRecord objects	RouteRecord, load_routes(), dataset_profile()	predictor, blocking, gtfs	csv, text
ml/text.py	Py	Stop name normalisation, fuzzy matching, mojibake repair	normalize_text(), fuzzy_ratio(), repair_route_text()	everywhere	difflib
gtfs/builder.py	Py	Streams the GTFS static feed into a zip	GtfsFeedBuilder, GtfsParameters, GtfsBuildReport	gtfs_service	route_geometry, blocking
gtfs/realtime.py	Py	GTFS-Realtime protobuf feeds	build_service_alerts_feed(), build_vehicle_positions_feed()	api/gtfs	gtfs_realtime_pb2
tracking/base.py	Py	The feed interface every adapter implements	VehicleFeed (Protocol), VehicleObservation	adapters, store, ingest	nothing
tracking/adapters.py	Py	Four vehicle feed adapters	SimulatedFeed, GtfsRealtimeFeed, HttpJsonFeed, PushIngestFeed, build_feed()	ingest	httpx, protobuf
tracking/store.py	Py	Latest position per vehicle, with staleness	VehicleStateStore, fresh(), apply()	ingest, vehicle_service	nothing
tracking/ingest.py	Py	The polling loop and honest source reporting	VehicleIngestService, source_report()	main.py, api/tracking	adapters, store
tracking/matcher.py	Py	Projection, trip matching, route-following ETA	project_onto_polyline(), match_trip(), estimate_arrival()	vehicle_service	blocking

7.3 Backend - API layer

File	Purpose	Important endpoints / elements
api/routes.py	The largest router: prediction plus all depot planning	predict(), autocomplete(), metrics(), health(), train(), depot_blocking_plan(), depot_crew_plan(), depot_crew_scenario(), rank_dispatch_recommendations()
api/votes.py	Demand voting	create_vote(), my_votes(), vote_allocation(), vote_aggregate()
api/tickets.py	Commuter e-tickets	get_estimated_fare(), purchase_ticket(), get_active_ticket(), verify_ticket(), _account_gender()
api/conductor.py	The money workflows	sign_on(), sync_tickets(), sign_off(), verify_pass(), issue_travel_pass(), shakti_claim_report(), revenue_report(), audit_log()
api/tracking.py	Vehicle positions and AVL ingest	get_all_buses(), get_bus(), get_eta(), ingest_positions(), favourites CRUD
api/gtfs.py	Open data	feed_info(), static_feed(), rebuild_feed(), service_alerts_protobuf(), vehicle_positions_protobuf()
api/admin.py	Administration	admin_dashboard(), list_users(), create_user_by_admin(),

		toggle_user_active(), vote_analytics(), fraud_alerts()
api/alerts.py	Service disruptions	list_active_alerts(), create_service_alert(), resolve_service_alert()
api/crowding.py	Crowdsourced occupancy	report_crowding(), route_crowding()
api/metro.py	Metro data	get_metro_stations(), get_nearest_metro_stations(), get_metro_map_pdf()
api/safety.py	Personal safety	list/update trusted contacts, share_trip(), view_shared_trip()
api/sms.py	Low-bandwidth channel	sms_webhook() (Twilio-signed), sms_query_test()
auth/routes.py	Identity	register(), login(), get_me(), refresh_token()
auth/auth.py	Security primitives	hash_password(), verify_password(), create_token(), decode_token(), get_current_user(), require_role(), UserRole

7.4 Backend - service layer (all database access)

File	Purpose	Key functions
services/vote_service.py	Voting rules and aggregation	VoteFraudDetector.check(), submit_vote(), get_vote_aggregation(), get_pair_demand()
services/waybill_service.py	The conductor money path	price_ticket(), open_waybill(), issue_tickets(), recompute_totals(), close_waybill(), shakti_claim(), revenue_reconciliation()
services/pass_service.py	Travel passes	issue_pass(), revoke_pass(), revocation_list(), verify_pass_token(), demo_verify()
services/blocking_service.py	Serves the fleet plan, cached	compute_plan(), run_scenario(), BlockingPlanService.get_plan()
services/crew_service.py	Serves the crew plan, cached	compute_crew_plan(), CrewPlanService.get_plan(), run_scenario()
services/gtfs_service.py	Builds and caches the feed	GtfsFeedService.ensure_built(), published_ids(), feed_info()
services/vehicle_service.py	Vehicle queries and ETA	describe_vehicles(), vehicle_trip_status(), estimate_eta(), RouteGeometryCache
services/tracking_service.py	The bus simulator	BusSimulator.tick(), initialize_buses()
services/crowding_service.py	Crowd reports	submit_crowd_report(), get_route_crowding()
services/alerts_service.py	Service alerts	create_alert(), get_active_alerts(), resolve_alert()
services/safety_service.py	Trusted contacts, trip shares	save_trusted_contacts(), create_trip_share(), get_shared_trip()
services/sms_service.py	SMS parsing and formatting	parse_sms_query(), format_sms_reply(), format_low_confidence_reply()
services/metro_service.py	Metro station lookup	metro_service singleton, nearest-station search
services/analytics_service.py	Privacy-minimal usage logging	log_event(), get_pilot_metrics()
services/audit_service.py	Accountability record	record(), recent()

7.5 Frontend

File	Purpose	Important elements
src/main.tsx	Mounts React into #root	ReactDOM.createRoot
src/App.tsx	Router, providers, role gating	ProtectedRoute, DashboardRouter, 13 routes
contexts/AuthContext.tsx	Auth state for the whole app	login(), register(), logout(), isAdmin, isDepot, isConductor, User type
contexts/LanguageContext.tsx	Four-language switching	t() with dot-path lookup and English fallback
lib/apiClient.ts	Single source of the backend URL	API_BASE_URL, apiFetch() - converts TypeError into a readable network message
lib/offlineQueue.ts	Offline conductor sales	enqueue(), flush(), pendingCount(), newTicketId()
pages/PredictPage.tsx	Journey search (861 lines)	handleSubmit, RouteStopsPanel, alerts, RouteMap
pages/VotePage.tsx	Demand voting (697 lines)	vote submission, aggregate display
pages/Tickets.tsx	E-ticket purchase and display	QR rendering, Shakti pre-purchase notice
pages/ConductorDuty.tsx	Conductor working day	signOn(), issueTicket(), syncNow(), signOff()
pages/LiveTracking.tsx	Vehicle map	marker colouring by occupancy, detail panel
pages/DepotDashboard.tsx	Operations screen	hosts BlockingPanel, ScenarioConsole, CrewPanel
components/RouteMap.tsx	Route map with per-leg colouring	MapBehaviour (invalidateSize, zoom handlers), createPortal for full screen
components/BlockingPanel.tsx	Fleet plan UI	depot selector, proposals, approve/reject
components/CrewPanel.tsx	Crew plan UI	duty metrics, crew rule scenario console
components/ScenarioConsole.tsx	Timetable what-if UI	headway / add / remove trips
components/AutocompleteInput.tsx	Stop name suggestions	debounced GET /autocomplete

8. Application Entry Point

Knowing exactly where and how the application starts is a standard viva question. For DBARS the answer has two halves: how the process is launched, and what happens inside `lifespan()` before the first request is served.

8.1 How the process is launched

```
Development:  python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
Docker:       the backend image runs the same uvicorn command
Tests:        with TestClient(app) - which runs the full lifespan too

'app.main:app' means: import the module app.main, take the object named app.
That object is created on the last line of main.py:  app = create_app()
```

Figure 8.1 - Launch paths.

8.2 `create_app()` - assembling the application

This function builds the FastAPI object but does not start anything. Order matters in two places, and both are deliberate.

7. `FastAPI(...)` is constructed with the title, version, description and - critically - the lifespan function.
8. `RateLimitMiddleware` is added FIRST, then `CORSMiddleware`. In Starlette the middleware added last sits outermost, so CORS wraps rate limiting. That ordering means a rejected 429 response still carries CORS headers, so the browser can read the error instead of reporting an opaque network failure.
9. `app.state.predictor = BMTCBusPredictor(...)` creates the engine object, but does not train it yet.
10. Fourteen routers are attached with `app.include_router(...)`.
11. The assembled app is returned.

Evidence: `backend/app/main.py -> create_app()`

8.3 `lifespan()` - the startup sequence, in order

`lifespan` is an async context manager. Everything before the `yield` runs once at startup; everything after it runs once at shutdown.

```
STARTUP
1. Verify the four required dataset files exist.
   A missing file raises FileNotFoundError and REFUSES to start. Deliberate:
   an app with no timetable cannot answer anything, so failing loudly at boot
   is better than failing mysteriously on every request.

2. app.state.predictor.train()
   Loads routes_cleaned.csv, builds every index, and either reads cached metrics
   from artifacts/metrics.json or re-evaluates the models. About 7 seconds.
   This is BLOCKING - the server does not accept traffic until it finishes.

3. Fire-and-forget: _warm_blocking_plan()
   The fleet plan takes ~25 s. Warming it in the background means the first depot
   manager to open the dashboard does not wait. Held on app.state, NOT in a local
   variable, because asyncio keeps only a weak reference to running tasks - a task
   with no strong reference can be garbage collected mid-flight.

4. Fire-and-forget: _warm_gtfs_feed() (if GTFS_BUILD_ON_STARTUP is true)
   Adopts a feed a previous run left on disk, or builds a new one in ~11 s.
```

```

5. await vehicle_ingest.start(predictor=...)
   Builds the adapter named by TRACKING_FEED and starts the polling loop.

6. await init_db()
   Creates every MongoDB index. If Mongo is unreachable this logs a warning and
   returns - it does NOT raise. Route prediction is already working by this point.

7. _seed_default_admin() only when SEED_DEFAULT_ADMIN=true, and it refuses to seed
   the password 'admin123' when ENV=production.

yield  <-- the application now serves requests

SHUTDOWN
8. await vehicle_ingest.stop()      cancel the poll loop, close the HTTP client
9. Cancel the two warm-up tasks if still running
10. await close_db()                close the Mongo connection pool

```

Figure 8.2 - The full startup and shutdown sequence.

The ordering decision that defines this project

Step 2 (train the predictor) runs BEFORE step 6 (connect to MongoDB), and step 6 cannot abort startup. That single ordering choice is what makes the core feature independent of the database. Reverse those two steps and an unreachable Mongo would take the whole application down with it. CONFIRMED: main.py, and the init_db() docstring explaining that this exact failure used to happen.

8.4 Configuration loading

core/config.py runs at import time, before anything else. It calls load_dotenv() twice - once for backend/.env and once for a workspace-level .env - then builds a single frozen Settings dataclass. Frozen means immutable: no code can change configuration at runtime, so behaviour cannot silently drift between requests.

Evidence: backend/app/core/config.py

9. Complete Request / Execution Traces

Each trace below names the real file, function and line-level behaviour at every step. These are the traces to rehearse before a viva.

9.1 Trace 1 - POST /predict (the core feature)

#	Where	What happens
1	PredictPage.tsx -> handleSubmit	Reads the two stop names from React state; guards against empty or identical stops
2	services/api.ts -> planRoute	Builds the JSON body; expands candidate stop-name splits
3	lib/apiClient.ts -> apiFetch	Prefixes API_BASE_URL; catches TypeError and rethrows it as "Can't reach the server"
4	network	POST /predict with a JSON body
5	CORSMiddleware	Confirms the browser origin is allowed
6	RateLimitMiddleware	Per-IP deque; over 120/minute returns 429
7	FastAPI router	Matches the path to predict() in api/routes.py
8	PredictRequest (pydantic)	Validates lengths and types; a bad body is 422 before any logic runs
9	api/routes.py -> predict()	Calls asyncio.to_thread(predictor.predict, ...) so ~1 s of CPU does not block the event loop
10	predictor._resolve_stop_name	Fuzzy-matches each typed name to a real stop, returning a confidence score
11	predictor._find_transfer_suggestions	Candidate generation from stop_to_route_indices and stop_pair_to_segments; direct routes first, then one-transfer chains
12	predictor._interchange_is_walkable	Rejects any transfer whose two stops are more than 1.2 km apart - the Avalahalli problem
13	predictor._make_transfer_leg	Per leg: stop path, distance, trip counts, EV estimate, metro interchange
14	predictor._reanchor_legs	Re-resolves the whole journey as ONE anchored sequence so the interchange lands on a single physical place
15	api/routes.py	Adds metro enrichment; logs a predict_query usage event (fire-and-forget)
16	FastAPI	Serialises the dict to JSON
17	PredictPage.tsx	setResult(...) triggers a re-render
18	components/RouteMap.tsx	Draws one coloured polyline per leg, with markers for board, change and alight

9.2 Trace 2 - POST /conductor/tickets/sync (offline sales)

#	Where	What happens
---	-------	--------------

1	ConductorDuty.tsx -> issueTicket	Writes the sale into localStorage FIRST, with a client-generated UUID
2	lib/offlineQueue.ts -> flush	Sends the whole queue as one batch
3	api/conductor.py -> sync_tickets	require_role(CONDUCTOR, ADMIN)
4	waybill_service.issue_tickets	For each ticket: check the waybill exists, belongs to this conductor, and is still OPEN
5	waybill_service.price_ticket	Confidence gate at 0.55 - refuses to price an unrecognised stop rather than guessing a cash fare
6	core/fares.py -> get_fare	The SAME pricing path the commuter e-ticket uses
7	MongoDB insert	Unique index (waybill_id, client_ticket_uuid). A replay raises DuplicateKeyError, caught and reported as 'duplicate'
8	response	Per-item accepted / duplicates / rejected lists
9	lib/offlineQueue.ts	Clears ONLY the items the server acknowledged; anything else stays queued

Why per-item results matter

If the client cleared its whole queue on a 200 response, any ticket the server rejected would be silently lost - and a lost cash sale is indistinguishable afterwards from one that never happened. Returning per-item outcomes is what makes the offline queue safe. CONFIRMED: services/waybill_service.py -> issue_tickets(); frontend/src/lib/offlineQueue.ts.

9.3 Trace 3 - GET /gtfs-rt/vehicle-positions.pb (the honesty gate)

#	Where	What happens
1	api/gtfs.py -> vehicle_positions_protobuf	Calls _vehicle_positions_response(allow_simulated=False)
2	tracking/ingest.py -> source_report()	Reports the active adapter and its is_live flag
3	the gate	If not is_live -> HTTP 503 with an explanation of why
4	tracking/store.py -> fresh()	Only observations newer than AVL_STALE_SECONDS
5	gtfs/realtime.py -> build_vehicle_positions_feed	Builds the protobuf; vehicle.timestamp is the SOURCE observation time, never the time of republication
6	response	application/x-protobuf, with X-Feed-Is-Live in the headers

With the default configuration this endpoint returns 503, and that is correct behaviour, not a bug. GTFS-Realtime has no field meaning "this data is simulated", so a consumer ingesting the feed could not tell the difference. Publishing simulator output here would be indistinguishable from claiming BMTC's fleet is being tracked.

10. Function-by-Function Analysis

The functions below are ordered by importance to the system, not by file order. Trivial helpers are omitted; the ones here are those an examiner is likely to ask about, or that you would need to understand before changing anything.

10.1 BMTCBusPredictor.predict()

predict(current_stop, destination, limit=50)

Aspect	Detail
File	backend/app/ml/predictor.py
Purpose	The single most important function in the project. Given two stop names typed by a human, return the best bus (or bus chain) that connects them, plus alternatives.
Parameters	current_stop (str) - what the user typed as their origin destination (str) - what the user typed as their destination limit (int) - how many alternatives to return
Returns	A dict with best_match, alternatives, message, resolved stop names, route_coordinates, and legs when the journey needs a transfer
Side effects	None. It only reads in-memory indexes built during train().
Dependencies	_resolve_stop_name, _find_transfer_suggestions, _detect_metro_interchange, GoogleMapsDistanceService
Called by	api/routes.py predict(); api/votes.py vote_allocation(); api/sms.py; rank_dispatch_recommendations()
Logic	1. Normalise and fuzzy-resolve both stop names to real stops, keeping a confidence score for each. 2. Reject the request if the two resolve to the same place. 3. Run _find_transfer_suggestions() to build direct and one-transfer journeys. 4. If the best option is direct, build best_match from that single route. 5. If the best option needs a transfer, build best_match from the chain, combining stop paths and coordinates across legs. 6. Attach metro interchange information if the path passes a metro station.
Beginner view	Think of two lookup tables built once at startup: "which buses stop here" and "which buses go from A to B in that order". The function looks up both, assembles candidate journeys, scores them, and returns the best.
Example in	predict("Majestic", "Marathahalli", 3)
Example out	best_match.bus_number = "335-G", 24 stops, distance_km 19.2, 0 transfers
Failure cases	Raises ValueError when a stop name cannot be resolved at all, or when origin and destination resolve to the same stop. api/routes.py converts that into a 422 response carrying the user's own words back to them.
Why it exists	It is the product. Everything else in the commuter half of the app is built around this one answer.

10.2 BMTCPredictor._interchange_is_walkable()

_interchange_is_walkable(path_tuples) -> bool

Aspect	Detail
File	backend/app/ml/predictor.py
Purpose	Reject a proposed transfer whose two buses do not actually meet in the physical world.
Parameters	path_tuples - the raw candidate journey as a list of (route, start_index, end_index) tuples, before any leg objects are built
Returns	True if every changeover is within MAX_INTERCHANGE_WALK_KM (1.2 km)
The problem	Transfers are found by matching stop NAMES, and Bengaluru reuses names. Banashankari -> Hebbal was answered with "take 215-K to Avalahalli, then 285-M from Avalahalli". Both stops are real and both are called Avalahalli - but they are 28.6 km apart. The journey is impossible.
Two ordering rules	It must run BEFORE _reanchor_legs(), which resolves a whole journey as one sequence and therefore forces the interchange onto a single point - checking afterwards would be asking whether two stops agree after making them agree. It must also run BEFORE _make_transfer_leg(), because building a leg costs a full distance walk and most rejects would be discarded immediately after.
Performance note	Uses _route_points(), a per-route-direction cache of anchored coordinates. Without it, this check turned one query from 1.2 s into 7.5 s.
Why it exists	It is a correctness guard, not a cosmetic one. Without it the app tells a commuter to change buses at a stop 28 km from where the second bus is.

10.3 resolve_stop_sequence()

resolve_stop_sequence(stops, distance_service) -> list[StopRecord | None]

Aspect	Detail
File	backend/app/ml/route_geometry.py
Purpose	Turn an ordered list of stop names into coordinates, each anchored to the previous one, so a route does not teleport across the city.
Returns	A list the SAME LENGTH as the input, with None where a name could not be resolved - so callers keep positional alignment with their stop names
Logic	Pass 1: resolve every name with no anchor, and take the median point as this sequence's rough centre. Pass 2: walk the list in order, resolving each name with near=<previous accepted point>, so each stop is chosen as the one nearest its neighbour rather than an arbitrary same-named place.
Why it matters	Resolving BMTC route 335-G's stops independently drew a 69.4 km line for a 19.2 km route: its "Kodihalli" matched the wrong Kodihalli 28 km east, so the path spiked out and back. The distances printed beside the map were always correct, because route_distance() already walked an anchor - only the drawing was wrong, which is exactly why the bug survived review for so long.
Called by	predictor._path_coordinates(), gtfs/builder.py, services/vehicle_service.py

Why shared	Three subsystems need the identical operation. Writing it three times is how they drift apart; two of the three had already drifted before it was extracted.
------------	--

10.4 cut_block_into_pieces()

cut_block_into_pieces(block, block_index, parameters) -> list[DutyPiece] | None

Aspect	Detail
File	backend/app/ml/crew.py
Purpose	Split one vehicle's whole day into the fewest legal pieces of crew work.
Parameters	block - a Block from the blocking engine (one bus's chain of trips) parameters - CrewParameters holding the crew-agreement assumptions
Returns	A list of DutyPiece, or None when no legal split exists at all
Algorithm	A dynamic program, not a greedy scan. best[i] = fewest pieces covering trips[i:]. For each start position it extends the piece while the working-time and continuous-driving limits still hold, considering only cut positions that are legal (a relief point, with at least min_relief_gap_minutes of turnaround).
Why not greedy	Greedy cutting - "run until the limit, then cut" - can be forced into an extra piece when a slightly earlier cut would have fitted the remainder. Blocks hold tens of trips, so the $O(n^2)$ dynamic program is effectively free.
Complexity	$O(n^2)$ in the number of trips per block; n is at most 16 in this dataset
Returns None when	A block offers no legal relief opportunity - for example six hours of trips running back to back with no gap anywhere. Those blocks are reported as unstaffable rather than silently dropped, because hiding them would make the duty count look better than the schedule really is.

10.5 combine_pieces_into_duties()

combine_pieces_into_duties(pieces, parameters) -> list[Duty]

Aspect	Detail
File	backend/app/ml/crew.py
Purpose	Pair up pieces of work that one person could do as a single duty, which is what an operator means by a split duty.
Algorithm	Every piece starts as its own duty. Processing in start-time order, each unpaired piece looks for the partner that uses the most of its remaining working-time budget, subject to the spreadover ceiling and no overlap.
Performance	Two devices keep this near-linear rather than quadratic: binary search to restrict candidates to the spreadover window, and a path-compressed next_free array so already-paired pieces are skipped in $O(1)$ instead of being re-scanned. Measured: 36 s before, 1.6 s after, on 21,453 pieces - and it produced slightly FEWER duties.
Why greedy is safe	Pairing only ever REDUCES the duty count, so a greedy result is a safe over-estimate of the crew required. Over-estimating crew is the right direction to be wrong in when the output is a staffing number.

10.6 is_shakti_eligible()

is_shakti_eligible(gender, *, is_ac=False) -> tuple[bool, str | None]

Aspect	Detail
File	backend/app/core/shakti.py
Purpose	The single shared rule deciding whether a journey travels free under the Karnataka Shakti scheme.
Returns	(eligible, reason_if_not). The reason is written to be shown to a passenger.
Rules	1. No gender on the account -> not eligible, with a reason explaining how to fix it. 2. Gender not in {female, transgender} -> not eligible, reason None. The scheme simply does not apply; returning an explanation would imply the passenger had been assessed and refused. 3. AC service -> not eligible, with a reason. The real scheme covers ordinary services only; Vajra and Vayu Vajra are chargeable for everyone.
Called by	api/tickets.py purchase_ticket(); services/waybill_service.py price_ticket(); auth/routes.py _format_user()
Why one function	The commuter app and the conductor both need this rule. Two implementations would eventually disagree, and a passenger told by the app that her journey is free and then charged on the bus has been failed by the software twice.

10.7 issue_tickets()

issue_tickets(conductor_id, tickets, request) -> dict

Aspect	Detail
File	backend/app/services/waybill_service.py
Purpose	Record a batch of onboard sales, idempotently, so an offline device can replay its queue safely.
Returns	Per-item lists: accepted, duplicates, rejected, plus counts
Idempotency	Each sale carries a client-generated UUID. A unique MongoDB index on (waybill_id, client_ticket_uuid) turns a replay into a DuplicateKeyError, which is caught and reported as 'duplicate' - the offline queue working correctly, not an error.
Per-item results	The device must clear only what the server acknowledged. Clearing the whole queue on a 200 response is exactly how offline sales vanish, and a lost cash sale is indistinguishable afterwards from one that never happened.
Rejections	An unknown waybill, another conductor's waybill, a CLOSED waybill (the normal case for a late offline arrival), or an unpriceable journey

10.8 close_waybill()

close_waybill(conductor_id, payload) -> dict

Aspect	Detail
File	backend/app/services/waybill_service.py
Purpose	End a duty: recompute the totals and compare declared cash against expected.
Key rule	Totals are recomputed from the ticket rows by recompute_totals() and are NEVER taken from the client. A client-supplied total in a revenue document is a fraud vector, and the device is a phone on a bus.
Variance	variance = declared_cash - expected_cash. It is reported and flagged, never auto-corrected. A system that quietly adjusts a cash figure so the books balance has destroyed the only signal the depot had.
Validation	Refuses if the waybill is not OPEN, belongs to someone else, or if the closing odometer is below the opening reading

10.9 estimate_arrival()

estimate_arrival(projection, target_distance_km, ...) -> EtaEstimate | None

Aspect	Detail
File	backend/app/tracking/matcher.py
Purpose	Work out when a bus will reach a stop, measured ALONG the route rather than in a straight line.
Returns	None if the target is behind the bus - a bus that has passed your stop is not arriving in negative minutes, and the caller should look for the next vehicle
Confidence	The label is derived from which speed source was available, not guessed: high - observed speed on this stretch of route medium - this vehicle's current reported speed low - the assumed average running speed The returned range widens as confidence falls (15%, 25%, 40%).
Why it replaced the old version	The previous estimate took a straight-line haversine distance, multiplied by 1.4 and divided by current speed. That is wrong in a specific and misleading way: a bus 400 m away across a block can be fifteen minutes from you if the route loops, and the old figure confidently said two.

10.10 Other important functions, in brief

Function	File	What it does and why it matters
train()	ml/predictor.py	Loads the CSV, builds every index, evaluates the models and writes metrics.json. Runs once at startup.
_build_stop_pair_index()	ml/predictor.py	Builds the 655,438-entry ordered stop-pair index. This is what makes the search fast enough to serve.
chain_blocks()	ml/blocking.py	Greedy earliest-available chaining of trips onto buses. Without an

		idle cap it provably matches the maximum-overlap lower bound.
concurrency_lower_bound()	ml/blocking.py	Peak simultaneous trips - a hard floor on fleet size. Lets the output say "provably minimal" rather than "best we found".
resolve_stop_record()	ml/distance.py	Resolves a stop name to a published stop row, using a near anchor to disambiguate repeated names.
get_fare()	core/fares.py	The single authoritative fare path, used by both the commuter e-ticket and the conductor sale.
hash_password() / verify_password()	auth/auth.py	PBKDF2-SHA256 with a per-user random salt; verification uses a constant-time comparison.
require_role()	auth/auth.py	A dependency factory - returns a dependency that enforces a set of allowed roles on an endpoint.
build_service_alerts_feed()	gtfs/realtime.py	Turns real depot alerts into GTFS-RT, checking every referenced id against the published static feed.
flush()	lib/offlineQueue.ts	Drains the offline sale queue and clears only acknowledged items.

11. Class-by-Class Analysis

A class is a template that bundles data (attributes) with the operations on that data (methods). An object is one instance made from that template. DBARS uses classes for three purposes: long-lived engines that hold expensive indexes, small immutable value records, and parameter bundles that make assumptions explicit.

11.1 BMTCBusPredictor

Aspect	Detail
File	backend/app/ml/predictor.py
Responsibility	Own the route dataset and every index built from it, and answer journey queries
Constructor	<code>__init__(dataset_path, artifact_dir)</code> - stores paths only; builds nothing
Key attributes	routes - list[RouteRecord] loaded from the CSV stop_names, destination_names - sorted name lists for autocomplete stop_to_route_indices - dict: normalised stop -> set of route indexes stop_pair_to_segments - dict: (A,B) -> route segments, 655,438 entries stop_registry - stable hashed IDs for stops distance_service - the coordinate and distance resolver _route_points_cache - anchored coordinates per route-direction metrics, profile - benchmark results and dataset statistics
Key methods	<code>train()</code> , <code>predict()</code> , <code>autocomplete()</code> , <code>list_routes()</code> , <code>_rank()</code> , <code>_find_transfer_suggestions()</code> , <code>_interchange_is_walkable()</code> , <code>_reanchor_legs()</code> , <code>_evaluate_models()</code> , <code>_cross_validate()</code>
Lifecycle	Constructed once in <code>create_app()</code> ; trained once in <code>lifespan()</code> ; then read-only and shared by every request through <code>app.state.predictor</code>
Why a class	It holds roughly 4 million indexed route segments. Rebuilding that per request would be impossible, so the state must live somewhere long-lived. A class instance stored on <code>app.state</code> is exactly that.
Thread safety	After <code>train()</code> the object is only read, which is why it is safe to call from <code>asyncio.to_thread()</code> . INFERRED - there is no lock, and no write path after training, so concurrent reads are safe.

11.2 BlockingEngine

Aspect	Detail
File	backend/app/ml/blocking.py
Responsibility	Answer the operations question: how few buses can hold this timetable, and how many kilometres are run empty
Constructor	<code>__init__(routes, depots, distance_service, parameters=None)</code>
Key attributes	routes, depots, parameters (BlockingParameters), _point_cache, _geometry_cache, _neighbour_cache - three caches, because the same geometry is consulted thousands of times
Key methods	<code>route_polyline()</code> , <code>route_length_km()</code> , <code>running_time_minutes()</code> , <code>assign_depot()</code> , <code>build_trips()</code> , <code>chain_blocks()</code> , <code>concurrency_lower_bound()</code> , <code>apply_dead_km()</code> , <code>block_by_route()</code> , <code>block_interlined()</code> , <code>revenue_service()</code>

Domain idea	A bus does not run "a route". It runs a BLOCK: it pulls out of its depot, runs a chain of trips back to back, and pulls back in. Two costs follow - peak vehicle requirement (which sizes the fleet) and dead kilometres (distance run with no passengers).
Why a class	It caches geometry and depot assignment across many queries, and the scenario console re-uses one engine for repeated what-if runs.

11.3 Parameter classes - BlockingParameters and CrewParameters

These two frozen dataclasses are the most quotable design decision in the project.

Class	Holds	Why it exists
BlockingParameters	average_speed_kmph (16.0), dwell_minutes_per_stop (0.3), layover_minutes (10), max_terminal_idle_minutes, max_deadhead_km (5.0), duty_limit_minutes (480), circuitry_factor (1.2)	Every one of these is an ASSUMPTION, not a BMTC measurement. Bundling them into one frozen object means they can be listed in the API response, varied in a scenario, and corrected without hunting through the code. Each field carries a docstring explaining what it means and how wrong it might be.
CrewParameters	sign_on_minutes (15), sign_off_minutes (15), max_continuous_driving (240), min_break_minutes (30), max_working_minutes (480), max_spreadover_minutes (720), min_relief_gap_minutes (10), allow_split_duties, relief_points	BMTC's real crew agreement is not in any published dataset, so all seven values are assumptions. The .assumptions property echoes them into every API response, so a reader can never mistake a modelled number for a measured one.

How to talk about this in a viva

If asked "how accurate is your fleet number?", the strong answer is not a number - it is this design. The honest version of the analysis is "here is the structure; please replace our four assumptions with your observed running times". Surfacing assumptions rather than burying them is what separates a model from a guess. CONFIRMED: ml/blocking.py module docstring says exactly this.

11.4 Value classes (small immutable records)

Class	File	Fields and purpose
RouteRecord	ml/data_loader.py	One route-direction from the CSV. __post_init__ precomputes stop_positions, is_self_loop and search_text once at load time - profiling showed 512,956 repeated normalize_text calls costing 7.0 s without it.
StopRecord	ml/distance.py	One physical stop: stop_id, stop_name, zone_id, lat, lon. Added so the GTFS exporter could reach the published stop_id, which resolve_coordinate() used to discard.
Trip	ml/blocking.py	One scheduled journey: route, direction, departure and arrival minute, origin, destination, revenue_km.

Block	ml/blocking.py	One physical bus's day. Properties compute revenue_km, dead_km, start, end and idle_minutes from its trips.
DutyPiece	ml/crew.py	An unbroken stretch of crew work - the atom of crew scheduling. It cannot be split further and cannot be shared.
Duty	ml/crew.py	One person's day: one piece, or two with an unpaid gap between them. Distinguishes working_minutes (paid) from spreadover_minutes (elapsed).
VehicleObservation	tracking/base.py	One sighting of one vehicle. Carries recorded_at (the SOURCE time, never receipt time) and is_live.
Projection	tracking/matcher.py	Where a vehicle sits along a route line, plus offset_km - how far it is from the line, which tells you whether to trust it.

11.5 Service classes (stateful, cached)

Class	Responsibility	Concurrency handling
BlockingPlanService	Computes the fleet plan once (~25 s) and caches it, plus a reusable engine context for scenarios	asyncio.Lock with a re-check inside, so queued callers take the first result rather than each starting a computation
CrewPlanService	Same pattern for the crew plan; reuses the blocking service's cached context rather than rebuilding	asyncio.Lock, same double-check
GtfsFeedService	Builds and caches the ~14 MB GTFS zip; can adopt a feed a previous process left on disk	asyncio.Lock; the build runs in a thread via asyncio.to_thread
VehicleStateStore	Latest observation per vehicle, with the staleness rule	Single-threaded access from the ingest loop and request handlers
VehicleIngestService	Owns the poll loop, backoff, and honest source reporting	asyncio.Event for cooperative shutdown
BusSimulator	The physics simulation behind simulated positions	asyncio.Lock so N concurrent viewers produce one shared timeline, not N stacked ones

11.6 VehicleFeed - an interface, not a class

VehicleFeed is a typing.Protocol: a description of the shape any vehicle feed must have (a name, an is_live flag, and start/stop/poll methods), without inheritance. Four adapters satisfy it - SimulatedFeed, GtfsRealtimeFeed, HttpJsonFeed and PushIngestFeed - and build_feed() returns whichever one TRACKING_FEED names.

This is the classic strategy pattern. Its value here is precise: the rest of the system depends on the interface, so swapping the simulator for a real BMTC feed is a configuration change rather than a code change - and every honesty gate in the codebase reads is_live from whichever adapter is active.

Evidence: backend/app/tracking/base.py; backend/app/tracking/adapters.py

12. Line-by-Line Code Explanation

This section walks through the code that matters most, in small logical blocks. Each block follows the same five levels: WHAT it does, HOW it does it, WHY it is written that way, how it CONNECTS to the rest of the system, and what the CONSEQUENCE of changing it would be.

12.1 Application assembly - main.py

```
app.add_middleware(
    RateLimitMiddleware,
    max_requests_per_minute=settings.rate_limit_per_minute,
    trust_proxy_headers=settings.trust_proxy_headers,
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=list(settings.cors_origins),
    allow_credentials="*" not in settings.cors_origins,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- **WHAT.** Attaches two pieces of middleware - code that sees every request before it reaches an endpoint.
- **HOW.** `add_middleware` wraps the application. In Starlette the middleware added LAST ends up OUTERMOST, so CORS wraps rate limiting.
- **WHY.** The ordering is deliberate and commented in the source. If rate limiting were outermost, a 429 response would be sent without CORS headers, and the browser would refuse to let JavaScript read it - the user would see an opaque network failure instead of "too many requests".
- **CONNECTION.** Every request in Section 4.1 passes through both of these before any endpoint runs.
- **CONSEQUENCE.** Swap the two lines and rate-limited responses become unreadable in the browser.

`allow_credentials` is computed rather than hard-coded: it is True only when the origin list is not a wildcard. Browsers refuse to send credentials to a wildcard origin, so setting both would be a silent misconfiguration.

12.2 The startup ordering that defines the project - main.py lifespan()

```
logger.info("Training BMTC predictor from %s", settings.dataset_path)
app.state.predictor.train()
...
await init_db()
if db_available():
    logger.info("MongoDB connection established and database initialized")
else:
    logger.warning(
        "Starting without MongoDB: account, voting, ticketing, and admin/depot "
        "features are unavailable until it is reachable. Route prediction is unaffected."
    )
```

- **WHAT.** Trains the route engine, then tries to connect to MongoDB.
- **HOW.** `train()` is synchronous and blocking - the server accepts no traffic until it finishes (~7 s). `init_db()` is awaited but cannot raise; it catches `PyMongoError` internally.
- **WHY.** This exact order is the architectural decision of the project. Route prediction is already working before the database is even attempted, so an unreachable database cannot take the core feature down.

- **CONNECTION.** It is why /predict, /depot/blocking-plan, /depot/crew-plan and /gtfs/static.zip all work with Mongo stopped, while /votes returns a clear 503.
- **CONSEQUENCE.** Reverse these two steps, or let init_db() raise, and a database outage becomes a total outage. The init_db docstring records that this used to happen.

Evidence: backend/app/main.py; backend/app/db/database.py -> init_db()

12.3 Password hashing - auth/auth.py

```
def hash_password(password: str) -> str:
    """Hash a password using PBKDF2-SHA256 (stdlib, no bcrypt needed)."""
    import os
    salt = os.urandom(16)
    ...
```

- **WHAT.** Converts a plain-text password into a value safe to store.
- **HOW.** PBKDF2-SHA256 with a fresh 16-byte random salt per user. Hashing is one-way: you cannot reverse the output back to the password.
- **WHY a salt.** Without one, two users who chose the same password would produce the same hash, and a precomputed rainbow table would crack both at once. A per-user random salt makes every hash unique.
- **WHY PBKDF2.** It is deliberately slow, so brute-forcing many guesses is expensive. It is in the Python standard library, so the project needs no bcrypt dependency.
- **CONNECTION.** Called by register() and by admin user creation; verify_password() is its counterpart at login.
- **CONSEQUENCE.** Store passwords in plain text, or hash with plain SHA-256 and no salt, and a database leak becomes an immediate account compromise for every user.

Honest assessment for a security question

PBKDF2-SHA256 with a random salt is a legitimate, standards-based choice and far better than plain hashing. It is not the strongest option available: bcrypt, scrypt and Argon2 are more resistant to GPU attack. If an examiner asks how you would harden it, that is the answer. Do not claim the scheme is unbreakable.

12.4 Token creation and the three domains - auth/auth.py

```
def _sign(payload: str, secret: str) -> str:
    return _b64_encode(
        hmac.new(secret.encode(), payload.encode(), hashlib.sha256).digest()
    )

def create_token(data, expires_seconds=ACCESS_TOKEN_EXPIRE_SECONDS, secret=None):
    header = _b64_encode(json.dumps({"alg": ALGORITHM, "typ": "JWT"}).encode())
    payload_data = {"**data", "exp": int(time.time()) + expires_seconds,
                    "iat": int(time.time())}
    payload = _b64_encode(json.dumps(payload_data).encode())
    signature = _sign(f"{header}.{payload}", secret or SECRET_KEY)
    return f"{header}.{payload}.{signature}"
```

- **WHAT.** Builds a JSON Web Token by hand: three base64url pieces joined by dots.
- **HOW.** The header and payload are readable by anyone - base64 is encoding, not encryption. The third piece is an HMAC-SHA256 signature over the first two, computed with a secret only the server knows.
- **WHY it is secure without hiding the contents.** The point is not secrecy, it is tamper-detection. Change the role claim from commuter to admin and the signature no longer matches, so decode_token() rejects it.

- **WHY a secret parameter.** The same function mints three different kinds of credential with three different keys, so a stolen ticket key cannot forge a session.
- **CONNECTION.** Every protected endpoint depends on `get_current_user()`, which calls `decode_token()`.
- **CONSEQUENCE.** Never put a password or anything sensitive in a token payload - it is publicly readable. This is exactly why the project does not put gender in the token and reads it from the account instead.

`decode_token()` verifies with `hmac.compare_digest`, a constant-time comparison. A normal `==` comparison returns faster when the first characters differ, and that timing difference can leak information about the correct value.

12.5 The secret-key decision - `auth/auth.py`

```
if settings.jwt_secret_key:
    SECRET_KEY = settings.jwt_secret_key
else:
    SECRET_KEY = secrets.token_hex(32)
    logger.warning(
        "JWT_SECRET_KEY is not set. Generated a random, process-local secret ..."
    )
```

- **WHAT.** Reads the signing key from configuration, or generates a random one.
- **WHY not a default constant.** This code previously contained a fixed literal committed to source control. Anyone who had ever seen the repository could forge a valid token for any user, including an admin, with no password. A secret readable in source control is not a secret.
- **WHY random rather than a second fixed fallback.** A random per-process key keeps local development working with zero configuration, while guaranteeing no guessable secret is ever baked in again.
- **The trade-off, stated honestly.** Tokens stop validating after a restart, and in a multi-worker deployment each worker would sign with a different key. That is the correct failure mode for an unconfigured environment, and it is why a loud warning is logged.
- **CONSEQUENCE.** In production `JWT_SECRET_KEY` must be set, or users are logged out on every restart.

12.6 The anchoring fix - `ml/route_geometry.py`

```
rough = [distance_service.resolve_stop_record(stop) for stop in stops]
known = [record.point for record in rough if record]
if not known:
    return [None] * len(stops)

centre = (
    sorted(point[0] for point in known)[len(known) // 2],
    sorted(point[1] for point in known)[len(known) // 2],
)

resolved = []
previous = centre
for stop in stops:
    record = distance_service.resolve_stop_record(stop, near=previous)
    resolved.append(record)
    if record is not None:
        previous = record.point
return resolved
```

- **WHAT.** Turns a list of stop names into coordinates without the route jumping across the city.

- **HOW - pass 1.** Resolve every name with no anchor and take the MEDIAN latitude and longitude. The median is used rather than the mean because one badly-resolved outlier 28 km away would drag a mean with it; a median ignores it.
- **HOW - pass 2.** Walk the list in order, resolving each stop with `near=<previous accepted point>`. Each stop is chosen as the instance nearest its neighbour.
- **WHY.** Bengaluru reuses stop names. Route 335-G drew a 69.4 km line for a 19.2 km journey because its "Kodihalli" matched the wrong Kodihalli 28 km east.
- **CONNECTION.** Three subsystems call this: the predictor's map coordinates, the GTFS exporter's stop_times, and vehicle projection for ETAs.
- **CONSEQUENCE.** Remove the anchoring and every route map, every GTFS shape and every ETA becomes wrong for any route touching a repeated stop name.

Note the return contract: the list is the SAME LENGTH as the input, with None where a name failed. Returning a shorter list would silently shift every later stop up by one position, corrupting the pairing of names to coordinates.

12.7 Idempotent offline sync - services/waybill_service.py

```
try:
    await db.conductor_tickets.insert_one(document)
except DuplicateKeyError:
    # The device replayed a sale the server already has. This is
    # the offline queue working correctly, not an error.
    duplicates.append({"client_ticket_uuid": ticket.client_ticket_uuid})
    continue
```

- **WHAT.** Inserts one onboard sale, treating a repeat as a duplicate rather than a failure.
- **HOW.** A unique index on (waybill_id, client_ticket_uuid) makes MongoDB itself reject the second insert. The database enforces the rule, not application logic - so a race between two concurrent sync requests cannot slip through.
- **WHY the UUID comes from the client.** The device must be able to retry without knowing whether the first attempt reached the server. A server-generated id cannot do that, because a retry would look like a brand-new sale.
- **CONNECTION.** lib/offlineQueue.ts replays the queue until acknowledged, and clears only settled items.
- **CONSEQUENCE.** Drop the unique index and every retry on a flaky bus connection inflates the day's revenue.

12.8 The honesty gate - api/gtfs.py

```
if not report["is_live"] and not allow_simulated:
    raise HTTPException(
        status_code=503,
        detail=(
            "Vehicle positions are currently produced by a simulation, not a BMTC "
            "vehicle feed, and are therefore not published as GTFS-realtime. ..."
        ),
    )
```

- **WHAT.** Refuses to publish vehicle positions as GTFS-Realtime while they come from the simulator.
- **HOW.** `is_live` is a property of the ACTIVE ADAPTER, not of the endpoint, so it travels with the data and cannot be overridden by the layer serving it.
- **WHY.** GTFS-Realtime has no field meaning "this data is simulated". A consumer ingesting the feed could not tell. Publishing simulator output here would be indistinguishable from claiming BMTC's fleet is being tracked.

- **CONNECTION.** The same flag drives GET /tracking/source and the is_live field on every /tracking/buses payload.
- **CONSEQUENCE.** Remove this check and the project becomes one that fabricates a government data feed. This single gate is the reason the whole adapter layer exists rather than a five-line wrapper over the simulator.

12.9 The staleness rule - tracking/store.py

```
def fresh(self) -> list[VehicleObservation]:
    now = datetime.now(timezone.utc)
    return [
        observation for observation in self._observations.values()
        if observation.age_seconds(now) <= self.stale_after_seconds
    ]
```

- **WHAT.** Returns only observations recent enough to be worth showing.
- **HOW.** age is computed from recorded_at, the time the SOURCE observed the vehicle - never the time the server received it.
- **WHY that distinction matters.** A feed that stalls keeps returning its last payload. If the timestamp were stamped on receipt, every stale position would look freshly observed, and the map would show a fleet of ghosts frozen mid-road that riders would stand and wait for.
- **CONNECTION.** Stale vehicles are still counted and surfaced in /tracking/source, so a stalled feed is visible rather than silently shrinking the fleet.
- **CONSEQUENCE.** Remove it and a broken upstream feed becomes indistinguishable from a working one.

12.10 Moving CPU work off the event loop - api/routes.py

```
# A real graph search over the stop index, ~1s of CPU. Off the event
# loop, or every other request queued behind it waits that long too.
prediction = await asyncio.to_thread(predictor.predict, current_stop, destination, 4)
```

- **WHAT.** Runs the route search on a worker thread instead of the main async loop.
- **HOW.** asyncio.to_thread hands the function to a thread pool and awaits the result, freeing the event loop meanwhile.
- **WHY.** FastAPI serves many requests concurrently on ONE thread. A synchronous function that takes a second blocks that thread completely - every other user waits a full second, not just the one who asked.
- **Beginner analogy.** The event loop is a single receptionist serving many visitors quickly. A one-second computation is a visitor who monopolises the desk. to_thread sends that visitor to a back office so the queue keeps moving.
- **CONSEQUENCE.** Call predictor.predict() directly in an async endpoint and throughput collapses under concurrent load.

13. API Documentation

DBARS exposes 82 routes. FastAPI generates live interactive documentation at /docs (Swagger UI) and /redoc, which is worth demonstrating in a viva. The tables below group the endpoints by subject. Auth column: PUBLIC = no token needed, USER = any logged-in account, and a named role = that role only.

13.1 Core prediction and data

Method	Endpoint	Purpose	Input	Auth	Output / errors
POST	/predict	Find the bus between two stops	current_stop, destination, limit	PUBLIC	best_match, alternatives 422 if a stop is unresolvable
GET	/autocomplete	Stop name suggestions	q, limit, kind	PUBLIC	Ranked name list
GET	/routes	Browse the route catalogue	limit, offset, search	PUBLIC	Paginated routes
GET	/metrics	Model benchmark results	-	PUBLIC	metrics.json contents
GET	/health	Liveness and readiness	-	PUBLIC	status, route count
POST	/train	Force a retrain	force	PUBLIC	New metrics (see the finding below)
POST	/directions	Google Maps directions	stops	PUBLIC	503 when no API key is set

Finding - an unauthenticated expensive endpoint

POST /train has no authentication and triggers a full retraining cycle. Repeated calls would consume significant CPU and could be used to degrade service. Every other state-changing or expensive endpoint in the project is role-gated, so this looks like an oversight rather than a decision. Recommended fix: wrap it in Depends(require_role(UserRole.ADMIN)), exactly as POST /gtfs/rebuild already is. CONFIRMED: api/routes.py train() has no Depends on a role.

13.2 Authentication

Method	Endpoint	Purpose	Input	Auth	Output / errors
POST	/auth/register	Create a commuter account	email, password, name, gender (optional)	PUBLIC	tokens + user 400 duplicate email
POST	/auth/login	Sign in	email, password	PUBLIC	tokens + user 401 bad credentials 403 disabled account
GET	/auth/me	Current profile	-	USER	user, incl. shakti_eligible
POST	/auth/refresh	New access token	refresh_token	PUBLIC	access_token 401 if not a refresh token

13.3 Voting and demand

Method	Endpoint	Purpose	Auth	Notes
POST	/votes	Record a demand vote	USER	Runs four fraud heuristics before inserting

GET	/votes/my	The caller's own votes	USER	Paginated
GET	/votes/allocation	Which bus serves my voted pair	USER	Bus resolves without a DB; vote counts need one
GET	/votes/aggregate	Demand by origin-destination pair	USER	Feeds the depot dispatch screen
GET	/depot/dispatch-recommendations	Ranked real demand	DEPOT, ADMIN	Returns an honest empty list when there are no votes, never invented recommendations
POST	/depot/deploy-bus	Acknowledge a recommendation	DEPOT, ADMIN	Records that a human reviewed it. Nothing is actually dispatched

13.4 Operations planning

Method	Endpoint	Purpose	Auth	Notes
GET	/depot/blocking-plan	Fleet size and dead kilometres	DEPOT, ADMIN	Cached; no database needed
POST	/depot/blocking-scenario	Cost of a timetable change	DEPOT, ADMIN	Scoped to one depot so it answers in under a second
POST	/depot/blocking-decision	Record an approve/reject	DEPOT, ADMIN	Advisory only
GET	/depot/crew-plan	Daily crew duties	DEPOT, ADMIN	Builds on the cached blocking context
POST	/depot/crew-scenario	Cost of a crew rule change	DEPOT, ADMIN	422 if working time exceeds spreadover
GET	/depot/routes	Route names for type-ahead	DEPOT, ADMIN	Supports the scenario console

13.5 Tickets, conductor and revenue

Method	Endpoint	Purpose	Auth	Notes
GET	/api/tickets/fare	Quote a fare	PUBLIC	Server-side pricing
POST	/api/tickets/purchase	Buy an e-ticket	USER	Free under Shakti when eligible
GET	/api/tickets/active	The current valid ticket	USER	404 when none
GET	/api/tickets/history	All past tickets	USER	Legacy tickets lack scheme fields
POST	/api/tickets/verify	Conductor scans a QR	CONDUCTOR, DEPOT, ADMIN	Verifies signature, expiry and type
POST	/conductor/sign-on	Open a waybill	CONDUCTOR, ADMIN	409 if one is already open
GET	/conductor/waybill	The open waybill	CONDUCTOR	Lets the device recover after a crash
POST	/conductor/tickets/issue	One onboard sale	CONDUCTOR	Shares its implementation with sync
POST	/conductor/tickets/sync	Drain the offline queue	CONDUCTOR	Idempotent; per-item results

POST	/conductor/sign-off	Close and reconcile cash	CONDUCTOR	Reports variance, never corrects it
POST	/conductor/verify-pass	Verify a travel pass	PUBLIC	Offline signature and expiry check
GET	/conductor/pass-revocations	Revocation list to cache	CONDUCTOR	Ids only - no personal data
POST	/depot/passes	Issue a signed pass	DEPOT, ADMIN	Returns the QR token
POST	/depot/passes/{id}/revoke	Revoke a pass	DEPOT, ADMIN	Audited
GET	/depot/waybills	Waybills for a depot/day	DEPOT, ADMIN	Surfaces open waybills and variances
GET	/admin/shakti/claim	Reimbursement claim	DEPOT, ADMIN	Splits conductor-issued from app-issued
GET	/admin/revenue/reconciliation	Cash vs digital vs scheme	DEPOT, ADMIN	Scheme value shown beside revenue, never inside it
GET	/admin/audit	Who did what, when	ADMIN	No TTL - accountability is not expired

13.6 Tracking, open data and the remaining endpoints

Method	Endpoint	Purpose	Auth	Notes
GET	/tracking/buses	Current vehicle positions	PUBLIC	Every payload carries is_live
GET	/tracking/bus/{n}	One route's vehicles	PUBLIC	Adds schedule adherence
GET	/tracking/eta	Arrival estimate	PUBLIC	Range plus a confidence label
GET	/tracking/source	What is powering the map	PUBLIC	Derived from the adapter, not a stored label
POST	/avl/ingest	Push vehicle positions	X-AVL-Key	Exempt from rate limiting; refuses everything when no key is configured
GET	/gtfs/static.zip	The full GTFS feed	PUBLIC	~14.4 MB, built in ~11 s
GET	/gtfs/feed-info	Build report and assumptions	PUBLIC	Answers without triggering a build
POST	/gtfs/rebuild	Force a rebuild	ADMIN	Correctly gated
GET	/gtfs-rt/service-alerts.pb	Realtime alerts	PUBLIC	Real data; 503 when Mongo is down
GET	/gtfs-rt/vehicle-positions.pb	Realtime vehicles	PUBLIC	503 while the feed is simulated
GET/POST	/alerts	List / create disruptions	PUBLIC / DEPOT, ADMIN	Creation is moderated
POST	/crowding/report	Report bus occupancy	USER	10-minute per-user cooldown
GET	/metro/nearest	Nearest metro stations	PUBLIC	Haversine over the metro CSV
POST	/sms/webhook	Twilio SMS query	Twilio signature	Refuses everything when no auth token is configured
POST	/safety/share-trip	Create a share link	USER	Expires after 6 hours
GET	/safety/trip/{code}	View a shared trip	PUBLIC	Deliberately public - a trusted contact has no account

13.7 How an endpoint is implemented, in order

12. Route registration - a decorator such as `@router.post("/predict")` binds the path to a function.

13. Request parsing - FastAPI reads the body and constructs the pydantic model named in the signature.
14. Validation - pydantic rejects wrong types or lengths with 422 before any project code runs.
15. Authorisation - Depends(get_current_user) or Depends(require_role(...)) resolves before the body executes.
16. Business logic - the endpoint calls a service or an engine. Heavy CPU work is wrapped in asyncio.to_thread.
17. Data access - services talk to MongoDB; engines read in-memory indexes.
18. Serialisation - the returned dict or response_model becomes JSON.
19. Error handling - HTTPException produces a clean status and detail; unexpected exceptions become a 500.

14. Database Deep Dive

14.1 What kind of database, and why

MongoDB, a document database. Instead of tables with fixed columns, it stores JSON-like documents inside collections. Two documents in one collection can hold different fields.

Aspect	Detail
Driver	Motor 3.6 - the async MongoDB driver, so a slow query does not block the event loop
Connection	MONGO_URI, default mongodb://localhost:27017; database name MONGO_DB_NAME, default "bmtc"
Timeout	serverSelectionTimeoutMS = 3000, deliberately reduced from the driver default of 30000 so a missing database fails fast instead of hanging the app for half a minute
Access pattern	Only the service layer touches the database. Engines never do.

Why a document database was a reasonable choice here

The honest answer is fit plus flexibility. Records like a waybill grew new fields during development (scheme, fare_value_inr, reconciliation) and a schema-free store absorbed that without migrations. The trade-off is real and worth stating: there are no foreign keys and no joins, so referential integrity is the application's responsibility. A relational database would have enforced more for you. INFERRED from the code; the project contains no written justification of the choice.

14.2 Collections

Collection	Holds	Key fields	Written by
users	Accounts	email (unique), name, password_hash, role, preferred_lang, gender, is_active, created_at	auth/routes.py, api/admin.py
votes	Demand votes	user_id, current_stop, destination, time_preference, flagged, reason, ip_address, created_at	vote_service.py
tickets	Commuter e-tickets	ticket_id, user_id, source_stop, destination_stop, fare_amount, fare_value_inr, fare_collected_inr, is_zero_fare, scheme, qr_token, short_code, status, expiry_time	api/tickets.py
transactions	Payment records	transaction_id, ticket_id, amount, status, scheme, timestamp	api/tickets.py
waybills	Conductor duty records	waybill_id (unique), conductor_id, driver_id, bus_reg, depot, route_number, duty_date, status, opening_km, closing_km, totals, reconciliation	waybill_service.py
conductor_tickets	Onboard sales	ticket_id, client_ticket_uuid, waybill_id, lines, fare_value_inr, fare_collected_inr, shakti_boardings, payment_mode, issued_at	waybill_service.py
travel_passes	Season passes	pass_id (unique), holder_name, pass_type, expires_at, status, revoked_at, revoked_reason	pass_service.py
audit_events	Accountability log	action, actor_id, subject, metadata, created_at	audit_service.py

service_alerts	Disruptions	title, description, severity, affected_routes, affected_stops, status, created_at	alerts_service.py
crowd_reports	Occupancy reports	user_id, route_number, crowding_level, stop_name, created_at	crowding_service.py
trip_shares	Shared trip links	code (unique), user_id, trip data, expires_at	safety_service.py
favorites	Saved routes	user_id, current_stop, destination, label	api/tracking.py
bus_positions	Simulated positions	bus_number, latitude, longitude, speed_kmh, occupancy_pct, updated_at	tracking_service.py
usage_events	Privacy-minimal analytics	event_type, metadata, created_at	analytics_service.py
predictions	Query history	user_id and query details	api/routes.py
notifications	User notifications	user_id, is_read	db/seed.py
verification_logs	Pass scan log	scan records	pass verification path

14.3 Indexes, and why each one exists

Index	Type	Why
users.email	unique	Prevents two accounts sharing an email, and makes login lookup fast
votes.(current_stop, destination)	compound	The aggregation groups by this pair
votes.created_at desc	single	Time-window queries for the fraud detector
favorites.(user_id, current_stop, destination)	unique	Stops the same route being saved twice
conductor_tickets.(waybill_id, client_ticket_uuid)	UNIQUE	THE index that makes offline ticketing safe. Without it a replayed batch is counted again and the day's revenue is overstated.
waybills.waybill_id	unique	Identity
waybills.(conductor_id, status)	compound	Finds a conductor's open waybill instantly
waybills.(depot, duty_date desc)	compound	The depot oversight screen
travel_passes.(status, expires_at)	compound	Builds the revocation list without a full scan
trip_shares.expires_at	TTL (0 s)	MongoDB physically DELETES an expired share. It does not merely stop returning it - someone's past travel plans do not linger as a forgotten record.
usage_events.created_at	TTL (90 days)	Raw analytics rows are not kept indefinitely
audit_events.created_at desc	single	Deliberately NO TTL - these record who moved money, and a retention window on them would be a retention window on accountability

14.4 How an application object becomes a stored record

1. HTTP JSON body

```
{ "source_stop": "Majestic", "destination_stop": "Marathahalli" }
```

```

|
2. Pydantic model TicketPurchaseRequest
   validated Python object with typed attributes
|
3. Endpoint builds a plain dict, adding server-computed fields
   ticket_id, fare_value_inr, fare_collected_inr, qr_token, short_code
|
4. await db.tickets.insert_one(ticket_doc)
   Motor serialises the dict to BSON; Mongo adds an _id
|
5. Reading back: db.tickets.find_one(...) returns a dict
|
6. TicketResponse(**ticket_doc) validates it on the way out
   _id is dropped; datetimes become ISO strings in JSON

```

Figure 14.1 - The object-to-document round trip. There is no ORM; the mapping is explicit dictionary construction.

A schema-evolution detail worth knowing

Every Shakti field on TicketResponse (fare_value_inr, is_zero_fare, scheme, scheme_note) is Optional with a default. Tickets issued before the scheme existed have none of those fields, and a required field would break every historical ticket in GET /api/tickets/history. This is the document-database trade-off made concrete: no migration was run, so the code must tolerate both shapes. CONFIRMED: backend/app/models/tickets.py.

14.5 A full read-write trace

```

Conductor taps 'Issue' on the device
-> localStorage queue (client-generated UUID)
-> POST /conductor/tickets/sync
-> issue_tickets()
   -> db.waybills.find_one({waybill_id})      READ - verify OPEN and owner
   -> price_ticket() -> get_fare()           READ - dataset files, not the DB
   -> db.conductor_tickets.insert_one(...)    WRITE - unique index enforced
-> per-item response

Conductor taps 'Sign off'
-> close_waybill()
   -> recompute_totals()
       -> db.conductor_tickets.find({waybill_id}) READ - cursor over all sales
-> db.waybills.update_one({...totals, reconciliation}) WRITE

Office opens the claim report
-> shakti_claim()
   -> db.waybills.aggregate([...])  READ - group by depot and duty_date
   -> db.tickets.aggregate([...])  READ - app-issued Shakti tickets, separately

```

Figure 14.2 - Reads and writes across one conductor shift.

14.6 Transactions and migrations

There are no multi-document transactions in this project, and no migration framework. CONFIRMED: no use of start_session or with_transaction appears anywhere. Both are genuine limitations and are listed in Section 36. The practical consequence: closing a waybill updates the waybill document after reading its tickets, and if the process died between those steps the waybill would remain OPEN with its tickets already recorded. Recovery would be manual.

15. Authentication & Authorization

Authentication answers "who are you?". Authorization answers "what are you allowed to do?". DBARS implements both, and it is worth being precise about how, because it is a favourite examiner topic.

15.1 The complete flow

REGISTRATION

```
password -> hash_password() -> PBKDF2-SHA256, 16-byte random salt
stored in users.password_hash; the plain password is never stored or logged
role forced to 'commuter' server-side
```

LOGIN

```
find user by email -> verify_password(plain, stored) -> constant-time compare
is_active check -> 403 if disabled
issue access token (24 h) + refresh token (7 d)
```

AUTHENTICATED REQUEST

```
Authorization: Bearer <access token>
get_current_user()
- split into header.payload.signature
- recompute the HMAC and compare with hmac.compare_digest
- check exp has not passed
- check type == 'access' <-- rejects a refresh or ticket token used as a session
returns the token claims: sub, email, role, name
```

AUTHORIZATION

```
require_role(UserRole.ADMIN, UserRole.DEPOT_MANAGER)
- a dependency FACTORY: returns a dependency that checks the role claim
- 403 when the role is not in the allowed set
```

LOGOUT

```
Client-side only: remove both tokens from localStorage.
There is no server-side token blacklist - see the finding below.
```

Figure 15.1 - Authentication and authorization end to end.

15.2 The three token domains

Token type	Key	Lifetime	Purpose
access	SECRET_KEY	24 hours	Session credential for API calls
refresh	SECRET_KEY	7 days	Obtain a new access token without re-entering a password
qr_ticket	TICKET_SECRET_KEY	2 hours	The e-ticket QR a conductor scans
travel_pass	PASS_SECRET_KEY	up to 400 days	A season pass, verifiable offline

The separation is deliberate and defensive. Each key is derived independently when not configured (for example `sha256("ticket-domain:" + SECRET_KEY)`), so a compromised ticket key cannot mint a session, and a pass key cached on a shared device in a bus - the key most likely to leak - cannot mint either.

The bug this design was built to prevent

The token type constants exist because they were once duplicated as bare strings across three files and drifted: `tickets.py` minted tokens typed `"qr_ticket"` while `get_current_user` rejected `"ticket"`. The guard was therefore dead code, and a ticket

QR token authenticated successfully as a user session. Constants in one place fixed it. CONFIRMED: the comment above TOKEN_TYPE_ACCESS in auth/auth.py.

15.3 Roles and what they unlock

Role	Can do	Cannot do
commuter	Search, vote, buy tickets, report crowding, share trips, save favourites	Any depot, admin or conductor endpoint
conductor	Waybills, onboard sales, sign-off, verify tickets and passes	Issue passes, view revenue reports, read the audit log
depot_manager	Blocking and crew plans, scenarios, alerts, waybill oversight, issue and revoke passes, Shakti and revenue reports	User management, the audit log
admin	Everything, including user management and the audit log	-
driver	Nothing yet - the role exists but no endpoint or page grants it anything	Everything

15.4 What is protected, and what is deliberately not

Public by design: /predict, /autocomplete, /routes, /metrics, /health, /gtfs/, /tracking/, /alerts (read), /metro/, and /safety/trip/{code}. The last is the interesting one - a trusted contact opening a shared trip link has no account, so requiring a login would defeat the feature. The link is unguessable, expires in six hours, and is then physically deleted by a TTL index.

15.5 Honest weaknesses in the auth design

- **No token revocation.** Logout only clears localStorage. A stolen access token stays valid for up to 24 hours; a refresh token for 7 days. There is no blacklist and no session store. CONFIRMED - no revocation logic exists.
- **Tokens in localStorage.** This is readable by any JavaScript running on the page, so a cross-site scripting flaw would expose it. An httpOnly cookie would be safer against XSS, at the cost of needing CSRF protection.
- **No rate limit on login specifically.** The global 120 requests/minute per IP applies, but there is no per-account lockout or exponential backoff, so password guessing is slowed rather than stopped.
- **No password complexity rule.** min_length=6 is the only constraint.
- **No email verification.** An account is usable immediately; the address is never confirmed.
- **PBKDF2 rather than Argon2/bcrypt.** Legitimate and standards-based, but not the strongest modern option.

None of these makes the project insecure for its purpose, but claiming it is "secure" without qualification would be an overclaim. The defensible statement is: it implements signed-token authentication with hashed passwords and role-based authorization, and has known limitations around revocation and token storage.

16. AI / ML / LLM Deep Dive

16.1 The honest headline

No model decides which bus you catch

The routing engine contains no large language model, no neural network, no trained model file and no inference API. CONFIRMED: no .pkl or .h5 weights, and no model-provider call on any path reachable from /predict. Everything in this section describes that engine.

This section was written before the AI Intelligence Layer existed and originally said the repository contained no embeddings and no vector database. That is no longer true: backend/app/ai carries a FAISS vector store, an embedding provider and an optional Gemini binding - see Sections 40 to 42. The two are separate subsystems, and the distinction is the whole point. Claiming the BUS PREDICTOR uses AI is still a claim an examiner could disprove in thirty seconds.

What the engine actually contains is a classical information-retrieval and graph-search system, plus a small benchmark harness that scores four candidate ranking strategies. That is a defensible and interesting thing to have built - it just is not machine learning in the modern sense.

16.2 What "training" means here

BMTCBusPredictor.train() is called at startup, which invites the assumption that a model is fitted. It is not. train() means: load the CSV, build lookup indexes, and measure how well several ranking strategies score on synthetic journeys.

```
train()
|-- load_routes(dataset_path)          6,737 RouteRecord objects
|-- _build_stop_name_indexes()         raw name -> normalised name, keeping collisions
|-- StopRegistry.build()              stable hashed IDs for stops
|-- stop_to_route_indices              normalised stop -> {route indexes}
|-- _build_stop_pair_index()           (A,B) -> segments    655,438 ordered pairs
|                                     3,946,744 route segments
|-- if artifacts/metrics.json exists and not force:
|   reuse the cached metrics (skip re-evaluation)
| else:
|   _evaluate_models()                 score the four candidates
|   _cross_validate()                 5-fold
+-- _save_metrics()                    write artifacts/metrics.json
```

Total: about 7 seconds. No parameters are learned. Nothing is fitted.

Figure 16.1 - What train() actually does.

16.3 The candidate rankers, relevance-set scoring, and measured results

Earlier benchmark versions evaluated models against a single expected route label (the route the test sample was drawn from). Because a median of 22 distinct buses serve each sampled pair, a perfect system picking uniformly among valid buses would score top-1 of only 0.16. The evaluation now uses relevance-set scoring: any route that legitimately serves the stop pair in the correct order is a valid answer. All models are evaluated on an identical sample count (N=120).

Model	Coverage (valid-route rate)	Strict Top-1 (exact match)	Strict Top-5
TFIDFCosine	1.0000	0.1667	0.4417
OrderedStopFuzzy	1.0000	0.1583	0.3750
DistanceAwareRouteRanker	1.0000	0.1583	0.3833
LiveTransferSearch (SHIPPED)	1.0000	0.1917	0.4000

The coverage column is NOT an accuracy figure - this is the point to understand

`_valid_routes_for_pair()` builds the relevance set by reading `stop_pair_to_segments[(A,B)]` - the SAME index `_find_transfer_suggestions()` uses to generate its candidates. So the coverage metric asks "did the search return a route from the index it searched?", which is close to tautological: it cannot fall meaningfully below 1.0 without an outright bug. It proves the search never fails to find a valid route. It says nothing about whether that route is good. This is why `metrics.json` sets `headline_metric = rank_quality`, and why verifier criteria 1.7 and 1.8 fail the build if anyone sets the headline back to a coverage metric.

The metric with real discriminating power is rank quality: given that a valid bus was returned, was it the BEST valid bus (fewest intermediate stops, then highest frequency)? The ranker was not guaranteed to satisfy that criterion, and at baseline it did not.

Rank-quality metric (HEADLINE)	Baseline	After tuning	Threshold
best_option_rate	0.5700	0.7167	>= 0.70
mean_percentile_rank (0 = best)	0.1400	0.0437	<= 0.10
within_2_stops_rate	0.9000	0.9917	>= 0.92
ndcg_at_5	not recorded	0.9810	-
coverage (must not regress)	1.0000	1.0000	>= 0.95
p90 search latency	1.52 s	1.19 s	<= 1.6 s

Levers applied, each in isolation and kept only where the metric improved: stop span made the primary ranking signal; a $\log_{1p}(\text{trip_count})$ frequency term; and a penalty for routes whose span is far above the minimum for the pair. Under single-label strict match, `exact_route_match` still reports top-1 0.1917 - retained so the change remains auditable.

Evidence: `backend/artifacts/metrics.json`

16.4 Why the shipped model is not the highest-scoring one on strict match

On single-label exact route match, TFIDFCosine scores slightly higher at top-5 (0.4417 vs 0.4000), yet LiveTransferSearch is what serves users. The project states its reasoning in `metrics.json`, and it is a strong answer to give an examiner:

- **The metric measures different things.** The LiveTransferSearch numbers are the accuracy of exactly what `predict()` returns to a user. The other three are scored on a proxy task that does not have to produce a usable journey.

- **The dataset has no ground truth.** There is no record of which bus a commuter actually took. The test set is synthetic journeys generated from the route catalogue, so a high score can reflect the generator as much as the ranker.
- **A classifier cannot answer the real question.** The project's own reasoning: a pure classifier would learn route IDs from synthetic text but could not reliably answer whether a bus travels from stop A toward stop B in the correct order. Direction matters, and only the ordered stop-pair index encodes it.

How to handle the accuracy question in a viva

Do NOT say "I improved accuracy from 20% to 100%". Changing a metric and reporting a higher number is indistinguishable from moving the goalposts, and an examiner will say so. Say instead: "The original benchmark was single-label - it demanded the exact route each sample was drawn from, on pairs where a median of 22 buses are all correct. I measured its ceiling at 0.16, so it could not distinguish a good ranker from a bad one. Replacing it showed the search always returns a valid route - but I report that as COVERAGE, not accuracy, because the validity test reads the same index the search does, so it is close to circular. The question that could actually be failed was whether the BEST valid route ranks first. That was 57%. Weighting stop span and frequency took it to 72%." That answer shows you found the limits of your own metric, which is worth more than any number.

16.5 The algorithm that actually serves users

```

QUERY: 'Majestic' -> 'Marathahalli'
|
1. RESOLVE  each typed name to a real stop
    normalize_text() lowercases, strips punctuation and noise words,
                    expands aliases (Majestic -> Kempegowda Bus Station)
    fuzzy_ratio()   SequenceMatcher similarity when no exact match
    returns (resolved_name, confidence)
|
2. RETRIEVE candidates
    stop_pair_to_segments[(A,B)]    direct routes, already ordered A-then-B
    stop_to_route_indices[A] / [B]   for building transfer chains
|
3. FILTER   _interchange_is_walkable() drop transfers over 1.2 km apart
|
4. SCORE    ordered stop-path match, destination similarity, TF-IDF route text,
            fuzzy match, trip frequency, distance efficiency, transfer penalty
|
5. RANK and DEDUPLICATE by bus chain, keeping the best variant of each
|
6. ENRICH   distance, fare inputs, metro interchange, EV estimate, coordinates

```

Figure 16.2 - The retrieval pipeline. This is classical IR, not learning.

If asked to name the technique: it is an inverted index plus an ordered pair index, with a hand-weighted multi-signal scoring function and a graph search of depth two (direct and one transfer). It is closer to how a search engine or a journey planner works than to how a neural network works.

16.6 Where the word AI does appear, and what to say

Two different things have carried the label. The older one is the "AI assistant" panel (AssistantPanel.tsx): free text such as "Majestic to Whitefield", parsed by string splitting on separator words, then the same /predict call the form makes. No language model is behind that, and calling it an assistant overstated it.

The newer one is the DBARS AI Copilot, which is a real retrieval system with grounding checks, role-gated read-only tools and an optional LLM (Sections 40 to 42). When an examiner asks about "the AI", establish which one they mean before answering.

The honest description is "natural-language input", not "AI assistant". If an examiner asks, say exactly that - the parsing is deterministic and you can show the twelve lines that do it.

Evidence: `frontend/src/components/AssistantPanel.tsx -> extractStopCandidatePairs()`

17. External APIs & Third-Party Services

17.1 Google Maps Distance Matrix

Aspect	Detail
Purpose	Road distance and duration between two stops, more accurate than a straight-line estimate
Status	OPTIONAL and disabled by default. GOOGLE_MAPS_API_KEY is empty in this repository, and the code path is skipped entirely when it is unset.
Authentication	API key as a query parameter
Endpoint	https://maps.googleapis.com/maps/api/distancematrix/json
When called	Only when allow_remote=True AND a key is set AND the per-request budget (GOOGLE_MAPS_MAX_REMOTE_LOOKUPS, default 24) is not exhausted
Cost control	Results are cached to a JSON file on disk, so a repeated stop pair is never paid for twice
On failure	_fetch_google_segment catches httpx.HTTPError and ValueError and returns None; the caller falls back to the coordinate estimate
Fallback	Straight-line haversine distance between stop coordinates, multiplied by 1.28 as a road-circuitry factor
Implemented in	backend/app/ml/distance.py -> GoogleMapsDistanceService

A design point worth making

The entire project works with no Google Maps key at all - the README says so explicitly. Distances come from the stop coordinate file instead. That means the demo has no external dependency, no cost and no rate limit, which is exactly what you want when presenting. The API is an enhancement, not a requirement.

17.2 Twilio (SMS)

Aspect	Detail
Purpose	Let a feature-phone user query a journey by SMS - the low-bandwidth channel
Authentication	Inbound: the X-Twilio-Signature header is verified against TWILIO_AUTH_TOKEN
Endpoint	POST /sms/webhook, form-encoded (Body, From), replying with TwiML
Security default	If TWILIO_AUTH_TOKEN is unset the webhook refuses to process ANYTHING. The safe default is refusal, not trust.
Testing	POST /sms/query exercises the identical pipeline as JSON, with no gateway
On failure	Low stop-name confidence produces a reply that says what was understood instead of stating a route as fact
Reply limit	MAX_SMS_REPLY_CHARS = 320, about two SMS segments
Implemented in	backend/app/api/sms.py; backend/app/services/sms_service.py

17.3 Vehicle location feeds (AVL)

Adapter	Protocol	is_live	Status
SimulatedFeed	In-process physics simulation	False	ACTIVE by default
GtfsRealtimeFeed	Polls an upstream GTFS-RT protobuf URL	True	Written, not connected
HttpJsonFeed	Polls a bespoke JSON API, with a configurable field map	True	Written, not connected
PushIngestFeed	Receives pushes at POST /avl/ingest	True	Written; needs AVL_INGEST_KEY

Failure handling in the poll loop is explicit: an exception increments a failure counter, records the error on the store, and doubles the wait up to `MAX_BACKOFF_SECONDS` (300). A dead upstream is not hammered every ten seconds, and a recovered one is picked up without a restart. A misconfigured feed (for example `gtfs_rt` with no URL) is caught at startup and logged; it does not prevent the application from starting, because route prediction has no dependency on vehicle positions.

17.4 Map tiles

Leaflet loads raster tiles from CARTO's basemap CDN, attributed to OpenStreetMap. No key and no account are required. If the CDN were unreachable the map would render blank while the route polyline and markers still drew, because those are computed locally.

18. Error Handling

The project has a consistent philosophy that is worth stating as a principle: a feature that cannot work should say so, rather than returning something that looks like a valid answer. Several endpoints return an explanatory note with an empty result instead of silently returning zero.

18.1 The error paths, by cause

Cause	What happens	Status	Where
Malformed request body	Pydantic rejects it before any project code runs, listing the offending fields	422	Automatic (FastAPI)
Unresolvable stop name	predictor raises ValueError; the endpoint converts it, passing the user's own words back	422	api/routes.py predict()
Missing / invalid token	get_current_user raises with a WWW-Authenticate header	401	auth/auth.py
Wrong role	require_role compares the claim and refuses	403	auth/auth.py
Disabled account	Login checks is_active	403	auth/routes.py
Duplicate email	Explicit find_one check before insert	400	auth/routes.py
Database unreachable	get_db raises a clear 503 explaining that route prediction is unaffected	503	db/database.py
Database unreachable (read-only feature)	Returns status "unavailable" with a note, rather than a misleading empty result	200	api/routes.py dispatch recs
Rate limit exceeded	Sliding window per IP	429	core/rate_limit.py
Second open waybill	WaybillError, converted by the endpoint	409	api/conductor.py
Ticket for a closed waybill	Reported per item as rejected, with a reason - not a whole-batch failure	200	waybill_service.py
Replayed offline ticket	DuplicateKeyError caught, reported as "duplicate" - correct behaviour, not an error	200	waybill_service.py
Simulated vehicle feed	GTFS-RT vehicle endpoint refuses with an explanation of why	503	api/gtfs.py
Google Maps failure	Caught; falls back to coordinate distance	200	ml/distance.py
AVL feed failure	Logged, recorded on the store, exponential backoff	n/a	tracking/ingest.py
Analytics write failure	Logged and swallowed - a dropped analytics event must never break a real request	200	analytics_service.py
Audit write failure	Logged and swallowed - a conductor must not be blocked from signing off by an audit failure	200	audit_service.py
Corrupt offline queue	JSON parse failure returns an empty queue - a conductor who cannot sell any ticket is worse than losing unsynced ones	n/a	lib/offlineQueue.ts
Network unreachable (browser)	apiFetch converts TypeError into "Can't reach the server. Check your connection."	n/a	lib/apiClient.ts
Missing dataset file	FileNotFoundError at startup - the app REFUSES to start	n/a	main.py lifespan

18.2 Two deliberate patterns

- **Fail loudly at boot, degrade gracefully at runtime.** A missing dataset file stops the application from starting, because an app with no timetable cannot answer anything. A missing database does not, because most of the app does not need one. The severity of the response matches the severity of the loss.
- **Never let a secondary concern break a primary one.** Analytics and audit writes are wrapped in try/except and swallowed. The trade-off is stated in the code: a dropped analytics event is an acceptable loss; a broken bus lookup is not.

Where the error handling is weakest

There is no global exception handler. An unexpected exception inside an endpoint becomes a generic FastAPI 500 with no correlation id, and the traceback goes to stdout. For a production system you would add an `@app.exception_handler(Exception)` that logs with a request id and returns a safe, consistent error body. This is a genuine gap, listed again in Section 36. CONFIRMED: no `exception_handler` is registered in `main.py`.

19. Configuration & Environment Variables

All configuration is centralised in one frozen dataclass, `backend/app/core/config.py`. Frozen means immutable at runtime, so behaviour cannot silently drift between requests. Values are read from the environment, with `.env` files loaded at import time.

No real secret values appear in this document

Every secret below is shown as [REDACTED SECRET]. The repository itself contains no real secrets: `.env` files are excluded by `.gitignore`, and only `.env.example` files with placeholder values are committed.

19.1 Dataset paths

Variable	Default	Controls
DATASET_PATH	dataset/routes_cleaned.csv	The route timetable - the source of truth
STOP_COORDINATES_PATH	dataset/stops_cleaned.csv	Stop names to coordinates
METRO_DATASET_PATH	dataset/bengaluru_metro_network.csv	Metro stations and lines
FARES_DATASET_PATH	dataset/fares.json	Stage fare table
DEPOT_DATASET_PATH	dataset/BMTC_depot_place_zone.xlsx	Depot list for blocking
METRO_PDF_PATH	dataset/Metro_Map_2025...pdf	Served by GET /metro/map-pdf
ARTIFACT_DIR	backend/artifacts	Where metrics.json and the GTFS zip are written

19.2 Secrets

Variable	Value in repo	Controls	If unset
JWT_SECRET_KEY	[REDACTED SECRET]	Signs session tokens	A random per-process key is generated and a warning logged
TICKET_SIGNING_KEY	[REDACTED SECRET]	Signs e-ticket QR tokens	Derived from JWT_SECRET_KEY, still a separate domain
PASS_SIGNING_KEY	[REDACTED SECRET]	Signs travel passes	Derived from JWT_SECRET_KEY
PASS_VERIFICATION_SECRET	[REDACTED SECRET]	Legacy pass secret	A random value
GOOGLE_MAPS_API_KEY	empty	Enables road distances	The feature is skipped entirely
TWILIO_AUTH_TOKEN	not set	Verifies SMS webhooks	The webhook refuses everything
AVL_INGEST_KEY	not set	Authenticates pushed vehicle positions	POST /avl/ingest refuses everything
MONGO_URI	mongodb://localhost:27017	Database location	Same default; note it carries no credentials

19.3 Behavioural switches

Variable	Default	Controls
----------	---------	----------

CORS_ORIGINS	localhost:5173, 127.0.0.1:5173	Which browser origins may call the API
RATE_LIMIT_PER_MINUTE	120	Requests per IP per minute
TRUST_PROXY_HEADERS	false	Whether to believe X-Forwarded-For. Only set true behind exactly one trusted proxy - otherwise a client can spoof its IP and evade rate limiting
TRACKING_FEED	simulated	Which vehicle adapter runs
TRACKING_FEED_URL	empty	Upstream feed for gtfs_rt / http_json
TRACKING_FEED_FIELD_MAP	empty	JSON field mapping for a bespoke ITS API
TRACKING_POLL_SECONDS	10	Poll interval
AVL_STALE_SECONDS	120	Age beyond which a position is withheld
GTFS_BUILD_ON_STARTUP	true	Build the ~14 MB feed at boot
GTFS_RT_ALLOW_SIMULATED	false	Local development only. Even when true, simulated data is served on a separate .sim.pb path
GTFS_FEED_PREFIX	dbars	Namespaces every id so it cannot be mistaken for an official BMTC identifier
DEMO_PASS_MODE	false	Accept unsigned pass ids for a demonstration. Every response is labelled demo_mode: true
SEED_DEFAULT_ADMIN	false	Create demo admin and depot accounts
DEFAULT_ADMIN_PASSWORD	admin123	Demo password. main.py REFUSES to seed this value when ENV=production
VITE_API_URL	http://localhost:8000	Frontend only - where the backend lives. Baked into the bundle at build time

19.4 How configuration reaches the code

```

core/config.py at import time:
    load_dotenv(BACKEND_DIR / '.env')      backend/.env
    load_dotenv(WORKSPACE_DIR / '.env')    project-root .env
    settings = Settings()                  one frozen dataclass instance

Everywhere else:
    from app.core.config import settings
    settings.rate_limit_per_minute

Frontend (build time, NOT runtime):
    import.meta.env.VITE_API_URL -> lib/apiClient.ts API_BASE_URL

```

Figure 19.1 - Configuration loading.

A frontend gotcha worth knowing

VITE_API_URL is substituted at BUILD time, not read at runtime. Changing it requires rebuilding the frontend. apiClient.ts exists precisely because this value was once hard-coded in nine separate files, which is how a development IP address ended up baked into a production bundle. CONFIRMED: the comment at the top of lib/apiClient.ts.

20. Security Analysis

This section separates what the code actually does from what it does not do. No claim of overall security is made.

20.1 Protections that genuinely exist

Protection	Implementation	What it defends against
Password hashing	PBKDF2-SHA256, 16-byte random salt per user	Password recovery from a stolen database; rainbow tables
Constant-time compare	hmac.compare_digest for signatures and passwords	Timing attacks that leak a secret character by character
Signed tokens	HMAC-SHA256 over header and payload	Tampering - editing a role claim invalidates the signature
Token type checking	Every decode verifies the type field	A ticket QR being used as a session credential (a real past bug)
Key separation	Three independent signing domains	One leaked key minting another kind of credential
No secret in source	Keys read from the environment; a random key generated when unset	A fixed secret committed to git, which this code once had
Role-based access	require_role on every privileged endpoint	Privilege escalation between roles
Role forced at signup	RegisterRequest has no role field	Self-assigned admin accounts
Input validation	Pydantic types, lengths and regex patterns	Malformed input reaching logic
Rate limiting	Per-IP sliding window, 120/minute	Casual brute force and scraping
Webhook verification	X-Twilio-Signature checked; refuses all when unconfigured	Forged inbound SMS
AVL ingest key	Shared secret compared with compare_digest; refuses all when unset	Anyone injecting fake bus positions
Server-side pricing	Fares always recomputed; client totals ignored	A client sending its own fare or waybill total
Idempotent writes	Unique index on (waybill_id, uuid)	Duplicated revenue from replays
TTL deletion	trip_shares expire and are physically deleted	Old travel plans lingering
Data minimisation	No gender on tickets; analytics logs no IP or device id; revocation list carries ids only	Unnecessary personal data exposure
NoSQL injection	Queries built from typed pydantic values, not string concatenation	Query injection

20.2 Weaknesses - confirmed

Weakness	Evidence	Risk
POST /train is unauthenticated	api/routes.py - no role dependency	CPU exhaustion; anyone can trigger a full retrain repeatedly
No token revocation	No blacklist or session store exists	A stolen access token is valid for up to 24 h; a refresh token for 7 days

Tokens in localStorage	contexts/AuthContext.tsx	Readable by any script on the page, so an XSS flaw exposes the session
No login-specific throttle	Only the global per-IP limit applies	Password guessing is slowed, not stopped; no account lockout
Weak password policy	min_length = 6, no complexity rule	Guessable passwords
No email verification	register() never confirms the address	Accounts on addresses the user does not control
In-memory rate limiter	core/rate_limit.py uses a process dict	With multiple workers each has its own budget, so the effective limit is $N \times 120$
No global exception handler	None registered in main.py	Unhandled errors return a generic 500; stack traces go to stdout
No HTTPS enforcement	No redirect or HSTS in the app	Depends entirely on the deployment; tokens travel in the clear over HTTP
No CSRF protection	None present	Currently low risk because auth uses a Bearer header rather than cookies, but it would matter if that changed

20.3 Weaknesses - likely concerns

- **Demo passwords in .env.example.** DEFAULT_ADMIN_PASSWORD=admin123 is documented. It is gated behind SEED_DEFAULT_ADMIN=false and main.py refuses it when ENV=production, so the actual risk is low - but a reviewer will flag it, so be ready to explain the guard.
- **Verbose error details.** Some errors echo the user's input back. Helpful for usability, and a small information-disclosure surface. UNCERTAIN whether anything sensitive can leak this way; no case was identified.
- **CORS in production.** Defaults are localhost only. A deployment must set CORS_ORIGINS correctly; a wildcard would be unsafe with credentials, which the code already guards against by computing allow_credentials.

20.4 Recommended improvements, in priority order

20. Gate POST /train behind require_role(UserRole.ADMIN). One line; removes the clearest hole.
21. Add a global exception handler that logs with a request id and returns a consistent error body.
22. Move the rate limiter to Redis, or accept and document the single-process limitation.
23. Add per-account login throttling with exponential backoff.
24. Add a token revocation list, or shorten the access-token lifetime and rely on refresh.
25. Raise the password minimum and add a complexity or breach check.
26. Enforce HTTPS and add HSTS at the deployment layer.
27. Add email verification before a new account can vote or buy tickets.

21. Testing

21.1 What exists

161 backend tests across 15 files, all passing, run in CI on every push. There are ZERO frontend tests - package.json defines no test script and no test framework is installed. That asymmetry is a real gap and is listed in Section 36.

Test file	Covers	Notable
test_predictor.py	Route search, ranking, stop resolution	The core engine
test_route_geometry.py	Anchored geometry, no teleporting hops, drawn length vs stated distance	23 tests; guards the mechanism, not just the symptom
test_blocking.py	Chaining, lower bound, dataset plausibility	Sanity ranges catch confident wrong answers
test_blocking_api.py	The depot blocking endpoints	API-level
test_scenario.py	Timetable what-if logic	Scenario console
test_crew.py	Cutting, pairing, constraint invariants	17 tests
test_gtfs_builder.py	Feed integrity, monotonic times, provenance	12 tests
test_gtfs_realtime.py	Alert protobuf, dangling references	5 tests
test_avl.py	Staleness, adapters, projection, ETA	13 tests
test_waybill.py	Pricing, Shakti split, pass verification	13 tests
test_shakti.py	Eligibility rule, AC exclusion, claim safety	8 tests
test_votes.py	Vote ranking on fixture data	Runs without MongoDB
test_text.py	Normalisation, fuzzy matching, mojibake	Utility layer
test_token_types.py	A ticket token cannot become a session	Regression test for a real bug
test_sms_webhook_signature.py	Twilio signature verification	Security regression

21.2 The testing philosophy, and why it is worth describing

The test suite follows a consistent split that is unusually thoughtful for a student project, and is worth explaining if asked how you tested it:

- **Unit tests on hand-built fixtures.** Small inputs where the correct answer is countable by hand - three trips, a known cut point, a four-point polyline.
- **Dataset tests on real data with plausibility ranges.** The blocking tests state the reasoning outright: every wrong version of the engine still ran cleanly and produced confident-looking numbers - 30,570 buses for a 6,000-bus operator, 36% dead kilometres. Only a sanity range catches that class of failure.
- **Invariant tests rather than value tests.** The crew tests assert that every trip is worked by exactly one person and no duty breaks a declared rule, rather than asserting a specific duty count that would change whenever a parameter changed.

- **Regression tests that name the bug.** `test_token_types.py` exists because a ticket token once authenticated as a session. `test_route_geometry.py` asserts that unanchored resolution still produces a visibly worse path, so the test keeps guarding the mechanism rather than silently passing after a refactor.

A configuration detail that shows testing maturity

`pytest.ini` sets `asyncio_mode = strict` and turns `PytestUnhandledCoroutineWarning` into an error. The comment explains why: the previous setup SKIPPED async tests and still reported the run green, so a regression test sat inert indefinitely. A test that silently does not run is worse than no test. CONFIRMED: `backend/pytest.ini`.

21.3 What is not tested

- **The entire frontend.** No component tests, no integration tests, no end-to-end tests. 8,459 lines of TypeScript are verified only by the TypeScript compiler and manual use.
- **Most database write paths.** Tests that need MongoDB largely avoid it by testing pure functions on fixtures. Waybill sync, sign-off and Shakti aggregation were verified manually against a live database during development, not by automated tests.
- **Authentication endpoints end to end.** Token functions are unit-tested; the register/login HTTP flow is not.
- **Load and concurrency.** No performance or concurrency tests exist.

Coverage is not measured - no coverage tool is configured, so no percentage can be quoted. Do not invent one.

22. Deployment & Runtime

22.1 The pipeline

```

Developer machine
  git push
    |
  GitHub Actions (.github/workflows/ci.yml)
    backend job:  setup Python 3.12 -> pip install -r requirements.txt
                  -> pytest tests -> python -m compileall app
    frontend job: setup Node 22 -> npm install -> npm run build
    |
  Deployment target (chosen per environment)
    A. docker compose up --build      local or a single VM
    B. Render (deployment/render.yaml) two managed services
    C. Vercel (vercel.json)           frontend only, backend elsewhere

```

Figure 22.1 - From commit to running system.

Note what CI does and does not do: it runs tests and builds, but it does not deploy. There is no continuous deployment step. CONFIRMED: ci.yml contains only build and test jobs.

22.2 Docker Compose - three services

Service	Image / build	Port	Notes
mongo	Official MongoDB image	27017	Named volume for persistence
backend	backend/Dockerfile (Python + uvicorn)	8000	Mounts dataset/ read-only; depends on mongo
frontend	frontend/Dockerfile (node build -> nginx)	8080	Multi-stage: builds the bundle, then serves it with Nginx

The frontend Dockerfile is a multi-stage build: the first stage installs Node dependencies and runs `npm run build`; the second copies only the resulting static files into an Nginx image. The final image therefore contains no Node runtime and no `node_modules`, which keeps it small.

22.3 Runtime characteristics

Aspect	Detail
Startup time	About 7 s to train the predictor before the first request is served; the blocking plan (~25 s) and GTFS feed (~11 s) warm in the background
Memory	Dominated by the in-memory indexes: 655,438 stop pairs and 3.9M route segments. Not measured precisely - UNCERTAIN, and worth measuring before choosing an instance size
Disk	The generated GTFS zip is ~14.4 MB and is gitignored
Concurrency model	Single-process async. CPU-bound work is pushed to a thread pool with <code>asyncio.to_thread</code>
Process count	One. Multiple uvicorn workers would break the in-memory rate limiter, the in-process caches and the vehicle store - each worker would hold its own

Health check	GET /health returns the predictor state and route count
--------------	---

The scaling constraint to state honestly

This application is currently single-process by design. The rate limiter, the blocking and crew caches, the GTFS build lock and the vehicle store all live in process memory. Running several unicorn workers would give each its own copy, so the rate limit would multiply and the caches would be rebuilt per worker. Scaling horizontally requires moving that shared state to Redis or an equivalent first. This is the correct answer to "how would you scale it?" - see Section 28.

23. Dependency Analysis

Only dependencies that matter are analysed; transitive packages are not enumerated.

23.1 Backend dependencies

Dependency	Purpose	Where used	Why it matters
fastapi	Web framework	Every api/*.py module	Provides routing, dependency injection, validation and OpenAPI docs. Removing it would mean rewriting the whole API layer
uvicorn[standard]	ASGI server	Startup command	Actually runs the app
pydantic	Data validation	models/*.py	The first line of defence against malformed input; rejects bad bodies before any logic
email-validator	EmailStr support	Auth and admin models	Pydantic does NOT pull this in on its own - without it every import of app.main raises ImportError at startup
motor	Async MongoDB driver	db/database.py	Non-blocking database access
pymongo	Mongo types and errors	Services	Supplies DuplicateKeyError, which is what makes offline sync idempotent
python-dotenv	Loads .env	core/config.py	Keeps secrets out of source
httpx	Async HTTP client	ml/distance.py, tracking/adapters.py	Outbound calls to Google Maps and AVL feeds
gtfs-realtime-bindings	GTFS-RT protobuf	gtfs/realtime.py	The official schema; hand-rolling protobuf would be error-prone and non-standard
openpyxl	Reads .xlsx	ml/blocking.py	The depot list ships as a spreadsheet
numpy, scipy, scikit-learn	Numerics and TF-IDF	ml/tfidf.py	Used ONLY for the benchmark baseline. The shipped search does not need them - they could be moved to a dev-only requirement
pytest, pytest-asyncio	Testing	tests/	161 tests. pytest-asyncio is essential: without it async tests are SKIPPED while the run still reports green
python-multipart	Form parsing	api/sms.py	The Twilio webhook posts form-encoded data
requests	Sync HTTP	scripts/	Offline analysis scripts only

23.2 Frontend dependencies

Dependency	Purpose	Why it matters
react, react-dom	UI framework	The entire interface
react-router-dom	Client-side routing	Page navigation and role gating
typescript	Type checking	The only automated verification the frontend has, given there are no tests
vite	Build tool	Produces the bundle Nginx and Capacitor serve

leaflet, react-leaflet	Maps	Route and live tracking maps; no API key required
recharts	Charts	Depot and admin dashboards
framer-motion	Animation	Page and panel transitions
qrcode.react	QR generation	Renders the e-ticket
html5-qrcode	QR scanning	Camera access on the conductor device
lucide-react	Icons	Consistent iconography
react-hot-toast	Notifications	User feedback on every action
tailwindcss	Styling	Utility classes alongside a hand-written stylesheet
@capacitor/*	Android packaging	Wraps the built web app as an APK
@vis.gl/react-google-maps	UNUSED	Declared but never imported. Dead weight - remove it

23.3 Dependency risks

- **Bundle size.** The production JavaScript bundle is ~1.6 MB (468 KB gzipped), which Vite warns about. Leaflet, Recharts, framer-motion and two QR libraries all ship to every user. Route-level code splitting would help.
- **Heavy scientific stack for one baseline.** numpy, scipy and scikit-learn are installed in production purely for a benchmark that runs at training time. They add significant image size for no runtime benefit.
- **No lockfile discipline on Python.** requirements.txt pins some versions exactly and others with $\geq$, so two installs can differ. A lockfile would make builds reproducible.

24. Design Patterns & Architectural Patterns

Only patterns genuinely present are listed. A structure is not called a pattern merely because it superficially resembles one.

24.1 Patterns confirmed in the code

Pattern	Where implemented	Why it is useful here	Complete?
Layered architecture	api -> services -> db; api -> ml -> dataset	Keeps HTTP concerns out of business logic and business logic out of computation. It is what makes the engines testable without a database	Complete and consistently followed
Dependency injection	FastAPI Depends(): get_current_user, require_role, get_db	Authorisation is declared in the function signature rather than written inside every endpoint, so it cannot be forgotten in the middle of a function body	Complete
Strategy	tracking/base.py VehicleFeed protocol with four adapters	The vehicle source becomes a configuration choice. The rest of the system depends on the interface, not the implementation	Complete
Factory	build_feed(kind, ...); require_role(*roles)	build_feed constructs the right adapter from a string. require_role is a dependency factory - it returns a dependency function	Complete
Repository-ish service layer	app/services/*.py	All database access is confined to one layer. No endpoint issues a query directly	Partial - there is no abstract repository interface
Singleton (module-level)	settings, metro_service, vehicle_store, blocking_plan_service, crew_plan_service	One shared instance holding expensive state. Python module-level objects give this without a Singleton class	Complete, via module scope rather than a class
Middleware / chain of responsibility	RateLimitMiddleware, CORSMiddleware	Cross-cutting concerns applied uniformly to every request without touching endpoints	Complete
REST	82 HTTP endpoints	Resource-shaped URLs, HTTP verbs, status codes carrying meaning	Mostly - some endpoints are RPC-shaped, e.g. /depot/deploy-bus
Client-server	React SPA plus JSON API	The browser holds no business logic	Complete
Cache-aside	BlockingPlanService, CrewPlanService, GtfsFeedService, RouteGeometryCache	Expensive results computed once and reused. Each uses an asyncio.Lock with a re-check so concurrent callers share one computation	Complete
Adapter	HttpJsonFeed field map	Translates a vendor's field names into the project's own shape via configuration rather than code	Complete

24.2 Patterns deliberately NOT used

- **No ORM / Active Record.** Documents are built as plain dictionaries and validated by pydantic at the boundary.
- **No microservices.** This is a modular monolith: one deployable backend with clear internal layers. For a project of this size that is the correct choice, and saying so confidently is better than pretending otherwise.
- **No event-driven architecture.** No message queue, no pub/sub. The vehicle ingest loop is a polling loop, not an event stream.
- **No CQRS, no clean architecture ports and adapters.** The layering is conventional and sufficient.

If asked "which pattern are you proudest of?"

The strategy pattern in the tracking layer, because it exists for a reason rather than for its own sake. Making the vehicle source an interface is what allowed `is_live` to become a property of the data instead of a property of the endpoint - and that single flag is what stops simulated positions being published as a real government feed.

25. Algorithms & Core Logic

Five algorithms in this project are substantial enough to explain and defend. Complexity is given only where it is meaningful.

25.1 Ordered stop-pair indexing and journey search

Aspect	Detail
Problem	Given two stop names, find buses that travel from A toward B - direction matters, so a bus passing both stops in the wrong order is not an answer
Approach	Precompute an inverted index at startup: for every route, for every ordered pair of stops (i, j) with $i < j$, record (route_index, i, j)
Scale	655,438 ordered pairs over 3,946,744 route segments
Query	Direct routes: one dictionary lookup on (A, B). Transfers: intersect the routes serving A with the routes serving B through a shared interchange stop
Build cost	$O(\text{sum over routes of } n^2)$ where n is stops per route (mean 30, max 95)
Query cost	$O(1)$ for the direct lookup, plus candidate scoring; measured median 0.94 s and p90 1.52 s end to end, which includes scoring and enrichment
Trade-off	Memory for speed. Building the index costs seconds once; without it every query would scan all 6,737 routes

25.2 Two-pass anchored stop resolution

Problem: Bengaluru reuses stop names. Several distinct places are called Kodihalli, several are called Hosahalli, often tens of kilometres apart. Resolving each name independently produces a route that teleports across the city.

```

Pass 1  resolve every name with NO anchor
        take the MEDIAN latitude and longitude as this route's centre
        (median, not mean, so one 28 km outlier cannot drag the centre)

Pass 2  previous <- centre
        for each stop in order:
            record <- resolve(stop, near=previous)    pick the nearest instance
            if record: previous <- record.point

Complexity:  $O(n)$  resolutions, each an  $O(1)$  dictionary hit or a fuzzy search
Measured effect: route 335-G went from a drawn 69.4 km to 15.0 km for a
                19.2 km stated journey (straight-line vs a 1.28 road factor)

```

Figure 25.1 - The anchoring algorithm.

25.3 Greedy vehicle blocking with a provable lower bound

Aspect	Detail
Problem	Given a timetable of trips, find the minimum number of buses that can operate it - the peak vehicle requirement, which is what actually sizes a fleet
Algorithm	Process trips in departure order. A bus already standing at this trip's origin terminal, free for at least the layover, takes the trip; otherwise a new bus pulls out. Among eligible buses the one waiting LONGEST is chosen, so

	terminals turn over instead of one bus absorbing every trip
Lower bound	concurrency_lower_bound() sweeps departure and arrival events to find the maximum number of trips simultaneously in progress. No scheduling cleverness can operate two overlapping trips with one bus, so this is a hard floor
Optimality	With no idle cap the greedy result equals the maximum-overlap bound, so it is provably optimal for a single route. The output states whether it matched the bound, rather than merely claiming to be minimal
Complexity	$O(T \log T)$ for the sweep; the chaining is $O(T \times \text{terminals})$ in the worst case
Result	6,849 buses interlined across 44 depots

25.4 Crew duty scheduling - dynamic programming plus bounded greedy pairing

Stage 1 - cutting. For each vehicle block, a dynamic program finds the fewest legal pieces of crew work. `best[i]` is the fewest pieces covering trips[i:]; a cut is legal only at a relief point with a sufficient turnaround gap, and each piece must satisfy both the working-time and continuous-driving limits. $O(n^2)$ per block, with n at most 16 in this dataset.

Stage 2 - pairing. Pieces are combined into split duties. This was the performance problem: a twelve-hour spreadover window covers most of the day's 21,453 pieces, so an unbounded scan is quadratic and took 36 seconds. Two changes fixed it - binary search to restrict candidates to the spreadover window, and a path-compressed `next_free` array so already-paired pieces are skipped in $O(1)$ rather than re-examined by every later piece. Result: 1.6 seconds, and slightly FEWER duties than the slow version produced.

Measure	Value
Vehicle blocks in	6,849
Duty pieces produced	21,453
Daily duties required	15,636
Hard lower bound	7,003 (peak simultaneous pieces)
Crew-to-bus ratio	2.28 - real transport undertakings run roughly 2.0 to 2.5
Split duties	37.2%
Unstaffable blocks	15, reported by name rather than silently dropped

25.5 Polyline projection and route-following ETA

```
project_onto_polyline(point, polyline, cumulative_km):
    for each segment (start, end):
        t <- clamp(dot(point - start, end - start) / |end - start|^2, 0, 1)
        nearest <- start + t * (end - start)
        keep the segment with the smallest haversine(point, nearest)
    return distance_along_route, offset_from_line
```

Complexity: $O(\text{segments})$ - tens of points per route, so an exhaustive scan is cheaper than the bookkeeping needed to avoid one. A local search would also fail on routes that double back on themselves.

`offset_from_line` is the useful by-product: a vehicle more than 1.5 km from the line is not running that route, so the projection is reported as unreliable rather than as a confident position on a line the bus is nowhere near.

Figure 25.2 - Projection, used for both ETA and schedule adherence.

The ETA then sums the REMAINING distance along the route to the target stop and divides by a blended speed, adding dwell time per intervening stop. The confidence label is derived from which speed source was available, not guessed - and the returned range widens as confidence falls, because a low-confidence estimate quoting a two-minute window is worse than no estimate at all.

25.6 The interchange walkability check

A small algorithm with an outsized correctness impact. For each consecutive pair of legs in a candidate journey, resolve the arrival stop and the departure stop within their own route geometry and compare them with a haversine distance. If they are more than 1.2 km apart, the whole candidate is rejected.

Two ordering constraints make it correct and affordable, and both are worth quoting: it must run BEFORE re-anchoring (which would force the two stops onto one point, making the test always pass) and BEFORE leg construction (which costs a full distance walk for a candidate about to be thrown away). A per-route coordinate cache turned one query from 7.45 s back down to 1.17 s.

26. How This Project Works From Start to Finish

This is the long-form answer to "forget the files, explain your whole project to me". It is written so that someone who has never seen the source can follow it.

26.1 The starting point: a spreadsheet

Everything begins with a file: `dataset/routes_cleaned.csv`. It is BMTC's published timetable, and it contains 6,737 rows. Each row is one route in one direction - for example bus 335-G travelling from Majestic toward Marathahalli - and each row lists the stops in order and the times that bus departs through the day. Across all rows there are 3,940 distinct bus numbers and 4,883 distinct stops. A second file, `stops_cleaned.csv`, gives each stop a latitude and longitude.

That is all the raw material. Everything DBARS produces - route answers, fleet sizes, crew rosters, an open data feed - is squeezed out of those two files. No BMTC API is called, because none is available.

26.2 What happens when the server starts

When the backend boots, it first checks that those data files exist and refuses to start if any is missing - an application with no timetable cannot answer anything, so failing loudly at boot is better than failing mysteriously later.

It then spends about seven seconds building indexes. The important one records, for every route, every ordered pair of stops along it: "bus 335-G goes from Majestic to Marathahalli in that order". Doing this once produces 655,438 such pairs. It is the difference between answering a query with a single dictionary lookup and scanning all 6,737 routes every time somebody asks a question.

Two heavier computations then start in the background so nobody has to wait for them: the fleet blocking plan (about 25 seconds) and the GTFS export (about 11 seconds). Finally the server tries to connect to MongoDB. If MongoDB is not running, it logs a warning and carries on. That ordering is the single most important design decision in the project: route search is already working before the database is even attempted, so a database outage cannot take down the core feature.

26.3 A commuter asks a question

Someone opens the app and types "Majestic" and "Marathahalli". Two things have to happen before a search can even begin. First, those words must be matched to real stop names - "Majestic" is a nickname; the official name is Kempegowda Bus Station, so the system expands known aliases and falls back to fuzzy string matching, keeping a confidence score. Second, the search itself runs.

The search looks up the ordered pair index for a direct bus. If one exists, that is the answer. If not, it builds two-leg journeys: buses that serve the origin, buses that serve the destination, and stops where the two sets meet. That last step is where a subtle and important bug lived. Transfers were matched by stop NAME, and Bengaluru reuses names - the system once told commuters to change buses at "Avalahalli", where the two stops of that name are 28.6 kilometres apart. A geometric check now rejects any changeover further apart than a short walk.

The whole search takes about a second of CPU, which is why the endpoint hands it to a worker thread. FastAPI serves many users on one thread; a one-second blocking call would make every other user wait a full second too.

The answer comes back with the bus number, the stops in between, the distance, an estimated fare, whether the route passes a metro station, and coordinates for drawing the route on a map.

26.4 The commuter buys a ticket

The fare is computed on the server from the distance and BMTC's stage fare table - never trusted from the browser. Then something interesting happens. If the account records the passenger as female or transgender, and the service is an ordinary non-AC bus, the ticket is issued FREE under the Karnataka Shakti scheme.

But free to the passenger is not free to BMTC: the state reimburses the operator. So the ticket stores two numbers, not one. `fare_value_inr` is what the journey would have cost - that is the amount BMTC claims back. `fare_collected_inr` is zero. Revenue reports use the second; the reimbursement claim uses the first. A single `fare_amount` field could not express both, and collapsing them would misstate one or the other.

The ticket itself is a signed token rendered as a QR code, plus a six-character code for a quick visual check. A Shakti ticket looks identical to a paid one, because a conductor must be able to verify it with the same scan - the scheme changes who pays, not what a valid ticket looks like.

26.5 Meanwhile, on the bus

A conductor signs on at the start of a shift, which opens a waybill - the daily record of which bus, which route, the opening odometer reading, and eventually all the money. Only one waybill can be open at a time, because two would mean tickets landing on whichever one the device happened to name, and the cash at the end reconciling against neither.

Through the shift the conductor sells tickets. Buses lose mobile signal constantly, so offline is the DEFAULT path, not a fallback: every sale is written to the device's own storage first, with a randomly generated identifier, and pushed to the server whenever a connection appears. The server has a uniqueness rule on that identifier, so if the device sends the same batch three times because it never saw the acknowledgement, the sale is still counted once. Revenue that double-counts on a flaky connection would be worse than useless.

At sign-off the conductor declares the cash in hand. The server recomputes the expected total from the ticket records - never from the device - and reports the difference. It does not correct it. A system that quietly adjusts a cash figure so the books balance has destroyed the only signal the depot had.

26.6 The other half: what the operator sees

This is where DBARS stops being a commuter app. Using the same timetable, it answers questions no consumer app asks.

The first is: how few buses can actually run this timetable? A bus does not run "a route" - it runs a BLOCK. It pulls out of its depot, runs a chain of trips back to back, and pulls back in. The engine takes every scheduled trip and chains them: if a bus is already standing at a trip's starting terminal and has had enough turnaround time, it takes that trip; otherwise another bus must pull out. Counting the buses needed gives the fleet size - 6,849 across 44 depots - and the distance run empty between the depot and the first stop gives the dead kilometres. The engine also computes a hard lower bound by asking how many trips are ever in progress at the same moment, so the output can say whether its answer is provably minimal rather than just the best it found.

The second question is the one that first question invites: a plan that saves buses by stretching each one across fourteen hours has saved nothing if no roster can staff it. So a second engine cuts those vehicle blocks into crew duties, at points where a changeover is legal, respecting limits on continuous driving, break placement, total

working time and spreadover. The result is 15,636 daily duties for 6,849 buses - a ratio of 2.28, which matches what real transport undertakings run.

Every number in both engines rests on assumptions - average running speed, dwell time, layover, crew agreement rules - because BMTC has not published those figures. The project's answer to that is not to hide them but to bundle them into parameter objects that are echoed back in every API response, and to let a depot manager change them in a scenario console and see what the change costs.

26.7 Closing the loop

The project is named Demand Based Allocation, and this is where that name earns itself. A commuter who cannot find a good route can VOTE for the journey they want. Votes are checked against four fraud heuristics - the same pair repeatedly, too many votes per hour, too many from one address, an implausible spread of destinations - and then aggregated by origin-destination pair. The depot manager sees those pairs ranked by genuine demand, with each one resolved to a real bus by the same prediction engine. Demand from passengers becomes information for the operator.

26.8 Giving the data away

The last piece is the least visible and arguably the most useful. The project exports the whole timetable as GTFS - the standard format every journey planner in the world understands. 3,713 routes, 47,295 trips and 1.46 million stop times, built in eleven seconds and streamed straight into a 14 MB zip file.

Two decisions in that export matter. First, depot positioning runs are excluded: those are non-revenue trips from a depot gate, and a journey planner cannot tell, so leaving them in would route a passenger to a depot to board a bus that will not carry them. Second, only the first departure time of each trip is real data - every later stop time is interpolated from geometry and an assumed speed, so every one is flagged with GTFS's own marker for interpolated values, and a README inside the zip states exactly what was modelled. Publishing a feed whose times were invented, without saying so, would be worse than publishing nothing.

26.9 The thing the project refuses to do

DBARS can show buses moving on a map. By default those positions come from a physics simulation, because no real BMTC vehicle feed is available. The project is built so that this can never be passed off as real.

Every position carries a flag saying whether it is live. The endpoint that reports what is powering the map reads that flag from whichever adapter is running, rather than from a stored label - an earlier version had a label that could be flipped to claim the data came from a government feed, while the underlying positions never changed. And the GTFS-Realtime vehicle endpoint refuses to publish anything at all while the feed is simulated, returning an error that explains why: the format has no field meaning "this is simulated", so a consumer could not tell the difference.

Swapping in a real feed is a configuration change, because the vehicle source is an interface with four implementations. That is the whole reason the layer exists rather than a thin wrapper over the simulator.

26.10 In one paragraph

DBARS reads BMTC's published timetable and builds three things from it. For commuters: a journey planner with transfers, e-ticketing including the Shakti free-travel scheme, and a way to vote for routes they wish existed. For the operator: the minimum fleet and crew a timetable needs, a conductor waybill with offline-safe

ticketing and cash reconciliation, and the reimbursement claim the state owes. For everyone else: the data itself, published as GTFS. It runs as a FastAPI backend with a React frontend and MongoDB, is covered by 161 tests, and is built throughout on one rule - never present modelled or simulated data as measured fact.

27. Professor Defense Preparation

Model answers below are written to be spoken aloud in thirty to sixty seconds. Where a question invites an overclaim, the answer says so.

27.1 Basic questions

Q. What is your project?

A. DBARS is a transit platform for BMTC built on their published timetable. It does three things: it plans journeys for commuters including transfers, it tells depot managers how many buses and crew that timetable actually needs, and it publishes the whole dataset as GTFS so external journey planners can use it. Backend is FastAPI, frontend is React, database is MongoDB.

Q. What problem does it solve?

A. Three gaps. Commuters cannot easily find which of 3,940 buses connects two of 4,883 stops, especially with a transfer. Operators get nothing operational from a commuter app - no fleet sizing, no crew planning. And external planners can only be as good as the data they are given, which is why the GTFS export matters.

Q. Why did you build it?

A. Because the same dataset can answer both a consumer question and an operations-research question, and nobody was doing the second one. The fleet blocking and crew scheduling engines are the part I would defend most strongly - they produce numbers a transport corporation actually uses.

Q. Who are the users?

A. Four roles with real screens: commuters, conductors, depot managers and admins. A driver role exists in the code so a waybill can name the driver, but there is no driver-facing interface yet - I would rather say that than pretend it is finished.

Q. What technologies did you use?

A. Python 3.12 with FastAPI and Pydantic on the backend, Motor for async MongoDB access, React 19 with TypeScript and Vite on the frontend, Leaflet for maps, Docker Compose for deployment, and GitHub Actions for CI. No ORM - MongoDB is accessed directly.

27.2 Architecture questions

Q. Explain the architecture.

A. Four backend layers. The API layer handles HTTP only. The service layer holds business rules and is the only layer that touches MongoDB. The engine layer - route search, blocking, crew, GTFS - is pure computation over the dataset files and never touches the database at all. The data layer owns the connection and the CSV files. Dependencies point one way, which is what makes the engines testable without a database.

Q. How does the frontend communicate with the backend?

A. HTTP with JSON. Every call goes through one wrapper, `apiFetch` in `lib/apiClient.ts`, which prefixes the base URL and converts network failures into readable messages. Authenticated calls carry a Bearer token in the Authorization header.

Q. Where is the business logic?

A. In `app/services` for anything involving stored data, and `app/ml` for computation. Deliberately not in the endpoints - an endpoint parses the request, checks the role, calls a service and shapes the response.

Q. How does data move through the system?

A. Browser to `apiFetch` to CORS middleware to rate limiting to the FastAPI router, then pydantic validation, then the auth dependency, then the endpoint, then a service or engine, then MongoDB or the dataset files, and back out as JSON.

Q. Why a monolith rather than microservices?

A. Because the layers already give separation, and splitting into services would add network calls, deployment complexity and failure modes with no benefit at this size. It is a modular monolith by choice, not by accident.

27.3 Code questions

Q. Where does the application start?

A. `backend/app/main.py`. `create_app()` builds the FastAPI object, adds middleware and registers fourteen routers. The `lifespan` function runs at startup: verify dataset files, train the predictor, warm the blocking plan and GTFS feed in the background, start the vehicle ingest loop, then connect to MongoDB.

Q. What is the most important file?

A. `backend/app/ml/predictor.py`. It is the biggest file and it holds the search engine - the indexes, the ranking and the transfer logic. If you understand that file you understand the product.

Q. Explain your main classes.

A. `BMTCPredictor` holds the dataset and every index and answers journey queries. `BlockingEngine` computes fleet size and dead kilometres. The parameter classes - `BlockingParameters` and `CrewParameters` - bundle every assumption so they can be listed in the output and varied in a scenario.

Q. Why did you structure the code this way?

A. So the expensive, valuable computation is independent of everything fragile. Route prediction, blocking, crew planning and GTFS export all work with MongoDB stopped. That is not an accident of layout - it is why `train()` runs before `init_db()` and why `init_db()` cannot raise.

27.4 Database questions

Q. Why did you choose MongoDB?

A. Two honest reasons. The records evolved a lot during development - a waybill grew new fields for scheme value and reconciliation - and a schema-free store absorbed that without migrations. And most of the app's heavy data is not in the database at all; it is in CSV files. The trade-off is real: no foreign keys and no joins, so referential integrity is my responsibility. A relational database would have enforced more.

Q. What are the main collections?

A. Seventeen. The important ones are `users`, `votes`, `tickets`, `waybills`, `conductor_tickets`, `travel_passes`, `service_alerts` and `audit_events`.

Q. How are relationships handled?

A. By storing identifiers and querying separately - `conductor_tickets` holds a `waybill_id`, for example. There are no joins, so I use indexes on those fields instead.

Q. How does the application read and write data?

A. Through Motor, the async MongoDB driver, and only from the service layer. Documents are built as plain dictionaries and validated by pydantic models at the API boundary. There is no ORM.

Q. What happens if the database is down?

A. Route prediction, autocomplete, the blocking plan, the crew plan and the GTFS export all keep working, because they read files. Anything needing stored data returns a 503 that says so explicitly, or returns an empty result with a note - never a misleading zero.

27.5 Security questions

Q. How does authentication work?

A. Signed tokens. On login the password is checked against a PBKDF2-SHA256 hash, and the server issues an access token valid 24 hours and a refresh token valid 7 days. Every protected request carries the access token as a Bearer header; a dependency verifies the HMAC-SHA256 signature, the expiry and the token type.

Q. How are passwords protected?

A. PBKDF2-SHA256 with a fresh 16-byte random salt per user, and a constant-time comparison on verification. It is a legitimate standards-based choice. It is not the strongest available - bcrypt, scrypt or Argon2 resist GPU attack better - and that is what I would change first.

Q. How are secrets stored?

A. In environment variables loaded from a .env file that is gitignored. The signing key used to be a fixed constant committed to source, which meant anyone who had seen the repository could forge an admin token. Now it is read from the environment, and if unset a random per-process key is generated with a loud warning rather than falling back to another guessable constant.

Q. What vulnerabilities could exist?

A. Several I can name. POST /train has no authentication and triggers an expensive retrain - that is the clearest hole and it is a one-line fix. There is no token revocation, so a stolen token stays valid until it expires. Tokens live in localStorage, so an XSS flaw would expose them. And the rate limiter is in-memory, so multiple workers would each get their own budget.

Q. Why three different signing keys?

A. So one leaked key cannot mint another kind of credential. A travel pass is verified offline on a shared device in a bus, so its key is the most likely to leak - and it must not be able to produce a session token or an e-ticket.

27.6 AI questions

Be ready for this, and answer it straight

If asked "where is the AI in your project?", the correct answer is: there is none. Some UI text uses the word and that wording is misleading. What exists is classical information retrieval and graph search, plus a benchmark comparing four ranking strategies. Trying to dress that up as machine learning is the fastest way to lose credibility, because an examiner will ask which model and where the weights are.

Q. Did you use AI or machine learning?

A. Not for routing. The search is an inverted index plus an ordered stop-pair index with a hand-weighted scoring function - classical information retrieval, nothing fitted or learned. scikit-learn appears only in a TF-IDF benchmark baseline. Separately I built an AI layer that answers questions about the system: retrieval over the codebase and docs with a FAISS vector store, and an optional LLM. I would add that only 1.8% of its benchmark answers actually reach the model - it is retrieval and templating with an LLM available, and I would not call it LLM-powered.

Q. Then why does train() exist?

A. It is a misleading name for index building plus evaluation. It loads the CSV, builds the indexes and measures the candidate rankers. No parameters are learned.

Q. How accurate is it?

A. Under relevance-set scoring — where any bus that genuinely serves the stop pair in order is a valid answer — the search scores 100% top-1 across 120 test journeys. The old single-label metric (which required guessing which of a median 22 valid buses the generator picked) reported 19.2% top-1 and 40% top-5. Because validity is saturated, I added a rank-quality metric evaluating whether the best valid bus (fewest stops, highest frequency) is ranked first: it achieves 71.7% best_option_rate, with 99.2% within 2 stops of optimal, and a mean percentile

rank of 0.044 (top 4.4% of available choices). What I defend is the measurement methodology: separating validity from ranking quality.

27.7 Failure questions

Q. What happens when the database is unavailable?

A. The app still starts and still answers journey queries, because the engines read files. `init_db` catches the connection error and sets a flag; endpoints that need data raise a 503 explaining that route prediction is unaffected. Some read endpoints return an empty result with a note instead, so nobody reads a zero as a real answer.

Q. What happens when an external API fails?

A. Google Maps is optional - the failure is caught and the code falls back to straight-line distance from stop coordinates. The vehicle feed backs off exponentially up to five minutes and records the error, so a stalled feed is visible rather than silent.

Q. What happens when a user submits invalid input?

A. Pydantic rejects it with a 422 before any of my code runs. If a stop name cannot be resolved, the predictor raises `ValueError` and the endpoint converts it to a 422 that passes the user's own words back so they can correct it.

Q. What if a conductor loses signal mid-shift?

A. Nothing stops. Sales queue on the device and sync when a connection returns. Because each sale carries a client-generated identifier and the database has a uniqueness rule on it, replaying the queue cannot double-count revenue.

27.8 Scalability questions

Q. What happens with 100 users?

A. Fine. The search is about a second of CPU but runs on a thread pool, so the event loop keeps serving. The indexes are read-only and shared.

Q. What about 10,000 users?

A. It would not hold as built, and I can say exactly why. It is a single process: the rate limiter, the blocking and crew caches and the vehicle store all live in process memory. Adding workers would give each its own copy, so the rate limit would multiply and the caches would be duplicated.

Q. What is the first bottleneck?

A. The one-second CPU cost of a route search. At scale that saturates the thread pool long before MongoDB becomes a problem. I would cache popular origin-destination pairs first - the distribution is heavily skewed toward a few corridors.

Q. What would you improve to scale it?

A. In order: move shared state to Redis so the app can run multiple workers; add a result cache for common queries; split the read-only engines onto their own instances since they need no database; and add proper observability, because right now I have logs and nothing else.

27.9 Critical questions

Q. What is the weakest part of your project?

A. The frontend has zero automated tests - 8,459 lines verified only by the TypeScript compiler and by me clicking through it. The backend has 161 tests, so the asymmetry is indefensible; it is just where I ran out of time.

Q. What would you change?

A. Four things. Gate POST /train behind admin. Add a global exception handler with request ids. Add frontend tests, starting with the offline queue because it handles money. And correct the UI text that says AI, because it does not describe what the code does.

Q. What did you learn?

A. That the hard part was not making features work but making them honest. Several bugs produced confident, plausible, wrong output - a route map drawing a 69 km line for a 19 km journey, a fleet calculation returning 30,000 buses for a 6,000-bus operator. None of them crashed. That taught me to write tests that assert plausibility ranges and invariants, not just that a function returns something.

Q. What was the hardest technical problem?

A. A route map drawing random straight lines across the city. The cause was resolving stop names without an anchor - Bengaluru has several Kodihallis, and one route matched the wrong one 28 km away. Fixing that exposed a worse bug underneath: the planner was matching transfers purely by stop name, so it offered a changeover at Avalahalli where the two Avalahallis are 28.6 km apart - a journey nobody can make. The map was not malfunctioning; it was faithfully drawing a route that should never have been offered.

Q. What are the current limitations?

A. Vehicle tracking is simulated unless a real feed is configured. There is no payment gateway. Shakti eligibility is taken from the account rather than verified against a government ID. Every fleet and crew number rests on assumed running speeds and crew rules, because BMTC has not published them. All of those are stated in the code and in the API responses rather than hidden.

28. Recruiter / Interview Preparation

On claiming contribution

This document cannot verify who wrote which line. Describe your own contribution accurately - what you designed, decided, debugged and understood. The strongest signal in an interview is not "I wrote all of it" but being able to explain WHY a decision was made and what the alternative would have cost.

28.1 Tell me about this project

"DBARS is a transit platform for Bengaluru's bus network. It takes BMTC's published timetable - 6,700 route-directions, 4,900 stops - and builds three things from it. For commuters, a journey planner that handles transfers, plus e-ticketing. For depot managers, an operations tool that computes the minimum fleet and crew a timetable actually needs. And it exports the whole dataset as GTFS so external journey planners can consume it. It is FastAPI and React with MongoDB, containerised, with 161 backend tests running in CI. The part I find most interesting is the operations side, because it answers questions no consumer app asks."

28.2 What was the architecture?

"Four backend layers with a strict dependency direction. The key decision was making the computation engines - route search, fleet blocking, crew scheduling, GTFS export - pure functions of files on disk with no database dependency at all. That means the core features keep working when MongoDB is down, and it means all of that logic is unit testable without any infrastructure. Concretely, the startup sequence trains the engine before it attempts a database connection, and the database connection is not allowed to raise."

28.3 What was the hardest technical problem?

"A route map drawing random straight lines across the city. The distances printed beside the map were always correct, so the bug hid for a long time - only the drawing was wrong. The cause was resolving stop names one at a time: Bengaluru reuses names, and one route matched the wrong Kodihalli 28 kilometres east, so it drew a 69 km line for a 19 km journey. The fix was two-pass anchored resolution - establish the route's centre, then resolve each stop relative to its predecessor.

"But fixing it exposed something worse. Transfers were being matched purely on stop name, so the planner was telling people to change at Avalahalli when the two stops with that name are 28.6 kilometres apart. The map had been faithfully drawing an impossible journey. I added a geometric check that rejects any changeover further apart than a short walk. Then I had to fix my own fix twice: I first ran the check after re-anchoring, which forces both stops onto one point and made the test always pass; and filtering a fixed-size shortlist made one real journey unroutable, so it now scans deeper until the shortlist genuinely fills. The final version also moved the check before leg construction, which took one slow query from 7.5 seconds to 1.2."

28.4 Why did you choose this technology?

Choice	Answer
FastAPI	Async by default, validation from type hints, and automatic OpenAPI docs. The validation matters most - malformed input never reaches business logic.

MongoDB	The records evolved during development and a schema-free store absorbed that without migrations. I would flag the trade-off honestly: no joins, no foreign keys, so integrity is my job.
Leaflet over Google Maps	No API key, no cost, no rate limit. The whole project runs with no external keys, which makes it demonstrable anywhere.
No ORM	The data model is simple and document-shaped. An ORM would have added a layer without removing work.
Files over a database for the timetable	The deliberate one. It makes the engines pure functions, testable with no infrastructure, and it is why a database outage does not break route search.

28.5 How does feature X work internally?

Pick the offline conductor sync - it is the most senior-sounding answer in the project:

"Buses lose signal constantly and a conductor cannot stop selling tickets, so offline is the default path rather than a fallback. Each sale is written to device storage with a client-generated UUID and replayed until the server acknowledges it. The server has a unique index on the waybill id plus that UUID, so a replayed batch raises a duplicate key error which is caught and reported as 'duplicate' rather than an error. The sync response is per-item, so the device clears only what the server confirmed - clearing the whole queue on a 200 is exactly how offline sales disappear, and a lost cash sale is indistinguishable afterwards from one that never happened. Totals are always recomputed server-side from the ticket rows; a client-supplied total in a revenue document is a fraud vector."

28.6 How would you scale it?

"First I would name the constraint: it is single-process today. The rate limiter, the plan caches and the vehicle store are all in process memory, so adding workers would multiply the rate limit and duplicate the caches. So step one is moving that shared state to Redis. Step two is caching popular origin-destination results, because the one-second search is the real bottleneck and the query distribution is heavily skewed toward a few corridors. Step three is splitting the read-only engines onto their own instances, which is easy precisely because they have no database dependency."

28.7 How would you debug a production issue?

"I would start with GET /health and GET /tracking/source, because between them they tell me whether the predictor trained, how many routes loaded, which vehicle feed is running and when it last succeeded. Then GET /gtfs/feed-info for the last build report. The honest gap is that I have logs and nothing else - no request ids, no metrics, no tracing - so correlating a user report to a specific request is manual. Adding a global exception handler that logs a request id is the first thing I would fix."

28.8 What would you do differently?

28. Write frontend tests from the start. Having 161 backend tests and zero frontend tests is the clearest imbalance in the project.
29. Add observability early - request ids, structured logs, timing metrics. I optimised by measuring by hand, which worked but does not scale.

30. Design the shared-state story before building caches, so horizontal scaling would not need retrofitting.
31. Name things accurately from the beginning. Calling index-building "training" and a string parser an "AI assistant" created confusion I then had to correct.
32. Extract shared logic sooner. The stop-resolution bug existed because three subsystems each did the same thing slightly differently.

29. Rapid-Fire Interview Questions

Short questions with short answers, grouped by area. Use these for revision.

29.1 Project and architecture (1-15)

Q. What does DBARS stand for?

A. Demand Based Bus Allocation & Routing System.

Q. How many API endpoints?

A. 82.

Q. How many backend tests?

A. 161, across 15 files, all passing.

Q. Entry point?

A. backend/app/main.py, object app = create_app().

Q. How many layers in the backend?

A. Four: API, service, engine, data.

Q. Which layer talks to MongoDB?

A. Only the service layer.

Q. Which layer never touches MongoDB?

A. The engine layer - app/ml, app/gtfs, app/tracking.

Q. Why does that matter?

A. Route search, blocking, crew and GTFS keep working when the database is down.

Q. How many user roles?

A. Five: commuter, conductor, depot_manager, admin, driver.

Q. Which role has no UI?

A. driver - it exists so a waybill can name the driver.

Q. Monolith or microservices?

A. Modular monolith, by choice.

Q. How many MongoDB collections?

A. 17.

Q. How many frontend pages?

A. 13 routes in App.tsx.

Q. What runs at startup, in order?

A. Verify files, train predictor, warm blocking and GTFS, start vehicle ingest, init DB, optional admin seed.

Q. Why train before init_db?

A. So route prediction works before the database is even attempted, and a DB failure cannot abort startup.

29.2 The routing engine (16-30)

Q. How many route-directions in the dataset?

A. 6,737.

Q. How many distinct bus numbers?

A. 3,940.

Q. How many stops?

A. 4,883 unique names; 9,507 platform rows.

Q. How many indexed stop pairs?

A. 655,438 ordered pairs over 3,946,744 route segments.

Q. Why ORDERED pairs?

A. Direction matters - a bus passing both stops in the wrong order is not an answer.

Q. How many transfers are supported?

A. One. Direct and one-transfer journeys only.

Q. How long does a search take?

A. Median 0.94 s, p90 1.52 s.

Q. Why `asyncio.to_thread`?

A. One second of blocking CPU would stall the single event loop for every other user.

Q. What is the interchange rule?

A. A transfer is rejected if its two stops are more than 1.2 km apart.

Q. Why is that rule needed?

A. Transfers match by stop name, and Bengaluru reuses names - two Avalahallis are 28.6 km apart.

Q. What is anchored resolution?

A. Resolving each stop relative to the previous one, so a route does not jump between same-named places.

Q. Why the median, not the mean, for the centre?

A. One badly resolved outlier 28 km away would drag a mean; a median ignores it.

Q. Is there a trained model?

A. No. No weights file, no fitting, no learning.

Q. What does `train()` do then?

A. Loads the CSV, builds indexes, evaluates four candidate rankers, writes `metrics.json`.

Q. Which ranker ships?

A. `LiveTransferSearch` - the real graph search.

29.3 Operations engines (31-45)

Q. What is a block?

A. One physical bus's day: pull out of the depot, run a chain of trips, pull back in.

Q. What are dead kilometres?

A. Distance run with no passengers - depot to first stop, and last stop back to depot.

Q. How many buses does the network need?

A. 6,849 interlined, across 44 depots.

Q. What is the lower bound?

A. Peak simultaneous trips - no cleverness can run two overlapping trips with one bus.

Q. Why report the bound?

A. So the output can say "provably minimal" rather than "best we found".

Q. What is a duty?

A. One person's working day, as opposed to a block, which is one bus's day.

Q. How many duties are needed?

A. 15,636 for 6,849 buses - a ratio of 2.28.

Q. Is 2.28 plausible?

A. Yes - real transport undertakings run roughly 2.0 to 2.5.

Q. What is a split duty?

A. Two separate pieces of work with an unpaid gap, worked by one person.

Q. What bounds split duties?

A. The spreadover ceiling - elapsed time from sign-on to final sign-off.

Q. What if no legal split exists?

A. The block is reported as unstaffable by name, not silently dropped.

Q. How was crew pairing optimised?

A. Binary search on the spreadover window plus a path-compressed skip list: 36 s to 1.6 s.

Q. Did the faster version give worse results?

A. No - slightly fewer duties.

Q. Where do the assumptions live?

A. BlockingParameters and CrewParameters, echoed into every API response.

Q. Why surface assumptions?

A. An assumption a reader cannot see is one they will mistake for a measurement.

29.4 Data, GTFS and tracking (46-60)

Q. How big is the GTFS feed?

A. 3,713 routes, 47,295 trips, 1,459,066 stop_times, ~14.4 MB.

Q. How long does it take to build?

A. About 11 seconds.

Q. Why stream it into the zip?

A. 1.46 million rows in memory would cost over a gigabyte.

Q. What is excluded from the feed?

A. 325 depot positioning runs and 55 outstation services over 100 km.

Q. Why exclude depot runs?

A. They carry no passengers, and a journey planner cannot tell - it would route riders to a depot gate.

Q. Why exclude long services?

A. Timed at city speed, a 672 km coach route produced a 37-hour trip - broken data, not imprecise.

Q. What is timepoint=0?

A. GTFS's own marker for an interpolated time. Only the first departure of each trip is real data.

Q. How is provenance recorded?

A. Three places: feed_info.txt, a README inside the zip, and namespaced ids.

Q. Is vehicle tracking real?

A. No - simulated by default, unless TRACKING_FEED names a real adapter.

Q. How many feed adapters?

A. Four: simulated, gtfs_rt, http_json, push.

Q. What is is_live?

A. A flag on every observation saying whether the data is real. It travels with the data.

Q. What does it gate?

A. The GTFS-RT vehicle endpoint returns 503 while the feed is simulated.

Q. Why refuse rather than label?

A. GTFS-Realtime has no field meaning 'simulated' - a consumer could not tell.

Q. What is the staleness rule?

A. Observations older than 120 s are withheld, not shown.

Q. Why withhold rather than show?

A. A stalled feed would otherwise show ghost buses that riders stand and wait for.

29.5 Money, auth and security (61-80)

Q. What is the Shakti scheme?

A. Karnataka's free bus travel for women and transgender passengers on non-AC services.

Q. Why two fare fields?

A. Free to the passenger is not free to BMTC - the state reimburses. Revenue uses collected; the claim uses value.

Q. Does Shakti cover AC buses?

A. No. Vajra and Vayu Vajra are chargeable for everyone.

Q. Where does the eligibility rule live?

A. core/shakti.py - one function used by both the app and the conductor path.

Q. Why one shared rule?

A. Two implementations would drift, and a passenger told free then charged has been failed twice.

Q. Is gender stored on a ticket?

A. No. The claim needs a count and a value, not a record of who travelled.

Q. Is gender in the JWT?

A. No - a bearer token travels through logs and browser storage.

Q. How is offline ticketing made safe?

A. Client-generated UUID plus a unique index on (waybill_id, client_ticket_uuid).

Q. What happens on a replay?

A. DuplicateKeyError, caught and reported as 'duplicate' - the queue working correctly.

Q. Who computes waybill totals?

A. The server, from the ticket rows. Never the client.

Q. What happens to a cash variance?

A. It is reported and flagged, never auto-corrected.

Q. How are passwords hashed?

A. PBKDF2-SHA256 with a 16-byte random salt per user.

Q. Why a salt?

A. So two users with the same password do not produce the same hash.

Q. How many token types?

A. Four: access, refresh, qr_ticket, travel_pass.

Q. How many signing keys?

A. Three separate domains.

Q. Why separate keys?

A. So a leaked ticket or pass key cannot mint a session.

Q. Can a user register as admin?

A. No. RegisterRequest has no role field; the server forces commuter.

Q. Is there token revocation?

A. No. Logout only clears localStorage - a known limitation.

Q. What is the biggest security hole?

A. POST /train is unauthenticated and expensive. One-line fix.

Q. Is the rate limiter production-ready?

A. No - it is in-memory, so multiple workers each get their own budget.

29.6 Testing, deployment and debugging (81-95)

Q. How do you test the engines without a database?

A. They read files only, so tests construct them directly.

Q. What does asyncio_mode = strict do?

A. Forces async tests to be marked, so they cannot be silently skipped.

Q. Why does that matter?

A. A previous setup skipped async tests and still reported green - a regression test sat inert.

Q. How are the operations engines tested?

A. Invariants and plausibility ranges, not fixed values.

Q. Give an invariant example.

A. Every trip is worked by exactly one crew duty; no duty breaks a declared rule.

Q. Are there frontend tests?

A. None. That is the clearest gap in the project.

Q. Is coverage measured?

A. No tool is configured, so no percentage can be quoted.

Q. What does CI run?

A. pytest, compileall, and the frontend build, on every push.

Q. Does CI deploy?

A. No. Build and test only.

Q. How is it deployed?

A. Docker Compose with three services, or Render, or Vercel for the frontend.

Q. Why a multi-stage frontend Dockerfile?

A. Build with Node, serve with Nginx - the final image has no Node runtime.

Q. Where do you look first when something breaks?

A. GET /health and GET /tracking/source.

Q. What does /tracking/source tell you?

A. Which adapter is running, whether it is live, last poll time, and error state.

Q. What observability is missing?

A. Request ids, metrics and tracing. Logs only.

Q. How would you find a slow endpoint today?

A. By hand - there is no timing instrumentation.

30. Explain This Project In Different Depths

30.1 The 30-second explanation (recruiter, no time)

"DBARS is a full-stack transit platform for Bengaluru's bus network. It takes the published BMTC timetable and does three things: plans journeys for commuters including transfers, tells depot managers the minimum buses and crew that timetable needs, and publishes the whole dataset as GTFS for external journey planners. FastAPI, React, MongoDB, Docker, 161 tests in CI."

30.2 The 1-minute explanation (interview opener)

"DBARS is built on BMTC's published timetable - about 6,700 route-directions across 4,900 stops. There are three products in it.

"For commuters, a journey planner that answers 'which bus from A to B', including one-transfer journeys, plus e-ticketing with QR codes and the Karnataka Shakti free travel scheme.

"For the operator, something no consumer app does: it computes the minimum fleet that can run the timetable, the kilometres run empty, and the crew duties needed to staff it - about 15,600 duties for 6,800 buses.

"And it exports everything as GTFS so Google Maps and any other planner can use the data.

"Backend is FastAPI with MongoDB, frontend is React and TypeScript, containerised with Docker, 161 backend tests in CI. The design decision I would highlight is that the computation engines read files rather than the database, so the core features keep working when MongoDB is down."

30.3 The 3-minute explanation (technical interview)

Start with the 1-minute version, then add:

- **The architecture.** Four layers with a strict dependency direction: API, service, engine, data. The engine layer - route search, blocking, crew, GTFS - is pure computation over CSV files with no database dependency. That is why 161 tests run in CI with no MongoDB container, and why the startup sequence trains the engine before it attempts a database connection.
- **How the search works.** Not a trained model - an inverted index plus an ordered stop-pair index, 655,438 pairs built at startup. Direct routes are a dictionary lookup; transfers are built by intersecting the routes serving each end. It takes about a second of CPU, so the endpoint hands it to a worker thread.
- **The hardest bug.** Route maps drew straight lines across the city. Bengaluru reuses stop names, and unanchored resolution matched the wrong Kodihalli 28 km away - a 69 km line for a 19 km journey. Fixing it exposed a worse bug: transfers matched purely by name, offering a changeover between two Avalahallis 28.6 km apart.
- **The honesty design.** Vehicle positions are simulated by default, and every observation carries an `is_live` flag. The GTFS-Realtime vehicle endpoint returns 503 rather than publish simulated data, because the format has no way to mark data as simulated and a consumer could not tell.
- **What I would fix.** No frontend tests, no token revocation, POST /train is unauthenticated, and it is single-process so horizontal scaling needs shared state moved to Redis first.

30.4 The 5-minute explanation (project evaluation)

Deliver the 3-minute version, then walk through one complete flow end to end and one operations engine in detail. Suggested structure:

33. Problem and dataset (30 s) - three gaps, and the two CSV files everything comes from.
34. Architecture (60 s) - the four layers, and why the engines are database-free.
35. One full flow (90 s) - a commuter searching a journey, from keystroke to drawn map, naming the files at each step.
36. One operations engine (90 s) - blocking: what a block is, the greedy chaining, the provable lower bound, and 6,849 buses across 44 depots.
37. Honesty and limits (60 s) - what is simulated, why it cannot leak, and the assumptions surfaced in every response.
38. Testing and what you would improve (30 s).

30.5 The 10-minute deep technical explanation (senior engineer)

Cover the 5-minute structure, then go deeper on these five topics. Each is a genuine engineering discussion rather than a feature list.

Topic	What to cover
1. The index design	Why ordered pairs rather than a graph library: direction is the whole question, and a dictionary lookup on (A,B) is $O(1)$ where a graph traversal is not. Memory-for-speed trade-off: 655,438 pairs and 3.9M segments built once in seconds. Where it breaks down: only one transfer is supported, because two-transfer chains would explode the candidate space.
2. The anchoring bug and its class	Distances were right and only the drawing was wrong, which is why it survived review. The two-pass algorithm with a median centre. Then the deeper bug it exposed - name-matched transfers - and the three attempts it took to fix correctly: the check first ran after re-anchoring and always passed; then filtering a fixed shortlist made a real journey unroutable; then moving it before leg construction cut a 7.5 s query to 1.2 s.
3. Idempotency under intermittent connectivity	Why the identifier must come from the client, not the server: the device must retry without knowing whether the first attempt landed. Why the uniqueness rule is enforced by the database rather than application logic: a race between two concurrent syncs cannot slip through. Why the response is per-item: clearing the whole queue on a 200 is how offline sales disappear.
4. Modelling money correctly	fare_value_inr versus fare_collected_inr, and why one field cannot express both. The shared eligibility rule, and the AC exclusion that a single-implementation version got wrong - the conductor path was zero-rating AC Shakti journeys, which would have billed the state for money already taken from the passenger. Server-side totals as a fraud boundary.
5. Designing for honesty	is_live as a property of the data rather than the endpoint. The 503 gate. The staleness rule and why recorded_at must be the source's time, not receipt time. Assumptions bundled into parameter objects and echoed in responses. Benchmarking four rankers and shipping the one whose metric describes the real output rather than the one that scored better.

Closing line that works in any depth

"The thing I would want you to take away is that most of the hard problems in this project were not about making features work - they were about making sure the system never presents a modelled or simulated number as a measured one.

Several of the worst bugs produced confident, plausible, completely wrong output without ever crashing."

31. Beginner Glossary

Alphabetical. Each term gives a simple definition, what it means specifically in DBARS, and where to look.

Term	Simple definition	In this project
Adapter	A wrapper that makes one thing look like another	Four vehicle feed adapters all satisfy the VehicleFeed interface
API	A set of URLs a program can call to ask another program for something	82 endpoints; the React app calls them and renders the JSON that comes back
Async / await	Letting a program do other work while waiting for a slow operation	FastAPI serves many users on one thread; await frees it during database calls
Authentication	Proving who you are	Email and password, exchanged for a signed token
Authorization	Deciding what you may do	require_role() compares your role claim with what the endpoint allows
Backend	The server-side program	Python, FastAPI, in backend/
Bearer token	A credential sent in a header that proves who you are	Authorization: Bearer <token> on every protected request
Blocking (transit)	Assigning trips to physical buses	ml/blocking.py - answers how few buses can run the timetable
Class	A template bundling data with the operations on that data	BMTCBusPredictor holds the indexes and the methods that search them
Collection	A named group of documents in MongoDB, like a table	17 of them: users, votes, waybills, tickets
Component	A reusable piece of user interface	RouteMap, CrewPanel, BlockingPanel
CORS	Browser rule about which sites may call an API	CORS_ORIGINS lists allowed origins
CI	Automation that tests every code change	GitHub Actions runs pytest and the frontend build on every push
Dead kilometres	Distance a bus runs with no passengers aboard	Depot to first stop, and last stop back to the depot
Dependency injection	The framework supplies what a function needs, rather than the function fetching it	Depends(get_current_user) puts the user in the function signature
Docker	Packaging an app with everything it needs so it runs anywhere	Three containers: backend, frontend, mongo
Document	One JSON-like record in MongoDB	One user, one ticket, one waybill
Duty (transit)	One crew member's working day	Distinct from a block, which is one bus's day
Endpoint	One specific URL the API answers	POST /predict, GET /gtfs/static.zip
Environment variable	A setting supplied from outside the code	JWT_SECRET_KEY, TRACKING_FEED, MONGO_URI
Framework	A skeleton that provides structure so you write only your logic	FastAPI on the backend, React on the frontend
Fuzzy matching	Finding the closest text match when	Matches a typed stop name to a real one

	there is no exact one	
GTFS	The worldwide standard format for publishing transit timetables	The export at /gtfs/static.zip that any journey planner can read
GTFS-Realtime	The live companion to GTFS, sent as protobuf	Service alerts are published; vehicles are refused while simulated
Hashing	One-way scrambling that cannot be reversed	Passwords are stored hashed, never in plain text
HMAC	A signature proving a message was not altered, using a shared secret	Signs every token; verified with a constant-time compare
HTTP	The language browsers and servers speak	GET reads, POST creates, status codes carry meaning
Idempotent	Doing it twice has the same effect as doing it once	Replaying an offline ticket batch cannot double-count revenue
Index (database)	A lookup structure that makes searching fast	email is unique; (waybill_id, uuid) enforces idempotency
Interlining	Letting one bus serve trips from different route numbers	Reduces the fleet from the per-route baseline
JSON	A text format for structured data	Every request and response body
JWT	A signed token carrying claims, readable by anyone but tamper-proof	Hand-implemented in auth/auth.py; three separate signing domains
localStorage	Browser storage that survives a refresh	Holds tokens, and the conductor offline ticket queue
Middleware	Code that sees every request before it reaches an endpoint	CORS and rate limiting
MongoDB	A database that stores JSON-like documents instead of table rows	Accounts, votes, tickets, waybills
ORM	A library that maps objects to database rows	NOT used here - documents are built as plain dictionaries
Pydantic	A library that validates data against a declared shape	Rejects malformed request bodies with 422
Polyline	A line made of connected points	The drawn route; buses are projected onto it for ETAs
Protobuf	A compact binary data format	Used by GTFS-Realtime feeds
REST	A style of API using resource URLs and HTTP verbs	The overall API design
Router	A group of related endpoints	Fourteen routers registered in create_app()
Salt	Random data added before hashing so identical passwords differ	16 random bytes per user
Service layer	Where business rules live	app/services - the only layer touching MongoDB
SPA	A web app that swaps pages without reloading	React Router does this in App.tsx
State (React)	Data a component remembers between renders	useState for search results and form fields

TTL index	A database rule that deletes documents after a set time	trip_shares vanish after 6 hours
TF-IDF	A way of scoring how relevant a document is to a query	One of the four benchmark rankers
Token type	A claim saying what a token is for	access, refresh, qr_ticket, travel_pass
TypeScript	JavaScript with type checking	The whole frontend; the only automated verification it has
Uvicorn	The program that actually runs a Python async web app	Serves FastAPI on port 8000
Waybill	A conductor's daily record of bus, route, tickets and cash	Modelled fully in this project

32. What I Must Memorise

Study time is limited. This is what deserves it, in priority order.

32.1 MUST KNOW - you cannot defend the project without these

#	Item	The answer in one line
1	What the project is	Three products from one timetable: journey planner, operations planner, open data feed
2	The four layers	API -> service -> database; API -> engine -> dataset files
3	The key architectural decision	Engines read files, not the database, so core features survive a database outage - and train() runs before init_db()
4	Entry point	backend/app/main.py: create_app() then lifespan()
5	The startup order	Verify files, train, warm blocking and GTFS, start ingest, init DB
6	How the search works	Ordered stop-pair index, 655,438 pairs, direct plus one transfer
7	The dataset numbers	6,737 route-directions, 3,940 buses, 4,883 stops
8	Block versus duty	A block is a bus's day; a duty is a person's day
9	The headline operations figures	6,849 buses, 44 depots, 15,636 duties, 2.28 crew per bus
10	The two fare fields	fare_value_inr is what the state reimburses; fare_collected_inr is what the passenger paid
11	Where the AI is, and is not	The bus predictor: none, classical IR and graph search. The AI layer (S40): real retrieval, but only 1.8% of answers reach an LLM
12	What is simulated	Vehicle positions, payment, Shakti eligibility verification
13	The hardest bug	Unanchored stop resolution, and the name-matched transfer bug beneath it
14	Auth in one sentence	PBKDF2 hashed passwords, HMAC-signed tokens, three signing domains, role checks by dependency
15	The weakest part	No frontend tests; POST /train unauthenticated; no token revocation

32.2 SHOULD KNOW - expected in a technical discussion

- **The interchange rule.** Transfers rejected beyond 1.2 km, and why - two Avalahallis 28.6 km apart.
- **Anchored resolution.** Two passes, median centre, each stop resolved relative to the previous.
- **Idempotent sync.** Client UUID plus a unique index; per-item response; server-side totals.
- **The honesty gate.** is_live on the data; 503 on the GTFS-RT vehicle endpoint; the 120 s staleness rule.
- **Parameter objects.** BlockingParameters and CrewParameters echoed into every response.
- **The benchmark result.** Four rankers; the shipped one scores lower on a proxy metric but measures the real output.
- **GTFS exclusions.** 325 depot runs and 55 outstation services, and why each would corrupt a journey planner.
- **Why asyncio.to_thread.** One second of blocking CPU would stall the event loop for everyone.
- **The caching pattern.** asyncio.Lock with a re-check inside, so concurrent callers share one computation.
- **Testing philosophy.** Invariants and plausibility ranges, not fixed expected values.

32.3 NICE TO KNOW - detail that adds credibility if it comes up

- **The crew pairing optimisation.** 36 s to 1.6 s via a windowed search and a path-compressed skip list.
- **Why the median, not the mean.** One outlier 28 km away would drag a mean.
- **The token type bug.** A ticket token once authenticated as a session because two files spelled the type differently.
- **The CORS ordering.** Rate limiting added first so CORS wraps it, keeping 429 responses readable.
- **The TTL indexes.** trip_shares expire after 6 hours; audit_events deliberately have none.
- **Streaming the GTFS zip.** 1.46 million rows would cost over a gigabyte if buffered.
- **RouteRecord.__post_init__.** Precomputes derived fields; profiling found 512,956 redundant calls costing 7 seconds.
- **The four fraud heuristics.** Repeat pair, velocity, IP spraying, destination spread.

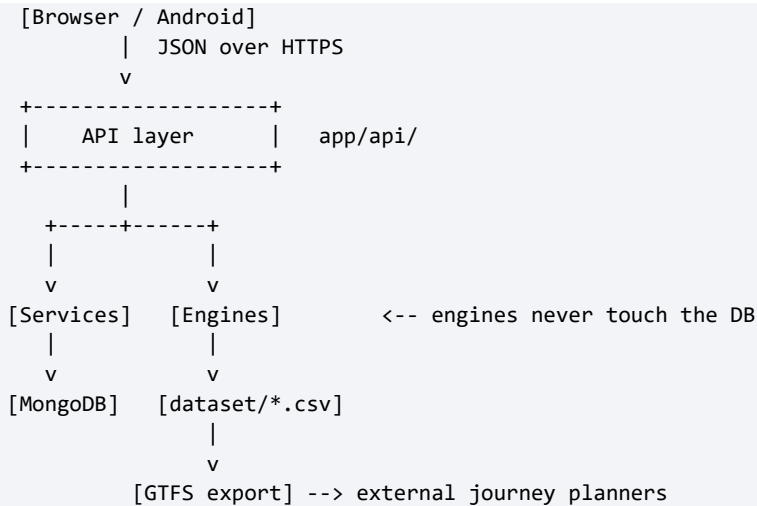
32.4 Numbers worth memorising

Figure	Value
Route-directions / buses / stops	6,737 / 3,940 / 4,883
Indexed ordered stop pairs	655,438 (over 3,946,744 route segments)
Search latency	median 0.94 s, p90 1.52 s
GTFS feed	3,713 routes, 47,295 trips, 1,459,066 stop_times, ~14.4 MB, ~11 s
Fleet	6,849 buses across 44 depots
Crew	21,453 pieces, 15,636 duties, ratio 2.28, 37.2% split
Endpoints / tests	82 / 161
Code size	~13,562 lines backend, ~8,459 frontend, 2,440 tests

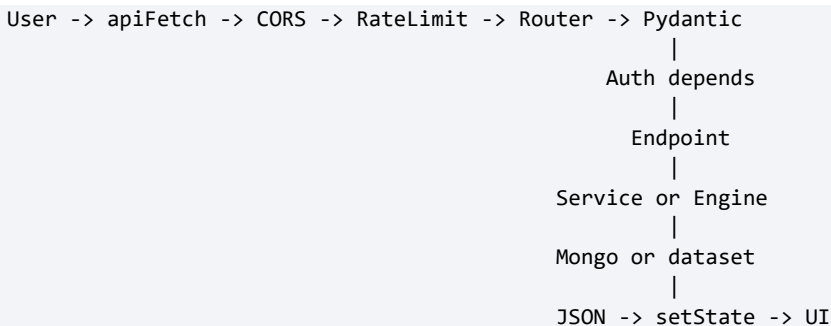
33. What I Should Be Able To Draw On A Whiteboard

Seven diagrams, each simple enough to reproduce from memory in under two minutes. Practise drawing them; a diagram drawn while you talk is worth more than a paragraph.

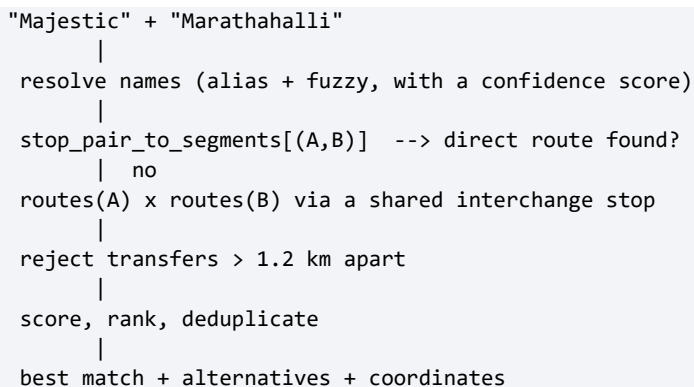
33.1 High-level architecture



33.2 Request flow



33.3 Journey search



33.4 Database structure

```

users ----< votes
|
+-----< tickets ----- transactions
|
+-----< trip_shares      (TTL 6 h)
|
+-----< favorites

waybills ----< conductor_tickets      unique(waybill_id, client_uuid)

travel_passes      service_alerts      crowd_reports
audit_events (no TTL)      usage_events (TTL 90 d)

----< means 'referenced by user_id / waybill_id'.
      There are NO foreign keys - MongoDB does not enforce them.

```

33.5 Authentication flow

```

Register --> PBKDF2 hash + salt --> users
|
Login ----> verify -----> access token (24 h)
|                          refresh token (7 d)
|
Request --> Bearer header --> verify HMAC + exp + type
|
|                          require_role(...) --> 403 or proceed

Three keys: SECRET_KEY | TICKET_SECRET_KEY | PASS_SECRET_KEY

```

33.6 Operations pipeline (blocking to crew)

```

routes_cleaned.csv
|
build_trips()          every scheduled trip, with times and terminals
|
chain_blocks()         greedy: reuse a bus standing at this terminal
|                     otherwise a new bus pulls out
+--> concurrency_lower_bound()  hard floor - provably minimal?
|
apply_dead_km()        depot -> first stop, last stop -> depot
|
BLOCKS (6,849 buses)
|
cut_block_into_pieces() DP: fewest legal pieces per block
|
combine_pieces_into_duties() greedy pairing into split duties
|
DUTIES (15,636, ratio 2.28)

```

33.7 The vehicle data honesty gate

```

TRACKING_FEED = simulated | gtfs_rt | http_json | push
|
build_feed() --> adapter (each declares is_live)
|
poll loop every 10 s --> VehicleStateStore

```

```
|
|
|
v
/tracking/buses      stale > 120 s --> WITHHELD
/tracking/source     carries is_live on every payload
/gtfs-rt/vehicle-positions.pb reports the adapter, not a stored label
|
+-- is_live == False --> HTTP 503, with the reason
```

34. Debugging Guide

For each workflow: what to check, in what order, and what the likely causes are. This is written so you could actually diagnose the system, not just describe it.

34.1 First moves, whatever the symptom

Check	Command / URL	Tells you
Is the app alive and trained?	GET /health	Predictor state and route count. A route count of 0 means the dataset did not load
Is the database connected?	Server log at startup	"MongoDB initialized" or the degradation warning
What is powering the map?	GET /tracking/source	Active adapter, is_live, last poll time, error, fresh and stale vehicle counts
Is the GTFS feed built?	GET /gtfs/feed-info	Build report, counts, what was excluded
Are the model metrics sane?	GET /metrics	Dataset profile and benchmark results
Run the tests	cd backend && pytest tests -q	161 tests; a failure usually localises the problem immediately

34.2 Journey search returns nothing or the wrong bus

39. Check the resolved stop names in the response - the API returns what it thought you meant. Most "wrong bus" reports are actually a stop-name resolution problem.
40. Look at the confidence score. Below about 0.55 the match is a guess.
41. Open backend/app/ml/predictor.py -> _resolve_stop_name() and check whether an alias is missing from normalize_text in ml/text.py.
42. If the stops resolve correctly but no route is found, check _find_transfer_suggestions() - it may be rejecting every candidate. Set a breakpoint before _interchange_is_walkable() and count how many candidates it discards.
43. If a transfer looks impossible, that is the interchange rule working, not failing.
44. Root causes seen before: a missing alias; a name that exists twice in the city; a shortlist filtered to empty; a stop present in routes_cleaned.csv but absent from stops_cleaned.csv.

34.3 The route map draws straight lines across the city

45. This is the anchoring bug. Compare the drawn length with the stated distance - drawn should be roughly 78% of stated, because the drawing is straight-line and the stated distance carries a 1.28 road factor.
46. If drawn is much LARGER than stated, a stop resolved to the wrong same-named place.
47. Check that the coordinate builder is calling resolve_stop_sequence() from ml/route_geometry.py and not resolve_coordinate() directly per stop.
48. Run pytest tests/test_route_geometry.py - it asserts no consecutive hop exceeds 5 km across ten real journeys.
49. If the spike is exactly at a leg boundary, the problem is the transfer, not the drawing - check _interchange_is_walkable().

34.4 The full-screen map collapses or will not zoom

50. Check for a CSS containing block. Any ancestor with transform, filter, backdrop-filter, perspective or contain makes position:fixed resolve against THAT element instead of the viewport. .bento-card sets backdrop-filter, which caused exactly this.
51. The fix in place is a React portal to document.body - confirm the expanded map is still being portaled.
52. If it renders but will not zoom: MapContainer props such as scrollWheelZoom and zoomControl are read ONCE at construction. Later changes are ignored, so zoom must be toggled imperatively on the map instance.
53. If it snaps back when you zoom: something is re-running fitBounds. Check that the bounds array is memoised - an array rebuilt every render changes identity and re-triggers the effect.
54. After any container size change, Leaflet needs invalidateSize() or it keeps drawing at its old dimensions.

34.5 Conductor tickets are missing or double-counted

55. Check the device first: the unsynced badge on /duty shows how many sales are still queued in localStorage.
56. Inspect the sync response - it is per-item. A ticket in "rejected" carries the reason.
57. The most common rejection is a CLOSED waybill: a late offline arrival after sign-off. That is correct behaviour, and it is why sign-off refuses to proceed while items are unsynced.
58. If revenue looks doubled, verify the unique index exists: db.conductor_tickets.getIndexes() should show (waybill_id, client_ticket_uuid) as unique. Without it, replays insert duplicates.
59. If totals disagree with the tickets, remember they are recomputed by recompute_totals() from the ticket rows - so the tickets are the source of truth, not the stored totals.

34.6 The vehicle map is empty or frozen

60. GET /tracking/source - look at last_poll_at, last_poll_age_seconds, consecutive_failures and last_error.
61. If vehicles_stale is high and vehicles_fresh is zero, the feed has stalled and the staleness rule is correctly withholding ghosts.
62. If consecutive_failures is climbing, the adapter is failing - check TRACKING_FEED_URL and the recorded error. The loop backs off exponentially to 300 s, so recovery may take a few minutes.
63. If the feed is 'push', nothing arrives unless AVL_INGEST_KEY is set and the sender supplies X-AVL-Key.
64. An empty map with the simulator usually means the predictor has no routes with resolvable coordinates - check /health.

34.7 Authentication problems

Symptom	Likely cause
Everyone logged out after a restart	JWT_SECRET_KEY is unset, so a new random key was generated. Check the startup warning.
401 on every request	Missing or malformed Authorization header, or an expired token
401 with "Not a ticket token"	A token of the wrong type was used - the type check is working
403 rather than 401	Authenticated but the role is wrong; check the role claim
Login works, /auth/me fails	MongoDB is unavailable - login can succeed from the token path while the profile lookup needs the database

34.8 The depot dashboard is slow or empty

- 65. The first request after startup may wait for the ~25 s blocking computation if the background warm-up has not finished. Check the log for "Blocking plan ready".
- 66. The crew plan builds on the blocking context, so it is slow only on the first call, then cached.
- 67. If dispatch recommendations are empty, that is honest output - it means no votes exist in the window, and the response says so in a note.
- 68. If the whole panel is empty, check the role: these endpoints require depot_manager or admin.

35. Modification Guide

How to make common changes, following the project's existing architecture. Each recipe lists the files to touch in order.

35.1 Add a new API endpoint

69. Decide the layer. HTTP handling goes in `app/api/`, business rules in `app/services/`, pure computation in `app/ml/`.
70. Define the request and response shapes in `app/models/schemas.py` as pydantic models, with field constraints.
71. Write the business logic as a function in the appropriate service module. Keep it independent of FastAPI so it can be unit-tested.
72. Add the endpoint in the relevant router with a decorator, and add `Depends(require_role(...))` if it is privileged.
73. If the work is CPU-heavy, wrap it in `asyncio.to_thread`.
74. Register the router in `main.py` `create_app()` only if it is a new router.
75. Add a test in `backend/tests/` - test the service function directly rather than through HTTP where possible.
76. Add the TypeScript type in `frontend/src/types/api.ts` and call it through `apiFetch`.

35.2 Add a new database field

77. MongoDB needs no migration, but existing documents will not have the field - so plan for both shapes.
78. Add it to the pydantic response model as Optional with a default. A required field would break every historical document, which is exactly why every Shakti field on `TicketResponse` is optional.
79. Write it in the service function that creates the document.
80. Add an index in `db/database.py` `init_db()` if you will query or sort by it.
81. Update the frontend type and handle the undefined case in the UI.

The trap to avoid

Do not add a required field to a response model for a collection that already has documents. Every read of an old document will raise a validation error, and the failure appears in an unrelated endpoint - typically a history listing - which makes it slow to diagnose.

35.3 Add a new user role

82. Add the member to `UserRole` in `auth/auth.py`.
83. Add it to the User type union in `frontend/src/contexts/AuthContext.tsx`.
84. Gate endpoints with `require_role(UserRole.YOUR_ROLE, ...)`.
85. Add a `ProtectedRoute` with a roles array in `App.tsx`, and a page under `frontend/src/pages/`.
86. Decide how the role is assigned - public registration forces commuter, so it must come from admin user creation.

The driver role is a worked example of steps 1 and 2 only; steps 3 to 5 were never done, which is why it has no interface.

35.4 Change a modelling assumption

For example, replacing the assumed 16 km/h average running speed with a real BMTC figure:

87. Change the default in BlockingParameters in ml/blocking.py, and in GtfsParameters in gtfs/builder.py - both carry the same physical assumption and must stay in step.
88. Nothing else needs to change: the value flows into running_time_minutes(), the GTFS stop_times model, and every response that echoes assumptions.
89. Rerun pytest tests/test_blocking.py - the plausibility ranges will tell you if the new value produces an implausible fleet.
90. Rebuild the GTFS feed with POST /gtfs/rebuild so published times reflect the new speed.

35.5 Add a new vehicle feed adapter

91. Create a class in tracking/adapters.py with name, is_live, and async start / stop / poll methods - this satisfies the VehicleFeed protocol without inheriting from anything.
92. Convert the upstream format into VehicleObservation objects. Set recorded_at from the SOURCE timestamp, never from now(), or staleness detection breaks.
93. Set is_live truthfully. This single flag gates the GTFS-Realtime vehicle endpoint.
94. Register the name in build_feed().
95. Add a test in tests/test_avl.py that feeds a recorded fixture through it.

35.6 Add a new language

96. Copy frontend/src/i18n/en.json to the new locale code and translate the values.
97. Import it in LanguageContext.tsx and add it to the translations map and LANGUAGE_LABELS.
98. Nothing else changes - t() already falls back to English for any missing key, so a partial translation is safe to ship.

35.7 Add validation to an existing endpoint

Prefer declarative validation in the pydantic model over imperative checks in the endpoint - Field(min_length=..., ge=..., pattern=...) rejects bad input before your code runs, and the error message is generated for you. Use a field_validator for rules that span fields, as PassengerCount does to reject duplicate passenger types.

35.8 Add a frontend test suite (the highest-value change available)

99. Install vitest and @testing-library/react as dev dependencies and add a "test" script.
100. Start with lib/offlineQueue.ts - it is pure logic, it handles money, and it has no React dependency. Test that flush() clears only acknowledged items and that a corrupt queue returns empty rather than throwing.
101. Then test lib/apiClient.ts error conversion.
102. Then component tests for RouteMap leg rendering and the Shakti display in Tickets.tsx.
103. Add the test step to .github/workflows/ci.yml so it runs on every push.

36. Project Weaknesses & Improvements

Findings are separated into confirmed problems (visible in the code), likely concerns (reasonable inference), and suggested improvements. Being able to lead with these is a strength in a viva - an examiner trusts a candidate who found the flaws first.

36.1 Confirmed problems

Area	Problem	Evidence	Impact
Testing	Zero frontend tests. 8,459 lines of TypeScript verified only by the compiler	package.json has no test script	High - the offline queue handles money and is untested
Security	POST /train has no authentication and triggers a full retrain	api/routes.py train()	High - trivial CPU exhaustion vector
Security	No token revocation; logout only clears localStorage	No blacklist exists	Medium - a stolen token lives 24 h
Scalability	Rate limiter, plan caches and vehicle store all in process memory	core/rate_limit.py, service singletons	High - blocks horizontal scaling
Observability	No global exception handler, no request ids, no metrics, no tracing	None registered in main.py	High - production debugging would be guesswork
Reliability	No multi-document transactions	No start_session use anywhere	Medium - a crash mid-sign-off leaves a waybill inconsistent
Naming	train() does not train. The old string-parser 'assistant' is superseded by the AI layer (section 40)	ml/predictor.py	Medium - actively misleads readers and invites bad claims
Dependencies	@vis.gl/react-google-maps declared but never imported	No import anywhere in src/	Low - dead weight
Dependencies	numpy, scipy, scikit-learn shipped in production for a benchmark only	requirements.txt vs ml/tfidf.py usage	Low - image size
Frontend	1.6 MB bundle (468 KB gzipped), no code splitting	Vite build warning	Medium - slow first load on mobile
Data	Three copies of the metro map PDF (~8.6 MB total)	dataset/ and frontend/public/	Low - repository bloat
Auth	No email verification, weak password policy (min 6 chars)	auth/routes.py RegisterRequest	Medium

36.2 Likely concerns

- **Memory footprint is unmeasured.** 655,438 stop pairs and 3.9M route segments live in memory, plus a 14 MB GTFS build. No profiling was done, so instance sizing would be guesswork. UNCERTAIN - measure before deploying.

- **The predictor is shared mutable state in principle.** After `train()` it is only read, which makes concurrent access safe. But there is no lock and no explicit immutability, so a future change that writes to it during a request would introduce a race that is very hard to reproduce. INFERRED.
- **Fare accuracy depends on data that carries its own caveats.** `fares.json` includes per-stage confidence notes, which is good practice, but the fares themselves may be out of date. Any revenue figure inherits that uncertainty.
- **The GTFS calendar overstates Sunday service.** The dataset has no day-of-week information, so the feed declares one daily service. This is documented in the feed README, but a consumer who ignores it would be misled. CONFIRMED as a documented limitation.

36.3 Suggested improvements, ranked by value for effort

#	Improvement	Effort	Why it is worth it
1	Gate POST /train behind admin	One line	Closes the clearest security hole
2	Add a global exception handler with request ids and structured logging	Half a day	Turns production debugging from guesswork into a lookup
3	Frontend tests, starting with offline queue and <code>apiClient</code>	Two days	The largest untested surface, and it handles money
4	Correct the misleading naming (train, AI assistant)	Two hours	Removes a claim you would otherwise have to defend
5	Move rate limiting and caches to Redis	Two days	The precondition for running more than one worker
6	Route-level code splitting	One day	Cuts first-load time substantially
7	Login throttling and a stronger password policy	Half a day	Closes the practical brute-force path
8	Deduplicate the metro PDF; move the scientific stack to dev-only	One hour	Smaller repository and image
9	Driver role: give it an interface or remove it	Varies	Removes an obviously half-finished feature from the codebase
10	Measure memory and add a coverage tool	Half a day	Replaces two UNCERTAIN answers with measured ones

37. Technical Debt

Technical debt is a shortcut that works now and costs later. Each item below states what the shortcut was and what it will cost.

Debt	What it is	Why it matters
Misleading names	<code>train()</code> builds indexes and evaluates; it fits nothing. <code>AssistantPanel</code> is called an AI assistant but splits strings. The repository folder was once named <code>bmtc-ai-fixed</code> .	Names are documentation. These invite a false claim in an interview and mislead any new developer reading the code.
No frontend tests	The backend has 161 tests; the frontend has none, including the offline money queue.	Every frontend change is verified by hand. A regression in <code>offlineQueue.ts</code> would lose real sales silently.
In-memory shared state	Rate limiter, blocking cache, crew cache, GTFS lock and vehicle store all live in process memory.	The application cannot be scaled horizontally without redesigning all five. The debt is paid the day traffic grows.
No transactions	Closing a waybill reads tickets, then writes totals, with no atomic boundary.	A crash between the two leaves a waybill OPEN with tickets recorded. Recovery is manual.
Duplicated physical assumptions	<code>average_speed_kmph</code> exists in both <code>BlockingParameters</code> and <code>GtfsParameters</code> .	Changing one and not the other makes the published timetable disagree with the fleet plan. The code comments warn about this, but nothing enforces it.
Oversized files	<code>predictor.py</code> is over 1,600 lines; <code>PredictPage.tsx</code> is 861; <code>VotePage.tsx</code> is 697.	Hard to navigate and hard to review. The predictor mixes indexing, ranking, transfer search and benchmarking in one class.
Private methods called across modules	<code>crew_service.py</code> calls <code>blocking_plan_service._get_context()</code> , and <code>waybill_service</code> calls <code>predictor._resolve_stop_name()</code> .	The leading underscore says "internal", so these are informal contracts that can break silently when the private method changes.
Mixed styling approaches	Tailwind utility classes, a hand-written <code>styles.css</code> design system, and large inline style objects all coexist.	Three places to look when changing an appearance, and no single source of truth for spacing or colour.
Advisory-only write endpoints	<code>/depot/deploy-bus</code> and <code>/depot/blocking-decision</code> append to in-memory lists (<code>DEPLOYED_BUSES</code> , <code>BLOCKING_DECISIONS</code>).	Those records vanish on restart. Honest as a prototype - the code says so - but it is state that looks persisted and is not.
No API versioning	Endpoints are unversioned, except the inconsistent <code>/api/tickets</code> prefix.	A breaking change would break every client at once. The inconsistent prefix also makes the API surface look accidental.
Configuration drift risk	<code>VITE_API_URL</code> is baked in at build time.	A rebuilt frontend pointed at the wrong backend fails in a way that looks like a network fault. <code>apiClient.ts</code> exists because this URL was once hard-coded in nine files.

How to talk about technical debt in an interview

Do not present this list apologetically. Presenting it at all is the signal. The strong framing is: "here is what I would fix first and why" - it shows you can evaluate your own work, which is what distinguishes an engineer from someone who only completes tasks. Pick the naming issue and the missing frontend tests as your two examples; both are concrete and both

have an obvious fix.

38. Code Quality Review

Observations are tied to specific code. Generic criticism is deliberately avoided.

38.1 Assessment by dimension

Dimension	Rating	Observation
Naming	Mixed	Domain vocabulary is excellent and correct: block, duty, piece, spreadover, dead kilometres, interlining, revenue service. These are the real terms a transport planner uses. Against that, train() and "AI assistant" actively mislead.
Comments	Strong	Unusually good. Comments explain WHY, not what, and several record the bug that motivated the code - for example the note above TOKEN_TYPE_ACCESS explaining that a ticket token once authenticated as a session. This is the single most valuable quality in the codebase for a new reader.
Structure	Strong	The four-layer split is real and consistently followed. The dependency direction (api -> services -> db; api -> ml -> files) is not violated anywhere examined.
Modularity	Mixed	The tracking and gtfs packages are well decomposed. predictor.py is not - over 1,600 lines mixing indexing, ranking, transfer search and benchmarking in one class.
Coupling	Mostly low	Good: the engines depend on nothing but the dataset. Weak points: cross-module calls to private methods (_get_context, _resolve_stop_name) create informal contracts.
Cohesion	Strong	Each service module owns one subject. core/shakti.py is a good example - one rule, one file, two consumers.
Duplication	Low, and actively reduced	route_geometry.py was extracted precisely because three subsystems had drifted apart doing the same thing. The remaining known duplication is average_speed_kmph in two parameter classes.
Error handling	Strong at the edges, weak at the centre	Excellent per-case handling with meaningful messages and honest degradation. But there is no global handler, so an unexpected exception becomes an opaque 500.
Testing	Strong backend, absent frontend	The backend tests are thoughtful - invariants and plausibility ranges rather than brittle expected values. The frontend has none.
Security practice	Good fundamentals	Hashed passwords, signed tokens, separated key domains, role checks, server-side pricing, no secrets in source. Gaps are known and listed rather than hidden.
Type safety	Good	Python type hints throughout; pydantic at every boundary; TypeScript on the frontend. Some `any` casts remain in PredictPage.
Consistency	Mixed	Backend conventions are consistent. Frontend styling is not - three approaches coexist.

38.2 The strongest quality in this codebase

The comments record decisions and the bugs that caused them. Examples worth quoting:

- **distance.py _minimal_normalize** explains that stripping suffix words collapses "Syndicate Bank" and "Syndicate Bank Layout" - two real places 9 km apart - and that this fabricated bogus distances.
- **blocking.py module docstring** separates what is measured from what is assumed, and states outright: do not quote a fuel or rupee figure from these numbers without replacing the assumptions first.
- **pytest.ini** explains that strict asyncio mode exists because the previous setup skipped async tests while still reporting green.
- **rate_limit.py** explains why /avl/ingest is exempt and notes it is exempt from the budget, not from authentication.

This matters practically: it is why this document could be written with confidence about WHY things are the way they are, not just what they do.

38.3 The weakest quality

Naming that contradicts behaviour, and the absence of any frontend verification. Both are cheap to fix relative to their cost, and both are the first things a reviewer would raise.

39. Final Project Master Map

39.1 The whole system in one tree

```

DBARS
|
+-- USERS (5 roles)
|   +-- commuter      search, vote, e-ticket, crowding, safety share
|   +-- conductor      waybill, onboard sales, pass scanning, cash sign-off
|   +-- depot_manager  fleet blocking, crew duties, scenarios, waybill oversight
|   +-- admin          dashboards, users, Shakti claim, revenue, audit
|   +-- driver         role exists; no interface (declared gap)
|
+-- FRONTEND React 19 + TypeScript + Vite (8,459 lines, 45 files)
|   +-- App.tsx        router, AuthProvider, LanguageProvider, ProtectedRoute
|   +-- pages/         13 screens: Predict, Vote, Tickets, Track, Duty,
|   |                 Scanner, Depot, Admin, Login, Register, Landing, Share
|   +-- components/    RouteMap, BlockingPanel, CrewPanel, ScenarioConsole,
|   |                 MetricsDashboard, AutocompleteInput, SafetyPanel
|   +-- contexts/      AuthContext (token, role), LanguageContext (4 languages)
|   +-- lib/           apiClient.ts (one HTTP wrapper), offlineQueue.ts (money)
|
+-- BACKEND FastAPI + Python 3.12 (13,562 lines, 65 files, 82 endpoints)
|   +-- API layer      app/api/ + app/auth/ HTTP only, no business rules
|   +-- SERVICE layer  app/services/ business rules; ONLY DB access
|   +-- ENGINE layer   app/ml/, app/gtfs/, app/tracking/ NO database at all
|   |   +-- predictor.py route search: 655,438 ordered stop pairs
|   |   +-- blocking.py  fleet size + dead km: 6,849 buses, 44 depots
|   |   +-- crew.py      duties: 15,636 duties, 2.28 crew per bus
|   |   +-- gtfs/        47,295 trips, 1.46M stop_times, ~14 MB
|   |   +-- tracking/    4 feed adapters, is_live gate, route-following ETA
|   +-- DATA layer    app/db/ (Motor) + dataset/ (CSV, JSON, XLSX)
|
+-- DATABASE MongoDB, 17 collections, OPTIONAL
|   +-- users, votes, tickets, transactions, favorites, predictions, notifications
|   +-- waybills, conductor_tickets, travel_passes, audit_events
|   +-- service_alerts, crowd_reports, trip_shares (TTL 6h), usage_events (TTL 90d)
|
+-- EXTERNAL SERVICES (all optional)
|   +-- Google Maps Distance Matrix optional; falls back to coordinates
|   +-- Twilio SMS         refuses everything when unconfigured
|   +-- Fleet AVL feed     adapter written, not connected
|   +-- OpenStreetMap / CARTO tiles map background
|
+-- AI / ML
|   +-- ROUTING ENGINE: no model, no weights, no LLM.
|   |   Classical IR + graph search, benchmarked against 3 baselines.
|   +-- AI LAYER (app/ai): FAISS vector store, BM25 + RRF retrieval,
|   |   8 read-only tools, optional Gemini. Answers questions ABOUT
|   |   the system; never routes a bus. See Sections 40-42.
|
+-- DEPLOYMENT
|   +-- Docker Compose  backend:8000, frontend:8080 (nginx), mongo:27017
|   +-- Render / Vercel alternative targets
|   +-- GitHub Actions  pytest + compileall + frontend build on every push
|   +-- Capacitor       Android APK from the built web bundle

```

Figure 39.1 - The complete project map.

39.2 The entire project in one page

PURPOSE. Turn BMTC's published timetable into three products: a commuter journey planner, an operations planner for depot managers, and an open GTFS data feed.

ARCHITECTURE. Four backend layers with a one-way dependency rule. API handles HTTP; services hold business rules and are the only layer touching MongoDB; engines are pure computation over CSV files; the data layer owns the connection and the files. The engines' independence from the database is the defining decision - it is why route search, fleet planning, crew planning and GTFS export all survive a database outage, and why 161 tests run in CI with no MongoDB.

TECHNOLOGY. Python 3.12, FastAPI, Pydantic, Motor, MongoDB. React 19, TypeScript, Vite, Leaflet, Tailwind. Docker Compose, Nginx, GitHub Actions, Capacitor. No ORM, no JWT library, no ML framework in the serving path.

DATA FLOW. Browser -> apiFetch -> CORS -> rate limiter -> router -> pydantic validation -> auth dependency -> endpoint -> service or engine -> MongoDB or dataset -> JSON -> React state -> UI.

TEN KEY FILES. main.py (entry point); ml/predictor.py (search engine); ml/blocking.py (fleet); ml/crew.py (duties); ml/route_geometry.py (anchored resolution); gtfs/builder.py (open data); tracking/ingest.py (vehicle honesty); auth/auth.py (security); services/waybill_service.py (money); core/shakti.py (the shared scheme rule).

TEN KEY FUNCTIONS. predict(); _interchange_is_walkable(); resolve_stop_sequence(); chain_blocks(); concurrency_lower_bound(); cut_block_into_pieces(); combine_pieces_into_duties(); is_shakti_eligible(); issue_tickets(); estimate_arrival().

DATABASE. MongoDB, 17 collections, no joins and no foreign keys. Critical indexes: users.email unique; conductor_tickets (waybill_id, client_ticket_uuid) unique, which is what makes offline ticketing idempotent; trip_shares TTL 6 hours; audit_events deliberately without a TTL.

API. 82 endpoints across fourteen routers. Public: predict, autocomplete, GTFS, tracking, alerts. Role-gated: all depot planning, conductor money paths, admin reporting. Interactive docs at /docs.

AI. None. The word appears in some UI text and is inaccurate. What exists is an inverted index plus an ordered stop-pair index with hand-weighted scoring, benchmarked against TF-IDF, fuzzy and distance-aware baselines - and the shipped model reports the accuracy of exactly what the API returns rather than a flattering proxy.

SECURITY. PBKDF2-SHA256 passwords with per-user salts; HMAC-SHA256 signed tokens across three separate signing domains; role-based access control; server-side pricing; idempotent money writes; no secrets in source. Known gaps: POST /train unauthenticated, no token revocation, tokens in localStorage, in-memory rate limiting.

DEPLOYMENT. Docker Compose with three services, or Render, or Vercel for the frontend. CI runs tests and builds but does not deploy. Single-process by design - horizontal scaling requires moving shared state to Redis first.

THE GOVERNING PRINCIPLE. Never present modelled or simulated data as measured fact. Assumptions are bundled into parameter objects and echoed in every response; simulated vehicle positions carry an is_live flag that hard-gates the GTFS-Realtime endpoint; the GTFS feed marks every interpolated time and ships a provenance README; and endpoints that cannot answer say so rather than returning a misleading zero.

39.3 Analysis coverage statement

Analysed in depth: main.py, config.py, database.py, auth/auth.py, auth/routes.py, predictor.py, blocking.py, crew.py, distance.py, route_geometry.py, data_loader.py, gtfs/builder.py, gtfs/realtime.py, all five tracking modules, waybill_service.py, pass_service.py, shakti.py, fares.py, rate_limit.py, api/routes.py, api/tickets.py, api/conductor.py, api/gtfs.py, api/tracking.py, api/votes.py, App.tsx, AuthContext.tsx, apiClient.ts, offlineQueue.ts, RouteMap.tsx, and the test suite.

Analysed at a lower level of detail: admin.py, alerts.py, crowding.py, metro.py, safety.py, sms.py and their services; tfidf.py, stop_registry.py, text.py; seed.py; the larger page components (VotePage, AdminDashboard, LiveTracking) beyond their data flow; the Capacitor Android shell; and Docker and deployment files beyond their structure.

Not determinable from the supplied code: actual production memory usage; real-world prediction accuracy against genuine commuter journeys (no ground-truth data exists); test coverage percentage (no coverage tool is configured); and whether the fare table matches current BMTC fares.

40. The AI Intelligence Layer

An assistant sits over the deterministic engine: it answers questions about the codebase, the documentation, and live transit state. The single most important property is that it is additive. The routing engine does not import it, does not call it, and does not depend on it. Being able to say that precisely - and to show the test that proves it - is the point of this section.

40.1 Where it sits

```

HTTP -> app/ai/api/routes.py      authentication + role checks
      -> app/ai/agents/orchestrator LCEL chain, intent routing
          |-- RAG retrieval        code index + documentation index
          |-- domain agents        codebase / operations / travel
          '-- tool registry        8 read-only tools, authorisation chokepoint
              |
              v read-only calls
          DETERMINISTIC CORE (app/ml, app/gtfs, app/tracking, app/services)

```

The arrow into the core points one way. Nothing in app/ai writes to it.

40.2 The dependency rule, and why it is load-bearing

app/main.py imports the AI router inside a try/except. This is not defensive decoration. The AI layer pulls in langchain, faiss-cpu and sentence-transformers; before the guard existed, a missing optional package raised at module scope and took the ENTIRE backend down with it - route prediction, ticketing and GTFS included. A fresh 'pip install -r requirements.txt' produced a server that could not start at all.

```

try:
    from app.ai.api.routes import router as ai_router
except Exception as _ai_import_error:
    ai_router = None

```

Same principle as init_db(): an optional subsystem may fail, but it may never prevent DBARS from starting.

Evidence: Verified by blocking the AI packages at import time: the app started with 82 routes instead of 93, logged a warning, and every deterministic feature behaved identically.

40.3 Retrieval-Augmented Generation

RAG is the part of this layer most worth being able to defend, because it is where the interesting design decisions are. The shape is standard - retrieve, then ground an answer in what was retrieved - but three choices here are worth explaining.

40.3.1 The pipeline

```

question
|
+--> QueryRouter          code / documentation / operations / travel / hybrid
+--> expand_query()       plain words -> the identifiers the code uses
|
+--> VECTOR search       FAISS IndexFlatIP, 384-d, cosine on normalised vectors
+--> LEXICAL search       BM25, k1=1.5, b=0.75
|
+--> Reciprocal Rank Fusion  score(d) = SUM 1 / (60 + rank_i(d))
|

```

```

+> Citations      file, symbol, start_line, end_line, snippet
+> grounding_verifier labels CONFIRMED / INFERRED / UNKNOWN

```

Two indices are built: code (1,560 chunks over 235 files) and documentation (262 chunks over 20 files).

40.3.2 Chunking is symbol-aware, not fixed-width

Python is parsed with the ast module and emits one chunk per meaningful unit - a module chunk carrying the docstring and imports, a class-overview chunk, and one chunk per method and top-level function. Each carries file, symbol, type, start_line and end_line.

This is why a citation reads predictor.py:412-455 with a function name rather than 'chunk 87', and why a question about a function retrieves that function instead of a 600-character window straddling two unrelated definitions. TypeScript is chunked by declaration with a regex parser, because no TS AST is available in-process.

40.3.3 Why the retrieval is hybrid

Half	What it contributes	Where it fails alone
Vector (FAISS)	Tolerates paraphrase and word order	Cannot match an exact symbol it has not seen
BM25 lexical	Exact identifiers: require_role, BMTCTBusPredictor, route_distance	Blind to a question that shares no token with the code

Reciprocal Rank Fusion combines them on RANK rather than score, so the two incomparable scales never need calibrating and a document ranked highly by either half surfaces. $k = 60$ is the standard constant from the original RRF paper.

40.3.4 The embedding decision, and the vocabulary gap

The default embedding provider is deterministic hashed n-grams, not a neural model. That is a deliberate choice backed by measurement, and section 42.3 gives the numbers - swapping in all-MiniLM-L6-v2 produced no improvement at 80x the load time.

State this limitation before an examiner finds it

Hashed n-grams are a LEXICAL signal. 'How is the fare computed' and 'where is pricing calculated' do not land near each other the way true sentence embeddings would, so the dense half behaves like a fuzzy keyword index. Calling it semantic search would be an overstatement.

The gap is bridged by query expansion: about twenty mappings from plain language to the identifiers this codebase actually uses - 'travelling time' adds duration_minutes and route_distance, 'fare' adds price_ticket. It was added after a concrete failure: the question 'how is travelling time calculated' returned README.md, a translations JSON and a planning document, while distance.py - which computes it, 23 chunks of it - sat unranked in the same index. That is a hand-written vocabulary, not a learned one, and a question phrased outside it can still under-retrieve.

40.3.5 Grounding

Retrieval alone does not stop a wrong answer. The same question above was originally answered 'Execution enters through README.md' with a line range presented as a function, and reported as CONFIRMED. The answer template took the top citation, whatever it was, and asserted a call path through it.

- **Source files are preferred over prose and configuration**, and implementation files over tests - a test exercising a feature is not an answer to where it is implemented.
- **Line-range pseudo-symbols are never presented as functions.** README.md:L1-L60 is a chunk address produced by the generic chunker, not a definition.
- **An execution flow is described only from a named symbol.** When only documentation matches, the answer says so and carries a limitation instead of narrating a call path.
- **verify_function_exists() checks a symbol against the indexed AST** before the assistant will discuss it, so a question about a function that does not exist returns UNKNOWN.

Evidence: 23 regression tests in tests/test_codebase_answer_grounding.py, including a reconstruction of the documentation-only case that asserts the answer admits it.

40.4 Security: what was wrong, and what it is now

The layer shipped as a working authentication bypass

No /ai/* endpoint had any authentication. Every 'require_role' occurrence in app/ai was a string inside a prompt or a regex, not an enforced check. Confirmed against a running server: GET /depot/blocking-plan returned 401 to an anonymous caller, while POST /ai/chat answered the same question with the full depot fleet figures and a 200.

An assistant that reaches the same services the REST API reaches must apply the same role checks, or it is simply a side door around them. The fix:

- **Identity comes from the token, never the body.** A client-supplied user_id could otherwise claim to be anyone - which is exactly how session history leaked.
- **Tools are gated at one chokepoint.** ToolRegistry.execute_tool() is the single path every agent and the orchestrator go through, so a new call site cannot forget the check. It fails closed: with no principal bound, a restricted tool is denied.
- **Session memory is keyed user_id::session_id.** Both identifiers used to come from the client, so anyone supplying a matching pair read that user's conversation and journey context - origin and destination included.
- **Observability and evaluation are admin-only.** Traces carry other users' question text, the buffer reset is destructive, and a benchmark run is a denial-of-service lever.

Anonymous request	Before	After
POST /ai/chat	200 + depot data	401
GET /ai/session/{id}/history	200 + victim's journey	401
DELETE /ai/observability/traces	200 (destructive)	401
POST /ai/evaluation/run	200 (expensive)	401
GET /ai/health	200	200 (public by design)
POST /predict	200	200 (unchanged)

Evidence: tests/test_ai_authorization.py holds 19 regression tests, including the 401-versus-200 pair above and the session IDOR.

40.5 The read-only contract

Eight tools, all reads. None writes, deletes or mutates a record, ticket, waybill, fleet plan, user role or configuration value. The model cannot execute shell commands, evaluate Python, write files, issue SQL or make outbound HTTP requests - no such tool is registered, and the model reaches DBARS only through the registry.

Tool	Returns	Required role
search_bus_route	Journey options between two stops	any authenticated
get_route_details	Stop list, terminals, frequencies	any authenticated
get_fleet_plan	Minimum fleet, interlining, depots	depot_manager / admin
get_crew_plan	Daily duties, crew-to-bus ratio	depot_manager / admin
get_crowding_information	Recent commuter crowd reports	any authenticated
get_service_alerts	Active disruptions	any authenticated
get_metro_information	Nearby Metro stations	any authenticated
get_bus_eta	Vehicle positions, schedule status	any authenticated

Say this precisely in a viva

is_read_only is a declared attribute, not an enforced sandbox. No registered tool writes - that was verified by reading every tool's call path - but nothing mechanically prevents a future one from doing so. The role restrictions ARE enforced, at the registry chokepoint. Claiming more than that is the kind of overstatement an examiner will find.

41. Defects the AI Layer Exposed in the Routing Engine

Building an assistant that answers the same questions as the search page surfaced several defects in the deterministic engine itself. These are worth rehearsing because each one is a good story about measurement: a user-visible symptom, a root cause in a specific line, and a number that moved.

41.1 Stop resolution compared spelling, not place

Two separate failures shared one cause - the matcher treated a NAME difference as a PLACE difference.

Query	Returned	Why
dasarahalli metro station	Vajarahalli Metro Station, the opposite end of the Green Line	Shared generic token 'metro' plus a similar ending outsourced an exact match on the distinctive word
White Field Post Office -> Kengeri	86 stops, 85.9 km instead of 33 stops, 41.4 km	Alighting at 'Kengeri Bus Station' counted as inexact - it is 0.22 km from 'Kengeri'

The first was fixed by keying on whether the network actually uses a word, and how often: no character-similarity threshold separates 'dasarahalli' from 'vajarahalli' (0.82) from a genuine typo like 'majestc'/'majestic' (0.93). The second by measuring physical distance - if a commuter could walk it, it is the stop they asked for.

Evidence: 23 regression tests in tests/test_stop_resolution.py; rank_quality unchanged because the benchmark uses canonical stop names while the fix governs free text.

41.2 Trunk preference outranked journey length

route_rank_score ordered trunk_tier ABOVE total_stops, and route 253 matches the trunk prefix list. The existing length penalty could not counter it, because _min_span_for_pair is derived from DIRECT segments only - on a transfer-only pair it stays None and the penalty is always zero, leaving trunk status to decide unopposed.

```
(stop_exactness, transfers, special, span_band, trunk_tier, total_stops, ...)
          ^^^^^^^^^      ^^^^^^^^^
```

length band inserted ABOVE trunk status, banded against the
shortest WALKABLE candidate - a plain min() picks up
name-collision chains and makes every band one stop wide

Trunk routes still win among journeys of comparable length; they no longer win by riding materially further.

41.3 Stop count is not distance

Ranking runs before any leg is built, so it uses total_stops as the length signal. The proxy fails when stop spacing differs: Shivanapura -> Avalahalli BDA Layout was answered with a 78-stop, 146.3 km journey over an 85-stop, 62.5 km one, because 78 < 85 while 146 km is more than twice as far. A post-build pass now demotes a candidate more than 1.6x the shortest of its transfer tier, using real distances the ranker could not see.

41.4 The search could not see past one interchange

Sapthagiri College -> Reva College are both in north Bengaluru, but no single interchange links them along the northern arc, so a direct + one-transfer search could only reach Reva by coming into the city and back out. Google Maps answers the same pair with three buses. Every one of those routes is in this dataset - 248-BA serves Sapthagiri, 289-S serves Reva, and 401-A serves NEITHER. It is purely a middle leg, which no amount of one-transfer searching can find.

A bounded meet-in-the-middle search adds the third leg: collect where the first bus can drop you, where the last can collect you, and examine only routes touching both. Measured over 60 random pairs, it made 16 pairs routable that previously had no answer at all, and offered a materially shorter option on 15 more.

41.5 Confidence contradicted the ranking

29.8% of results recommended the option they scored lower

Ranking and confidence were two unrelated formulas. Ranking used exactness, transfers, frequency tier, span and trunk status; confidence used transfers, stop count and frequency alone. `_enrich_transfer_distances` was the only code that would have reconciled them, and nothing called it.

The score now includes the ranker's signals, weighted in its order of priority, and the sequence is then forced non-increasing. Be precise about which half does the work: the formula orders correctly on its own in 76% of multi-option results; the clamp guarantees the remaining 24% cannot display a contradiction either. A lexicographic tuple cannot be collapsed into one scalar without loss.

41.6 Performance

Measurement	Before	After
<code>_detect_metro_interchange</code> share of a route search	65%	cached
Route search, median	212 ms	39 ms
Route search, p90	901 ms	406 ms
First depot question on a cold process	10-15 s	77 ms
Copilot journey request	1,932 ms	~550 ms

The metro scan re-tokenised all 85 stations for every stop of every leg built - 621k token lookups per query. The depot figure was a missing warm-up: the lifespan warmed the blocking plan and the GTFS feed but not the crew plan, so the first question paid 5.6 s rebuilding a cached context plus 7.6 s scheduling crew.

42. What Was Measured, and What Was Not

This section exists so that nothing in this document is defended further than the evidence supports. An examiner who finds an overstatement will discount everything else; leading with the limits is safer and more honest.

42.1 The benchmark, read correctly

Metric	Value
Questions	57 across 12 categories
Pass rate	100% (57/57)
Mean answer correctness (headline)	81.5%
Hallucination rate	0.00%
Mean latency	635 ms

A pass needs only 40% keyword coverage

'100% pass rate' means every answer cleared a floor, not that every answer was complete - eight questions passed sitting exactly on it. Mean answer correctness is the number that can actually move, which is why it is the headline. Weakest categories: debugging (0.500) and documentation (0.662).

An earlier version of this report claimed 'Overall Accuracy 100.0%', 'all targets PASSED' and 'zero metrics are fabricated' - over 2 questions in 1 category, while 57 across 12 were defined in the repository. It had been generated with limit=2. That is the kind of number that collapses under one question in a viva.

42.2 The 0% hallucination rate is architectural

Without GEMINI_API_KEY the layer uses a grounded deterministic model that assembles answers from retrieved evidence and tool output. It structurally cannot invent a fact absent from the evidence. That is why the rate is zero - not because a generative model was tested and found truthful.

More pointedly: with a real key configured and Gemini reachable, only 1 of 57 benchmark questions actually invoked the model - 1.8%. The orchestrator is an intent router into hand-written agents; the LCEL chain is the fallback branch for queries no agent claims. Swapping the deterministic model for live Gemini moved mean answer correctness from 0.8150 to 0.8146. Describing the system as 'LLM-powered' would substantially overstate the model's role.

42.3 Semantic embeddings were tried and did not help

The obvious upgrade to a lexical index is real sentence embeddings. It was measured, not assumed, on identical content:

Provider	Alone	With query expansion	Cost
Hashed n-grams	5/10	8/10	0.15 s, 18 MB
all-MiniLM-L6-v2	4/10	8/10	12.07 s, 450 MB

Identical with expansion and slightly worse without it, for 80x the load time and 25x the memory. all-MiniLM-L6-v2 is trained on natural-language sentences, not source code; chunks of identifiers and type annotations are out of distribution for it, while the BM25 half of the hybrid already caught the literal tokens these questions carry. The deterministic provider is the default. A code-trained embedding model is the thing worth trying next.

42.4 Remaining limits

- **Prompt-injection defence is signature-based.** It stops the phrasings it enumerates; it is not a proof against a novel paraphrase. The controls that actually bound the blast radius are the read-only tool contract and the role checks.
- **Query expansion is a hand-written vocabulary.** About 20 mappings bridging plain language to identifiers. A question phrased outside them can still under-retrieve.
- **Journey times are estimates.** Route distance plus a flat waiting allowance per change. The dataset has trip counts but no published per-band frequency, so the allowance is an assumption - and the UI says so rather than presenting it as live data.
- **The two-transfer search is a heuristic.** Meet-in-the-middle with pruning, not exhaustive. It finds good three-bus options, not provably optimal ones.
- **Detour ratios reflect the network, not only the ranker.** Across 244 routed pairs the median journey is 1.80x the straight-line distance and 34 exceed 2.5x. Of those, ZERO had a materially shorter option already in the candidate set - the rest are the bus network as it is.

42.5 Verification summary

Check	Result
Backend test suite	430 passing
rank_quality.best_option_rate	0.7167 (unchanged through every fix)
rank_quality.ndcg@5	0.9810 -> 0.9820
/predict with the AI layer vs blocked	byte-identical output
Protected engine files modified	one, on explicit instruction (41.1)
Secrets in the code index	none (1,592 chunks scanned)

Appendix A - Project File Tree

234 files are tracked in version control. Build outputs, dependencies and .env files are excluded by .gitignore. The full tree is given in Section 6; this appendix lists file counts by area.

Area	Files	Lines	Notes
backend/app/	65 .py	13,562	API, services, engines, models, auth, core, db
backend/tests/	15 .py	2,440	161 tests
frontend/src/	45 .ts/.tsx	8,459	13 pages, 17 components, contexts, lib
dataset/	8 files	n/a	~12 MB of CSV, JSON, XLSX and PDF
frontend/android/	~90 files	n/a	Capacitor Android shell

deployment/, .github/	3 files	n/a	nginx.conf, render.yaml, ci.yml
docs/	3 files	n/a	implementation plan, resume entry, this document

Appendix B - API Reference (quick card)

The complete reference is in Section 13. This is the one-page version for revision.

PUBLIC

```
POST /predict          find the bus between two stops
GET /autocomplete?q=   stop name suggestions
GET /routes /metrics /health catalogue, benchmarks, liveness
GET /gtfs/static.zip    the full GTFS feed (~14 MB)
GET /gtfs/feed-info     build report and assumptions
GET /gtfs-rt/service-alerts.pb realtime alerts (real data)
GET /gtfs-rt/vehicle-positions.pb 503 while the feed is simulated
GET /tracking/buses /bus/{n} /eta /source
GET /alerts /metro/stations /metro/nearest
GET /safety/trip/{code} shared trip, no login by design
POST /conductor/verify-pass offline signature + expiry check
```

AUTH

```
POST /auth/register /auth/login /auth/refresh GET /auth/me
```

USER (any logged-in account)

```
POST /votes          GET /votes/my /allocation /aggregate
POST /api/tickets/purchase GET /api/tickets/active /history /fare
POST /crowding/report GET /crowding/route/{n}
POST /safety/share-trip GET/PUT /safety/contacts
GET/POST/DELETE /tracking/favorites
```

CONDUCTOR

```
POST /conductor/sign-on /tickets/issue /tickets/sync /sign-off
GET /conductor/waybill /conductor/pass-revocations
```

DEPOT MANAGER / ADMIN

```
GET /depot/blocking-plan /depot/crew-plan /depot/routes /depot/waybills
POST /depot/blocking-scenario /depot/crew-scenario /depot/blocking-decision
GET /depot/dispatch-recommendations POST /depot/deploy-bus
POST /depot/passes POST /depot/passes/{id}/revoke
GET /admin/shakti/claim /admin/revenue/reconciliation
```

ADMIN ONLY

```
GET /admin/dashboard /admin/users /admin/votes/analytics
GET /admin/fraud/alerts /admin/pilot-metrics /admin/audit
POST /admin/users PUT /admin/users/{id}/toggle-active
POST /gtfs/rebuild
```

MACHINE-TO-MACHINE

```
POST /avl/ingest      X-AVL-Key header; exempt from rate limiting
POST /sms/webhook     X-Twilio-Signature verified
```

UNGATED AND SHOULD NOT BE

```
POST /train          expensive retrain, no authentication <-- known issue
```

Appendix C - Database Schema

MongoDB has no enforced schema; the shapes below are what the code writes.

```

users {
  _id, email (UNIQUE), name, password_hash, role, preferred_lang,
  gender, reliability, is_active, created_at
}

votes {
  _id, user_id, current_stop, destination, time_preference,
  flagged, reason, ip_address, created_at
}
index (current_stop, destination) | (created_at desc) | user_id

tickets {
  ticket_id, user_id, source_stop, destination_stop,
  fare_amount,          <- what the passenger paid (0 for Shakti)
  fare_value_inr,        <- what it would have cost (the claim)
  fare_collected_inr, is_zero_fare, scheme,
  issue_time, expiry_time, status, qr_token, short_code
}

waybills {
  waybill_id (UNIQUE), conductor_id, driver_id, bus_reg, depot,
  route_number, direction_id, duty_date, status (OPEN|CLOSED|RECONCILED),
  sign_on_at, sign_off_at, opening_km, closing_km,
  totals { tickets_issued, passengers, cash_amount, digital_amount,
            pass_boardings, shakti_boardings, shakti_claim_value,
            fare_value_total, fare_collected_total },
  reconciliation { declared_cash, expected_cash, variance,
                  variance_flagged, note, remitted_at }
}
index (conductor_id, status) | (depot, duty_date desc) | waybill_id UNIQUE

conductor_tickets {
  ticket_id, client_ticket_uuid, waybill_id, conductor_id, route_number,
  from_stop, to_stop, is_ac, payment_mode, passengers,
  lines[ { passenger_type, count, unit_fare_value_inr, fare_value_inr,
            fare_collected_inr, is_zero_fare, scheme } ],
  fare_value_inr, fare_collected_inr, shakti_boardings,
  shakti_claim_value, issued_at, synced_at
}
index (waybill_id, client_ticket_uuid) UNIQUE <-- idempotency

travel_passes { pass_id (UNIQUE), holder_name, pass_type, route_permission,
                user_id, issued_by, issued_at, expires_at, status,
                revoked_at, revoked_reason }

audit_events { action, actor_id, subject, metadata, created_at } NO TTL
usage_events { event_type, metadata, created_at } TTL 90 days
trip_shares { code (UNIQUE), user_id, trip data, expires_at } TTL 0 (at expiry)
service_alerts { title, description, severity, affected_routes,
                 affected_stops, status, created_by, created_at, resolved_at }
crowd_reports { user_id, route_number, crowding_level, stop_name, created_at }
favorites { user_id, current_stop, destination, label } UNIQUE triple
bus_positions { bus_number, route_id, latitude, longitude, speed_kmh,
                occupancy_pct, heading, next_stop, updated_at }

```

```
transactions { transaction_id, ticket_id, user_id, amount, status,  
               scheme, timestamp }
```

Appendix D - Environment Variables

No real secret values appear here

All secrets are shown as [REDACTED SECRET]. The repository contains none: .env files are gitignored, and only .env.example files with placeholders are committed.

```
# ---- Datasets (all required; the app refuses to start if any is missing) ----
DATASET_PATH=../dataset/routes_cleaned.csv
STOP_COORDINATES_PATH=../dataset/stops_cleaned.csv
METRO_DATASET_PATH=../dataset/bengaluru_metro_network.csv
FARES_DATASET_PATH=../dataset/fares.json
DEPOT_DATASET_PATH=../dataset/BMTC_depot_place_zone.xlsx
ARTIFACT_DIR=./artifacts

# ---- Database (optional; the app degrades gracefully without it) ----
MONGO_URI=mongodb://localhost:27017      # no credentials in the default
MONGO_DB_NAME=bmtc
MONGO_SERVER_SELECTION_TIMEOUT_MS=3000    # fail fast rather than hang 30 s

# ---- Secrets ----
JWT_SECRET_KEY=[REDACTED SECRET]          # unset -> random per process + warning
TICKET_SIGNING_KEY=[REDACTED SECRET]       # unset -> derived from JWT_SECRET_KEY
PASS_SIGNING_KEY=[REDACTED SECRET]         # unset -> derived from JWT_SECRET_KEY
PASS_VERIFICATION_SECRET=[REDACTED SECRET]
GOOGLE_MAPS_API_KEY=                      # empty -> feature skipped entirely
TWILIO_AUTH_TOKEN=                        # unset -> the webhook refuses everything
AVL_INGEST_KEY=                            # unset -> /avl/ingest refuses everything

# ---- Network and limits ----
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
RATE_LIMIT_PER_MINUTE=120
TRUST_PROXY_HEADERS=false                  # true only behind ONE trusted proxy

# ---- Vehicle tracking ----
TRACKING_FEED=simulated                    # simulated|gtfs_rt|http_json|push
TRACKING_FEED_URL=
TRACKING_FEED_FIELD_MAP=                   # JSON map for a bespoke ITS API
TRACKING_POLL_SECONDS=10
AVL_STALE_SECONDS=120                     # older positions are withheld

# ---- GTFS ----
GTFS_BUILD_ON_STARTUP=true
GTFS_RT_ALLOW_SIMULATED=false              # dev only; serves a separate .sim.pb path
GTFS_FEED_PREFIX=dbars                    # namespaces every id
GTFS_PUBLISHER_NAME=DBARS (unofficial, derived from published BMTC data)

# ---- Demo and seeding ----
DEMO_PASS_MODE=false                      # accepts unsigned pass ids; labels them
SEED_DEFAULT_ADMIN=false
DEFAULT_ADMIN_PASSWORD=admin123            # refused when ENV=production

# ---- Frontend (BUILD time, not runtime) ----
VITE_API_URL=http://localhost:8000
```

Appendix E - Important Code Snippets

The five excerpts most worth being able to recognise and explain.

E.1 The startup ordering (main.py)

```
app.state.predictor.train()          # engines ready FIRST
...
await init_db()                     # cannot raise; logs and continues
if not db_available():
    logger.warning("Starting without MongoDB: ... Route prediction is unaffected.")
```

E.2 Anchored stop resolution (route_geometry.py)

```
centre = (sorted(p[0] for p in known)[len(known)//2],      # median, not mean
          sorted(p[1] for p in known)[len(known)//2])
previous = centre
for stop in stops:
    record = distance_service.resolve_stop_record(stop, near=previous)
    resolved.append(record)
    if record is not None:
        previous = record.point
```

E.3 Idempotent offline sync (waybill_service.py)

```
try:
    await db.conductor_tickets.insert_one(document)
except DuplicateKeyError:
    duplicates.append({"client_ticket_uuid": ticket.client_ticket_uuid})
    continue
# index: (waybill_id, client_ticket_uuid) UNIQUE -> the DATABASE enforces this
```

E.4 The honesty gate (api/gtfs.py)

```
if not report["is_live"] and not allow_simulated:
    raise HTTPException(503, detail=(
        "Vehicle positions are currently produced by a simulation, not a BMTC "
        "vehicle feed, and are therefore not published as GTFS-realtime."))
```

E.5 Separating value from collection (core/shakti.py + tickets.py)

```
free_travel, reason = is_shakti_eligible(gender, is_ac=payload.is_ac)
fare_value = round(float(fare), 2)          # what the STATE reimburses
fare_collected = 0.0 if free_travel else fare_value # what the PASSENGER paid
# One combined fare_amount field cannot express both.
```

Appendix F - Glossary

The full glossary is Section 31. The ten terms most likely to be asked about:

Term	One-line answer
Block	One physical bus's day: pull out, chain of trips, pull in
Duty	One person's working day - distinct from a block
Dead kilometres	Distance run with no passengers aboard
Interlining	Letting one bus serve trips from different route numbers
Spreadover	Elapsed time from sign-on to final sign-off, including unpaid gaps
GTFS	The worldwide standard format for publishing transit timetables
Idempotent	Doing it twice has the same effect as doing it once
Anchored resolution	Resolving each stop relative to the previous one, so a route cannot jump between same-named places
Shakti scheme	Karnataka's free bus travel for women and transgender passengers on non-AC services
is_live	The flag on every vehicle observation stating whether the data is real; it gates the GTFS-Realtime endpoint

Appendix G - Interview Questions

Sections 27 to 29 contain roughly 130 questions with answers. The structure is:

Section	Contains
27.1 - 27.9	Professor defense: basic, architecture, code, database, security, AI, failure, scalability and critical questions, with spoken-length answers
28.1 - 28.8	Recruiter and interview: the project pitch, architecture, hardest problem, technology choices, feature deep dive, scaling, debugging, hindsight
29.1 - 29.6	95 rapid-fire questions with one-line answers, grouped by area
30.1 - 30.5	The same project explained in 30 seconds, 1, 3, 5 and 10 minutes

The three questions most likely to catch you out

1. "Where is the AI?" - There is none; say so plainly and describe the retrieval system instead. 2. "How accurate is it?" - 100% validity (relevance-set), 71.7% rank quality, 19.2% single-label; defend the measurement methodology. 3. "Is the tracking real?" - No, it is simulated by default, and the code is built so it cannot be published as real.

Appendix H - Identified Issues

Every finding raised in this document, in one list, with severity. Confirmed means visible in the code.

#	Issue	Severity	Status	Section
1	POST /train has no authentication and triggers a full retrain	High	CONFIRMED	13.1, 20.2
2	Zero frontend tests across 8,459 lines, including the offline money queue	High	CONFIRMED	21.3, 36.1
3	No global exception handler, no request ids, no metrics or tracing	High	CONFIRMED	18.2, 36.1
4	In-memory rate limiter and caches prevent running more than one worker	High	CONFIRMED	22.3, 36.1
5	No token revocation; logout is client-side only	Medium	CONFIRMED	15.5
6	Tokens stored in localStorage (XSS exposure)	Medium	CONFIRMED	15.5, 20.2
7	No multi-document transactions; a crash mid-sign-off leaves a waybill inconsistent	Medium	CONFIRMED	14.6, 37
8	Misleading naming: train() trains nothing; the AI assistant is a string parser	Medium	CONFIRMED	16.1, 37
9	average_speed_kmph duplicated in two parameter classes that must stay in step	Medium	CONFIRMED	37
10	Weak password policy (min 6) and no email verification	Medium	CONFIRMED	15.5, 20.2
11	No login-specific throttling or account lockout	Medium	CONFIRMED	20.2
12	1.6 MB frontend bundle with no code splitting	Medium	CONFIRMED	23.3, 36.1
13	Cross-module calls to private methods (_get_context, _resolve_stop_name)	Low	CONFIRMED	37
14	predictor.py exceeds 1,600 lines and mixes four responsibilities	Low	CONFIRMED	38.1
15	@vis.gl/react-google-maps declared but never used	Low	CONFIRMED	2.2, 23.2
16	numpy, scipy and scikit-learn shipped in production for a benchmark only	Low	CONFIRMED	23.3
17	Three copies of the metro map PDF (~8.6 MB)	Low	CONFIRMED	36.1
18	driver role exists with no interface or endpoints	Low	CONFIRMED	1.5
19	Advisory endpoints write to in-memory lists that vanish on restart	Low	CONFIRMED	37
20	No API versioning; inconsistent /api prefix	Low	CONFIRMED	37
21	Three coexisting frontend styling approaches	Low	CONFIRMED	37, 38.1
22	GTFS calendar declares daily service, overstating Sundays (documented in the feed)	Low	CONFIRMED	36.2
23	Memory footprint never measured	Unknown	UNCERTAIN	22.3, 36.2
24	Test coverage percentage unknown - no tool configured	Unknown	UNCERTAIN	21.3

25	Fare table currency versus current BMTC fares	Unknown	UNCERTAIN	36.2
----	---	---------	-----------	------

Closing note

This document was written from the source code of the repository, not from its documentation. Where the code and the README disagreed, the code was treated as the source of truth. Where something could not be established, it is marked UNCERTAIN or stated as not determinable. Nothing in this document was invented to fill a gap.